

Politecnico di Torino

Master's Degree in Mechatronics Engineering

Academic Year 2025/2026

Graduation Session December 2026

Vision-Based Row Following Navigation for Horticultural Robots with Automatically Labeled Semantic Segmentation

Supervisors:

Marcello Chiaberge

Alessandro Navone

Umberto Albertin

Candidate:

Emanuele Pruiti

Abstract

Precision agriculture aims to increase crop yield and the efficiency of field operations while reducing waste. At the same time, the growing world population increases the demand for agricultural production, while the shortage of skilled labor and the rising cost of farm management push agriculture towards autonomous robots. Row-based crops are among the most studied applications of these robots.

Navigating between crop rows requires precise positioning, because the robot must stay within the corridor without damaging the plants. Satellite positioning alone does not guarantee this precision, since dense canopies and greenhouse structures degrade the GNSS signal and its accuracy. In addition, RTK corrections and 3D LiDAR sensors are expensive, which limits the large-scale adoption of agricultural robots. Camera-based methods are cheaper, but classical pipelines based on color thresholds are sensitive to illumination changes and weeds. Segmentation networks handle these conditions better, but they need large datasets annotated manually at pixel level, which take a long time to produce, and they must run fast on onboard hardware.

In this thesis, we propose a low-cost solution based on a monocular RGB camera and composed of two stages: segmentation and path extraction. In the first stage, a semantic segmentation network predicts the path between the rows. In the second stage, a path extractor estimates from the predicted mask the centerline of this path, which the robot follows to move along the row. The extractor divides the region of interest into horizontal strips, associates the corridor intervals from the bottom of the image upwards, and fits the left and right edges separately with iteratively reweighted least squares, so that short gaps and local errors in the predicted mask have a limited effect on the result. The centerline is the mean of the two edge lines.

The FCN, DeepLabV3, and LR-ASPP deep learning architectures were compared on a manually annotated dataset of lettuce images. LR-ASPP was selected for its speed: its accuracy is about 1% lower than that of the other two networks, but its inference is 7 to 11 times faster, providing lightweight execution on the onboard computer.

To reduce the annotation effort, we investigate whether manual annotation

can be replaced by labels generated automatically by SAM 3 from text prompts, without manual correction. In operating conditions, on the images taken between parallel rows, networks trained on SAM 3 labels reach a path Intersection over Union about 6% lower than the network trained on manual labels. Since the path is the only class used for navigation, the robot uses a binary network trained on the SAM 3 labels of lettuce, tomato, and cucumber crops, so the tomato and cucumber images required no manual annotation.

The images were collected in two experimental campaigns, and a third campaign was dedicated to the tests on the robot. The proposed navigation method was compared with three state-of-the-art solutions. The tests showed that the robot can autonomously navigate in horticultural rows with a monocular RGB camera and a segmentation neural network trained using only automatic labels without manual correction. These results represent a possible robust solution for precise motion inside the crop rows and a lower risk of damaging the crops, paving the way for further horticultural operations.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures VIII

1 Introduction 1
- 1.1 Context and motivation . . . 1
- 1.2 Objectives of the thesis . . . 2
- 1.3 Organization of the work . . . 3

2 Background and Problem Formulation 4
- 2.1 Agricultural autonomous navigation . . . 4
  - 2.1.1 Navigation tasks and requirements . . . 4
  - 2.1.2 Environmental challenges . . . 4
- 2.2 Vision-based local guidance . . . 5
  - 2.2.1 Sensing modalities . . . 5
  - 2.2.2 Image-space reference extraction . . . 5
- 2.3 The segmentation-to-control pipeline . . . 7
  - 2.3.1 Perception stage . . . 7
  - 2.3.2 Geometric extraction stage . . . 7
  - 2.3.3 Control integration . . . 8

3 State of the Art 9
- 3.1 Classical image-processing methods . . . 9
- 3.2 Learning-based segmentation for agricultural navigation . . . 9
  - 3.2.1 Semantic segmentation approaches . . . 9
  - 3.2.2 Training data and generalization . . . 10
- 3.3 Semantic segmentation architectures . . . 10
  - 3.3.1 FCN, DeepLabV3, and mobile architectures . . . 10
  - 3.3.2 Transfer learning and pretraining . . . 10
- 3.4 Dataset preparation and model-assisted annotation . . . 11
  - 3.4.1 Manual annotation challenges . . . 11
  - 3.4.2 SAM and foundation models for segmentation . . . 11
- 3.5 Geometric extraction methods . . . 12

3.5.1 Line fitting and crop-edge detection . . . 12
3.5.2 Column-wise histogram methods . . . 12
3.5.3 Horizontal strips and adaptive multi-ROI methods . . . 12
3.5.4 Triangle Scan Method . . . 13
3.5.5 Bilateral boundary tracking versus centroid reduction . . . 13
3.5.6 Comparison of geometric representations . . . 15
3.6 System integration and experimental validation . . . 16
3.6.1 Perception and control integration . . . 16
3.6.2 Scope of reported evaluations . . . 16
3.7 Positioning of this thesis . . . 16

4 Deep Learning Architectures for Semantic Segmentation 18
4.1 Problem formulation and encoder-decoder principles . . . 19
4.2 Fully Convolutional Networks (FCN) . . . 20
4.2.1 Convolutionalization of classification networks . . . 21
4.2.2 Skip architecture and upsampling . . . 21
4.3 DeepLabV3 and atrous convolutions . . . 22
4.3.1 Atrous convolution formulation . . . 23
4.3.2 Atrous Spatial Pyramid Pooling (ASPP) . . . 23
4.4 MobileNetV3 and the Lite Reduced ASPP (LR-ASPP) head . . . 24
4.4.1 Depthwise separable convolutions . . . 25
4.4.2 Inverted residual blocks with linear bottlenecks . . . 25
4.4.3 Squeeze-and-Excitation (SE) channel attention . . . 26
4.4.4 Efficient non-linearities: hard-swish and hard-sigmoid . . . 27
4.4.5 The Lite Reduced ASPP (LR-ASPP) architecture . . . 27
4.5 Segment Anything Model 3 (SAM 3) for dataset annotation . . . 28
4.5.1 Architecture overview . . . 30
4.5.2 Operational distinction from onboard models . . . 31
4.6 Summary of architectural characteristics . . . 31

5 Dataset Preparation and Comparative Model Training 33
5.1 Image acquisition and class definition . . . 34
5.1.1 Image sources and acquisition conditions . . . 34
5.1.2 Semantic classes and labeling targets . . . 34
5.2 Dataset preparation . . . 35
5.2.1 Lettuce dataset with manual annotation . . . 35
5.2.2 Automated zero-shot annotation via SAM 3 . . . 35
5.2.3 Tomato and cucumber images . . . 36
5.2.4 Dataset partitioning . . . 37
5.3 Architecture comparison: FCN, DeepLabV3, and LR-ASPP . . . 37
5.3.1 Model selection and pretrained weights . . . 38

5.3.2 Training configuration and loss formulation 38
5.3.3 Evaluation metrics and results 40
5.3.4 Selection of LR-ASPP 42
5.4 Manual versus SAM 3 labels 42
5.4.1 Training configurations and evaluation 42
5.4.2 Results on the lettuce test images 43
5.4.3 Extension to the multi-crop dataset and deployed network . 45

6 Extraction of the Local Navigation Reference 46
6.1 Input mask and line models 46
6.2 Strip-IRLS 47
6.2.1 Region of interest and mask cleaning 47
6.2.2 Horizontal strips and intervals 48
6.2.3 Bottom-up association and chain selection 49
6.2.4 Separate fitting of the corridor edges 51
6.2.5 Validation, confidence, and output 54
6.2.6 Setting the parameters 57
6.3 EdgeNet, an edge regression network 60
6.4 Baseline extractors 62
6.4.1 Center points 62
6.4.2 Column histogram 63
6.4.3 RANSAC edges 63
6.5 Structural differences and limitations 64

7 Robot Platform and Integration 66
7.1 Robot hardware and sensors 66
7.1.1 Robot platform 66
7.1.2 Camera configuration 68
7.1.3 Computing platform 68
7.2 Software architecture 68
7.2.1 ROS integration 68
7.2.2 Control interface 69
7.2.3 Controller node 69
7.3 Deployment and testing 76
7.3.1 Timing and latency 76
7.3.2 Processor-in-the-Loop test 76

8 Experimental Validation 79
8.1 Offline evaluation 79
8.1.1 Segmentation metrics 79
8.1.2 Geometric reference accuracy 81

8.2 Field validation . . . 91
8.2.1 Ground-truth reference: laser sensor . . . 91
8.2.2 Field evaluation metrics . . . 91
8.2.3 Test conditions . . . 93
8.2.4 Closed-loop navigation results . . . 93
8.2.5 Qualitative observations . . . 94
8.3 Discussion . . . 94
8.3.1 Segmentation performance . . . 94
8.3.2 Geometric extraction trade-offs . . . 95
8.3.3 EdgeNet: first results . . . 95

9 Conclusions and Future Work 97
9.1 Main contributions . . . 97
9.2 Conclusions on segmentation and annotation . . . 98
9.3 Conclusions on path extraction . . . 99
9.4 Limitations . . . 100
9.5 Future work . . . 101
9.5.1 Dataset and annotation . . . 101
9.5.2 Segmentation and model deployment . . . 102
9.5.3 Geometric extraction . . . 102
9.5.4 System integration and field validation . . . 103
9.6 Closing remarks . . . 103

Bibliography 104

List of Figures

2.1 Agricultural inter-row environment, vehicle frames, and geometric error definition 6
2.2 Complete vision-based local guidance pipeline from camera stream to actuator commands 7

4.1 Fully Convolutional Network (FCN) architecture with progressive skip connections combining deep semantic features with shallow appearance representations (adapted from Long et al. [9]). 22
4.2 DeepLabV3 architecture featuring the Atrous Spatial Pyramid Pooling (ASPP) module with parallel atrous convolutions and image-level pooling (adapted from Chen et al. [10]). 24
4.3 MobileNetV3 and Lite Reduced ASPP (LR-ASPP) architecture: (a) inverted residual block combining depthwise separable convolution, linear bottleneck, and Squeeze-and-Excitation channel attention; (b) segmentation head combining stride-16 features modulated by global context gating with stride-8 fine features (adapted from Howard et al. [11]). 29
4.4 Original Segment Anything Model (SAM) architecture with ViT image encoder, prompt encoder, and mask decoder (adapted from Kirillov et al. [12]). SAM 3 extends this foundation with a dual detector-tracker architecture and open-vocabulary concept segmentation. 31

5.1 Sample field images across crops and corresponding training masks 34
5.2 SAM 3 zero-shot autolabeling and mask composition workflow . . . 36
5.3 Training and validation loss per epoch of FCN, DeepLabV3, and LR-ASPP on the manual lettuce dataset. The best checkpoint is the epoch with the minimum validation loss. The backbone is unfrozen after epoch 10. 41

6.1 Example of the ROI and of the mask cleaning of Strip-IRLS on an example mask, with the parameters used on the robot. The rows above row 64 are excluded, and the dark gray regions inside the ROI are removed by the cleaning. 48
6.2 Strips and intervals on the mask of Figure 6.1, after the cleaning of Section 6.2.1, with the parameters used on the robot ($N = 16$, $h = 10$ px, ROI from row 64). The blue lines are the strip boundaries, and the orange boxes are the intervals found in each strip. 49
6.3 Chains built from the intervals of Figure 6.2. (a) The two chains grown from the seeds of strip 0, in purple from the blob and in orange from the corridor, with their number of intervals, samples and total cost. The dots are the samples, the endpoints that are not clipped. The dashed red boxes are intervals that are not in any chain. (b) The selected chain, the one with more samples. The yellow dots are its samples, used in the edge fits. The regions in light red are not in the selected chain and are not used in the fits. . 51
6.4 Reweighted fit of the two edges of Strip-IRLS on the samples of the selected chain of Figure 6.3b, at the start and after iterations 1, 3 and 10. The points are the samples of the chain, colored by their weight, the blue and green lines are the fits of the left and right edge, and the red line is the centerline, their mean. The three upper samples of the right edge, on the region attached to the corridor, pull the first fit toward the vertical. Their weights fall to 0.21, 0.10 and 0.09 only after several iterations, and after the tenth the line follows the edge. 54
6.5 Architecture of *EdgeNet*. Each box is a feature map, colored by the layer that produces it and labeled with channels × height × width. The two maps of a block have the same size and share one label. The thickness of a box grows with the channels and its height with the size of the map, not to scale. The mask is averaged over 2×2 blocks, and two channels with the x and y coordinates of each pixel, scaled to $[-1,1]$, are added, so the convolutions also see where each pixel is. Each of the four blocks halves the size with a stride-2 convolution and then applies a second convolution. The $96 \times 7 \times 7$ map is flattened and passed to two fully connected layers, the first followed by ReLU and dropout 0.1. 61

7.1 Side view of Demetra, with the wheel diameter, the wheelbase L, the camera height h_{cam} and the adjustable tilt θ of the camera. . . . 67

7.2 Top view of the opposite-phase 4WS kinematics of Demetra, with the equivalent bicycle in orange, the ICR and the turning radius R of the mid-axle point M (Equation (7.1)). 67
7.3 ROS 2 node graph on the robot, as shown by `rqt_graph`. Ellipses are nodes and rectangles are topics. 69
7.4 Frame of the debug video recorded during a field run with Strip-IRLS. From left to right: segmentation (the segmentation before the morphological cleaning), region of interest (the cleaned segmentation, on which the path extractor operates) and path (the extracted path drawn on the camera image). 69
7.5 Measured lateral error e_k and filtered error $\hat{e}_k$ during 1.5 s of a field run with Strip-IRLS. The filtered error was computed from the recorded measurements with Equation (7.3) and the smoothing factor of Equation (7.5) for the recorded time between frames. . . . 71
7.6 Response of the filter of Equation (7.3) to a step of the input, with $\alpha_{\text{ref}} = 0.20$. At 15 frames per second, the factor converted with Equation (7.5) gives the same response in time as at 30 frames per second (dashed line). 72
7.7 Angular velocity target ω^* of Equation (7.9) as a function of the command error e_{cmd}. (a) Effect of the confidence, with the gain used on the robot, $K_p = 1.2$. The curves are for $C = 1$, $C = 0.60$ (threshold between `ok` and `degraded`) and $C = 0.38$ (threshold below which the status is `lost`). (b) Effect of the gain, with $C = 1$. The command saturates at 36 px with $K_p = 1.8$, at 54 px with $K_p = 1.2$ and at 72 px with $K_p = 0.9$. In both plots the command is zero inside the deadband ($|e_{\text{cmd}}| < 4$ px) and is limited to ± 0.58 rad/s. The right axis gives the corresponding steering angle δ at $v = 0.20$ m/s, where Equation (7.2) becomes $\delta = \arctan(\omega)$, so 0.58 rad/s corresponds to 30°. 74
7.8 Processor-in-the-Loop test. Top: frame of the simulation node shifted with Equation (7.13), with the edges (blue and green), the centerline (red) and the lateral error at y_{ref} (magenta) extracted offline by the network and Strip-IRLS. Bottom: the raised robot during the test, at similar lateral errors. (a) and (b) Lateral error of -48 px and $+39$ px: the wheels steer in opposite directions to bring the simulated robot back to the center. (c) Error below 1 px: the simulated robot is back at the center and the wheels are straight. The mirrored vegetation comes from the border that the simulation node adds before the shift. 78

8.1 Visual comparison of segmentation outputs. (a) Four of ten test images of the manual lettuce dataset, taken at evenly spaced positions of the test split: RGB image, manual ground truth, and the predictions of the LR-ASPP network trained on the manual labels and of the three LR-ASPP networks trained only on SAM 3 lettuce labels. In the third image the SAM 3 networks mark the whole floor as path, including the strips beside the central corridor, and in the fourth, where the manual ground truth has no path, they mark the soil in front of the robot. (b) Tomato and cucumber images, which have no ground truth: RGB image and prediction of the binary network installed on the robot, as a qualitative comparison only. . . 80
8.2 Lines estimated by the five extractors on four masks with disturbances, chosen among the masks where Strip-IRLS has an error below 4 px and at least two other extractors fail. (a) `hole_bottom_tall`, (b) `pom_ondulato` and (c) `pom_spostato` are masks of the synthetic test sets, and (d) is the real frame `pomodori2_01051`, whose true centerline comes from the hand annotation. The magenta line is the estimated centerline, the diamond its position at y_{ref} (dashed line), and the yellow circle the true centerline at y_{ref}. Below each panel are the status and the error at y_{ref}. A panel without a line is `lost`. 84
8.3 Lateral tracking error and heading error profiles during inter-row autonomous navigation 93

Chapter 1

Introduction

▶ **DA COMPLETARE: Contesto di tesi e motivazione applicativa**

Inquadrare l'attività svolta nell'ambito della tesi di laurea magistrale:

- Collaborazione con il gruppo di ricerca di riferimento (Politecnico di Torino).
- Esigenze dell'agricoltura di precisione: diserbo meccanico selettivo, monitoraggio delle colture e riduzione dell'impatto ambientale senza dipendenza da segnali GNSS RTK sotto chioma o in serra.
- Obiettivo di realizzare una soluzione autonoma, modulare e fruibile su calcolatori embedded a basso costo (famiglia NVIDIA Jetson).

1.1 Context and motivation

Global population growth and rising food demand exert increasing pressure on agricultural productivity, while the sector faces persistent labor shortages and rising operating costs [1, 2]. Addressing these demands requires higher automation in field operations such as weeding, spraying, and crop monitoring. Row-based crops are among the most studied applications of agricultural robots and account for about 75% of the planted cropland in the United States [3]. Autonomous ground vehicles designed for these settings must navigate along inter-row corridors reliably without damaging crops or compacting productive soil beds.

Outdoor agricultural navigation historically relied on Global Navigation Satellite Systems (GNSS) augmented by Real-Time Kinematic (RTK) corrections. However, satellite positioning presents well-documented limitations in field environments: thick crop canopies cause multi-path reflections and severe signal attenuation, while greenhouse installations obstruct satellite reception entirely [1, 3]. Furthermore, RTK base station infrastructure adds cost. Alternative exteroceptive sensors such as 3D LiDAR provide direct spatial measurements, but their high unit cost hinders

economic scalability across commercial robotic fleets, and their point clouds lack the color and textural information required to distinguish crops from weeds [1, 3]. Monocular RGB cameras offer a cost-effective alternative that captures detailed chromatic and texture cues directly from the scene.

Early vision-based guidance pipelines extracted navigation lines through handcrafted image processing techniques. These approaches compute color vegetation indices to isolate plant matter from soil, apply intensity thresholding such as Otsu's method, and detect corridor edges using line-fitting algorithms or Hough transforms [1, 4]. Although computationally lightweight, these calibrated pipelines degrade when ambient field conditions shift. Variable sunlight angles, cast shadows from foliage, soil moisture gradients, specular reflections on wet leaves, and weed infestations frequently disrupt fixed thresholding rules and generate false boundary estimates.

Deep learning architectures, particularly convolutional neural networks for semantic segmentation, overcome these hand-tuning limitations. By learning hierarchical visual representations directly from image datasets, segmentation networks achieve higher tolerance to illumination shifts, ground irregularities, and weed clutter than handcrafted color models.

However, deep neural networks introduce a practical operational bottleneck: supervised training requires extensive pixel-level ground truth. Manually labeling agricultural scenes is labor-intensive and must cover multiple crop types, growth stages, and ambient conditions. In addition, running neural networks on compact onboard computers requires balancing segmentation accuracy against processing speed and power consumption, as steering commands must be updated fast enough to guide the vehicle in real time.

This thesis addresses these challenges through a modular pipeline that combines automatic dataset annotation with SAM 3, real-time onboard semantic segmentation, and deterministic geometric reference extraction for local robot guidance.

1.2 Objectives of the thesis

The main objectives of this work are:

- Evaluate whether segmentation networks trained on SAM 3 labels, used without manual correction, approach a network trained on manual masks, using manually annotated lettuce images as ground truth.
- Select a segmentation architecture balancing accuracy and inference speed for robotic deployment, through offline evaluation of FCN, DeepLabV3, and LR-ASPP on manually annotated lettuce images.

- Evaluate the effect of output label configurations (four-class, three-class, and binary path) on path segmentation accuracy.
- Implement an edge-based geometric extractor that converts segmentation masks into local image-space navigation references, and compare it with three baseline algorithms.
- Integrate the selected perception and geometric components on a robot platform for field validation.

1.3 Organization of the work

The thesis is organized as follows:

Chapter 2 introduces the agricultural navigation problem, the role of vision-based local guidance, and the segmentation-to-control pipeline structure.

Chapter 3 reviews vision-based methods for agricultural row navigation, covering classical and learning-based segmentation approaches, geometric extraction techniques, and experimental validation in related work.

Chapter 4 details the mathematical formulations and designs of the deep learning architectures evaluated in this work, focusing on FCN, DeepLabV3, MobileNetV3 with the LR-ASPP head, and SAM 3.

Chapter 5 describes dataset preparation with manual annotation and automatic SAM 3 labeling, empirical comparisons among the three architectures on lettuce, and evaluations of supervision strategies across label configurations.

Chapter 6 presents the two extractors developed in this work, which convert corridor masks into navigation references: Strip-IRLS, based on horizontal-strip interval selection and independent edge fitting, and EdgeNet, a network that regresses the corridor edges, which is a first experiment. Three baseline algorithms are described for comparison.

Chapter 7 describes the robot hardware, camera configuration, software integration using ROS, and the deployment of the trained segmentation model.

Chapter 8 reports offline and field validation results for segmentation accuracy, geometric reference extraction, and closed-loop navigation performance.

Chapter 9 summarizes the main findings, discusses limitations, and identifies directions for future work.

Chapter 2

Background and Problem Formulation

This chapter establishes the agricultural navigation context and formulates the vision-based local guidance problem addressed in this thesis.

2.1 Agricultural autonomous navigation

2.1.1 Navigation tasks and requirements

Agricultural robots perform tasks such as monitoring, spraying, harvesting, and weeding in structured crop environments [1]. Navigation between crop rows requires the robot to follow a path aligned with the row structure while avoiding plants and obstacles. The guidance reference must be computed from local sensing, as global GNSS positioning may be obstructed by crop canopies or greenhouse structures [1, 3].

Vision-based methods identify crop rows, inter-row paths, or crop boundaries from camera images and convert these observations into steering commands [1, 2]. The task is local: the robot maintains its position relative to the visible row structure, not a global map.

2.1.2 Environmental challenges

Field conditions vary with illumination, shadows, weather, growth stage, and weed presence [5, 2]. Plants may occlude parts of the path, and gaps or irregular spacing disrupt row structure. These changes affect both the appearance of image features and the reliability of geometric references extracted from them.

Segmentation models trained on one set of conditions must generalize to new lighting, crops, or row configurations. Geometric extractors must handle incomplete or ambiguous corridor observations without assuming perfect segmentation [6].

2.2 Vision-based local guidance

2.2.1 Sensing modalities

Camera-based perception provides appearance information for distinguishing plants, soil, and paths. Monocular RGB cameras are simpler and less expensive than LiDAR or stereo systems, but do not directly measure depth [1]. While RGB-D cameras provide both appearance and depth measurements, their performance depends on field conditions and hardware cost constraints.

This thesis uses monocular RGB sensing for local guidance. It does not reconstruct three-dimensional field structure or maintain a global map.

2.2.2 Image-space reference extraction

The guidance pipeline converts image observations into a reference suitable for control. Depending on the method, this may be a horizontal position, a fitted line, or a curve [7, 6, 4]. Image-space references describe the path geometry in pixel coordinates; they must be interpreted by the controller to generate velocity commands.

Straight-line models assume locally linear boundaries and produce simple references for proportional control [4]. They do not capture distant curves or complex row patterns, but are computationally efficient and suitable for short prediction horizons.

▶ DA COMPLETARE: Modello cinematico 4WS e formulazione analitica degli errori

Integrare la formalizzazione matematica del veicolo e del controllo:

- Modello cinematico per piattaforma 4WS a sterzatura simmetrica in controfase ($\delta_f = -\delta_r$) e relazione cinematica: $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega = \frac{2v\tan\delta}{L}$.
- Trasformazione rigida e proiettiva tra coordinate pixel (u, v) nel piano immagine e coordinate metriche nel frame robot $\{R\}$.
- Definizione rigorosa dell'offset laterale e_y e dell'heading error e_θ calcolati rispetto alla retta di mezzeria stimata.
- Formulazione della legge di guida proporzionale / Stanley per la determinazione della velocità angolare $\omega = K_p e_y + K_\theta e_\theta$.



Figure 2.1: Agricultural inter-row environment, vehicle frames, and geometric error definition

2.3 The segmentation-to-control pipeline

[SPAZIO FIGURA / SCHEMA]

Complete vision-based local guidance pipeline from camera stream to actuator commands

Schema a blocchi dell'architettura di navigazione: flusso continuo dall'acquisizione delle immagini monoculari, attraverso la rete di segmentazione semantica, l'estrazione geometrica dei bordi del corridoio, il calcolo degli errori di guida e la generazione dei comandi di velocità (v, ω).

Figure 2.2: Complete vision-based local guidance pipeline from camera stream to actuator commands

2.3.1 Perception stage

The perception stage assigns semantic labels to image pixels. For inter-row navigation, target classes typically comprise crop plants, the traversable path, covering material such as plastic mulch, and background [6]. The definition of output classes directly determines what the geometric stage can extract: a binary vegetation mask identifies crop rows, whereas a path mask isolates the drivable space between them [2, 3].

Segmentation models are trained on annotated field images. While transfer learning from pretrained backbones reduces training duration and improves feature generalization, the classification head must be tailored to the specific target crop classes and deployed at low latency on embedded hardware [3].

2.3.2 Geometric extraction stage

The geometric stage reduces the dense segmentation mask to a low-dimensional guidance reference for robot steering. Existing methods differ in their spatial search strategy and fitted geometric model: some aggregate mask pixels across image columns [4, 3], whereas others extract local intervals from horizontal strips [6] or search for line candidates within bounded angular regions [7].

The extraction algorithm must tolerate segmentation noise, missing plants, and cast shadows without producing erratic steering targets. Validation checks can reject estimates that fail geometric consistency thresholds, requiring the controller to handle transient frames where no valid reference is recovered [5].

2.3.3 Control integration

The controller converts the extracted geometric reference into forward velocity and steering commands. Proportional and look-ahead laws regulate steering using the lateral displacement and heading orientation errors relative to the image center [4].

In field operations, raw error signals extracted from semantic segmentation exhibit high-frequency jitter and occasional impulse disturbances caused by irregular foliage contours or transient detection dropouts. Directly feeding unfiltered errors into the steering loop induces actuator chattering and abrupt steering adjustments that compromise vehicle stability. Consequently, the control interface incorporates error conditioning structured across three complementary mechanisms:

1. **High-frequency smoothing:** a recursive low-pass filter attenuates persistent image-space tracking jitter without introducing excessive phase delay during row transitions;

2. **Impulse outlier rejection:** large step variations between consecutive frames are evaluated over temporal windows to isolate single-frame false detections while preserving genuine trajectory adjustments;

3. **Temporal hold and deadband:** a zero-order hold maintains command continuity across short-term perception dropouts, while a symmetric deadband around the corridor center prevents steering hunting.

The mathematical formulation and parameter tuning of this conditioning pipeline on the Demetra robotic platform are detailed in Section 7.2.3 of Chapter 7.

Closed-loop field evaluation measures the robot's ability to maintain row alignment under operational conditions. High segmentation accuracy and accurate geometric fitting are necessary prerequisites, but stable inter-row tracking ultimately depends on real-time execution and disturbance rejection over continuous runs [4, 2, 3].

Chapter 3

State of the Art

This chapter reviews vision-based methods for agricultural row navigation, focusing on the conversion of image observations into a local guidance reference. As surveyed by Bai et al. [1], vision sensors provide rich appearance and geometry cues for autonomous field guidance at low hardware cost. The review covers visual perception, geometric extraction, and the integration of these stages with robot control. Dataset preparation and the implementation used in this thesis are described in Chapters 5 and 6.

3.1 Classical image-processing methods

Classical pipelines combine image filtering, thresholding, and application-specific geometric rules. Mendez et al. convert vineyard images to grayscale, apply Gaussian filtering and Otsu thresholding, select the largest blob, and use column pixel counts to estimate the row center [4, Section 3.4]. Such methods can be effective in the conditions for which their parameters were selected, but changes in illumination, weeds, and crop appearance can require retuning [1].

3.2 Learning-based segmentation for agricultural navigation

3.2.1 Semantic segmentation approaches

Semantic segmentation assigns a class to each pixel, but the label definition determines what the geometric stage can recover. De Silva et al. train U-Net to predict narrow crop-row masks in sugar-beet images. Their labels represent row structure rather than individual plant pixels or traversable soil [2, Sections 3.3 and 4.1]. Navone et al. instead predict a binary vegetation mask from RGB images

using a MobileNetV3 backbone and an LR-ASPP head, then combine that output with depth for reference extraction [8, 3].

Like other path-oriented approaches in the literature, this work segments the traversable soil corridor between crop rows rather than the plants themselves. The segmentation model provides a candidate region of drivable ground, and the downstream geometric stage then extracts the lateral boundaries and computes the steering reference.

3.2.2 Training data and generalization

Pixel-level annotation is time-consuming, and agricultural datasets must represent changes in crop, growth stage, and lighting. De Silva et al. include 11 field variations in their dataset [2, Section 3]. Navone et al. initialize their segmentation model with ImageNet weights and train using synthetic AgriSeg images supplemented by 100 real crop images [3, Section 4.1, Table 1]. Pretraining and the choice of training images are separate design choices.

3.3 Semantic segmentation architectures

3.3.1 FCN, DeepLabV3, and mobile architectures

Fully Convolutional Networks (FCN) replace fully connected layers with convolutions to produce dense pixel-level predictions [9]. DeepLabV3 incorporates atrous convolutions and atrous spatial pyramid pooling to capture multi-scale context without decreasing feature resolution [10]. For embedded applications where compute resources are constrained, Howard et al. [11] introduced MobileNetV3 together with the Lite Reduced Atrous Spatial Pyramid Pooling (LR-ASPP) head, which reduces latency by using depthwise separable convolutions.

The choice of architecture affects both segmentation accuracy and inference speed. Deeper backbones or larger receptive fields can yield higher IoU values at the cost of additional compute. For robotic systems operating in closed loop, inference latency on edge hardware dictates the achievable control frequency.

3.3.2 Transfer learning and pretraining

Transfer learning initializes network weights from models pretrained on large visual corpora such as ImageNet. The pretrained backbone is a generic feature extractor, while the task-specific segmentation head is adapted to produce class scores matching the target dataset. In agricultural vision, pretraining reduces training time and mitigates overfitting when domain-specific annotated datasets are limited in size. For instance, Li et al. [6, Section 2.2.2] demonstrate that transferring

weights learned on corn fields to other crops (such as tomatoes, cucumbers, and wheat) preserves segmentation accuracy with small training sets. Similarly, Navone et al. [3, Section 4.1] initialize their segmentation backbone with ImageNet weights prior to fine-tuning on tree-row imagery.

The extent of pretraining differs across implementations. Some setups retain pretrained segmentation heads and replace only the final classification layer, whereas others initialize only the backbone and train the segmentation head from scratch.

3.4 Dataset preparation and model-assisted annotation

3.4.1 Manual annotation challenges

Manual pixel-level annotation requires annotators to label every pixel in training images. For agricultural datasets spanning multiple crops, growth stages, and lighting conditions, this process is time-consuming and expensive. Annotation consistency depends on annotator interpretation of ambiguous boundaries, particularly where plants overlap soil or covering material.

3.4.2 SAM and foundation models for segmentation

Foundation models for segmentation have established model-assisted annotation as an effective alternative to purely manual labeling. The Segment Anything project built its dataset with a model-in-the-loop data engine in three stages (assisted-manual, semi-automatic, and fully automatic), obtaining more than one billion masks on 11 million images [12]. While early foundation models required spatial prompts such as bounding boxes or foreground/background clicks for every target instance, Segment Anything Model 3 (SAM 3) introduces open-vocabulary concept segmentation driven directly by textual descriptions and visual exemplars [13].

In precision agriculture, model-assisted annotation with foundation models has been explored primarily for satellite-based field boundary delineation and canopy phenotyping. However, its systematic application to agricultural ground navigation remains largely unexplored. In this thesis, SAM 3 is employed offline as a zero-shot concept annotator, and its masks are used without manual correction. On lettuce, networks trained on SAM 3 labels with different class sets (path, plants, plastic mulch) are compared against manual ground truth. Binary path labels generated for the complete lettuce, tomato, and cucumber collection train the network deployed on the robot. This offline annotation pipeline decouples the heavy computational burden of foundation models from the real-time inference requirements of the onboard robot controller.

3.5 Geometric extraction methods

The geometric stage reduces a mask to a quantity the controller can use. Depending on the representation and method, the output may be a horizontal image position, a straight line, a crop-edge estimate, or a curve. These outputs describe different tasks and coordinate systems.

3.5.1 Line fitting and crop-edge detection

Chen et al. estimate a path line for greenhouse cucumber rows from navigation points selected in plant–soil segmentation. Their prediction-point Hough transform uses a least-squares estimate to restrict the line search and is compared with traditional Hough and least-squares fitting [7, Sections 2–3].

Kneip et al. detect crop–ground transitions in scanlines of a stereo elevation map, then fit a linear crop-edge model using a Huber M-estimator and iteratively reweighted least squares. Their output is a crop boundary in ground-plane coordinates, not an image-space estimate of the inter-row path [5, Sections 6–7].

3.5.2 Column-wise histogram methods

Histogram methods aggregate mask pixels by image column. Mendez et al. estimate the row center from the five columns with the largest pixel counts in a thresholded background region [4, Section 3.4.3]. Navone et al. use a different input: a temporally combined vegetation mask refined with depth. SegMin smooths its column histogram and selects the minimum. SegMinD weights the vegetation mask by normalized inverse depth before calculating the histogram [3, Section 3]. In both cases the output is a horizontal position, not a full path line.

3.5.3 Horizontal strips and adaptive multi-ROI methods

Li et al. combine a Faster-U-net inter-row-path mask with an adaptive multi-ROI procedure. Horizontal regions supply ordered local points, which are used as control points for a multisegment cubic B-spline. Their experiments use 40-pixel search regions and discuss the trade-off between the number of points and fitting behavior [6, Section 2.2.3, Figure 9].

This is the closest geometric comparison for the present work because both pipelines select local evidence from horizontal image regions. The representation and fitted model differ: Li et al. use an inter-row-path mask and a spline, whereas this thesis estimates a local straight reference from the soil corridor between parallel crop rows.

3.5.4 Triangle Scan Method

De Silva et al. propose the Triangle Scan Method (TSM) for crop-row masks. It first scans an upper horizontal band to find an anchor column, using a preset anchor if the scan fails a threshold. Candidate lines from that anchor to positions along the base of a triangular region are then evaluated, and the line with the greatest mask support defines the central crop row. Its two endpoints provide position and angular errors for visual servoing [2, Section 4.2, Algorithm 1].

TSM is not the same as extracting one point from each of several horizontal strips: the upper scan establishes an anchor for a bounded line search. It also operates on crop-row masks, so its output is not directly equivalent to a reference extracted from a soil mask.

3.5.5 Bilateral boundary tracking versus centroid reduction

In agricultural vision pipelines that convert binary segmentations into geometric references, the existing methods can be grouped as follows:

1. **Crop-canopy feature extraction with RANSAC:** A classical paradigm detects feature points along crop canopies or plant stalks, partitions them into left and right row clusters, and fits linear models using Random Sample Consensus (RANSAC) combined with ordinary least squares [1]. Recent monocular vision implementations extract crop row anchor points via DBSCAN clustering followed by RANSAC fitting to generate navigation lines, with reported angular and lateral errors minimized on the central crop row [14]. While functional in dense, uniform vegetation, this approach depends on continuous plant growth and degrades when seedlings are missing or canopy density is irregular. RANSAC draws random samples: each iteration selects a minimal subset of points (typically two for line fitting), computes a candidate model, and counts inliers within a residual threshold. To achieve 99% confidence of finding an all-inlier sample for a two-point linear model when outliers constitute 50% of the data, the algorithm requires 17 iterations, rising to 36 iterations when the outlier ratio reaches 65%, and the number of iterations grows quickly with the outlier ratio [15].

2. **Global contour tracking with line fitting:** An alternative approach applies edge operators (such as Canny filtering) or boundary tracing algorithms (`findContours`) over the full segmented corridor mask, separating the resulting pixel cloud into lateral boundaries before fitting lines with RANSAC.

3. **Horizontal strip decomposition with centroid reduction:** Methods operating on segmented drivable soil corridors (such as the multi-ROI scheme of Li et al. [6]) partition the image into horizontal bands and extract the bounding

interval $[x_L, x_R]$ within each strip. However, these methods immediately collapse each interval into its midpoint:

$$x_C = \frac{x_L + x_R}{2} \tag{3.1}$$

fitting curves (such as cubic B-splines or polynomials) directly to the sequence of centroids x_C. This premature reduction discards the distinction between the two physical edges. Under asymmetric disturbances, such as protruding leaves, localized weeds, or shadows extending along one side of the corridor, the midpoint shifts toward the intrusion, which introduces systematic bias in both lateral offset and heading angle.

4. **Learned extraction of the navigation reference:** In autonomous driving, Müller et al. give the binary road segmentation predicted by a first network to a second network, which outputs two waypoints for a low-level controller [16]. This policy is trained only in simulation and then runs on a 1/5-scale truck. The binary segmentation removes texture, lighting, shading and weather, so simulated and real inputs look alike. The policy is trained on the masks that the real segmentation network predicts on simulated images, not on ground-truth masks, so it learns to handle the errors of the segmentation. In row-crop navigation, CropFollow regresses the heading of the robot relative to the row and its placement between the rows directly from the RGB image, with a ResNet-18 backbone [17]. These methods learn the reference from data instead of using hand-designed geometric rules. In the sources consulted for this thesis, no work regresses the two corridor edges from the segmentation mask of a crop corridor.

5. **Single-edge elevation fitting:** M-estimators have also been applied to crop boundaries, such as Huber-loss Iteratively Reweighted Least Squares (IRLS) on stereo elevation scanlines [5]. However, these formulations track a single cutting boundary along dense harvesting fronts, rather than parallel crop boundaries defining an inter-row corridor from monocular RGB imagery.

This thesis investigates a deterministic geometric formulation with two separate edge lines:

$$x_L(y) = m_L y + q_L, \qquad x_R(y) = m_R y + q_R \tag{3.2}$$

Both boundaries are estimated by Iteratively Reweighted Least Squares (IRLS) with the weight rule

$$w_i = \frac{1}{1 + \left(\frac{r_i}{\sigma}\right)^2} \tag{3.3}$$

applied for a fixed number of ten iterations to at most one boundary sample per horizontal strip ($N = 16$ on the robot). Chapter 6 details how these weights enter

the fit. A perturbation on one boundary changes only the weights and fit of that boundary. The centerline reference $x_C(y) = (x_L(y) + x_R(y))/2$ is computed after both boundary fits.

3.5.6 Comparison of geometric representations

Table 3.1 summarizes the input representation and geometric output of the methods discussed above.

Table 3.1: Perceptual inputs and geometric references in selected agricultural navigation studies.

Study	Perceptual input	Geometric operation	Output
Chen et al. [7]	Plant–soil segmentation and navigation points	Restricted Hough search around a predicted point	Image-space path line
Kneip et al. [5]	Stereo elevation map	Scanline transitions and IRLS linear fit	Ground-plane crop edge
Mendez et al. [4]	Thresholded background region	Column counts and mean of the five largest	Horizontal row-center position
Navone et al. [3]	Vegetation mask with depth	Smoothed column histogram, inverse-depth weighting in SegMinD	Histogram-minimum position
Li et al. [6]	Predicted inter-row-path mask	Adaptive horizontal regions and cubic B-spline	Curved image-space path
De Silva et al. [2]	Predicted crop-row mask	Upper anchor and triangular line search	Central crop-row line
This thesis (Strip-IRLS)	Predicted soil-corridor / mulch mask	Discrete horizontal strips, separate IRLS fits of the two boundaries	Straight image-space centerline $x_C(y)$ and (e_y, e_θ)

The methods are not directly ranked by this comparison: their inputs and outputs differ. Li et al. provide the closest structural precedent for strip-based point selection, while TSM illustrates a line search for a crop-row representation. Neither method uses the same soil-corridor mask and straight-line model as the pipeline developed in this thesis.

3.6 System integration and experimental validation

3.6.1 Perception and control integration

Navigation systems use the extracted reference to generate motion commands. Navone et al. convert the displacement of their histogram reference into linear and angular velocity commands and smooth them with an Exponential Moving Average [3, Section 3.4]. Mendez et al. use proportional control based on row-center displacement and the distribution of background pixels [4, Section 3.5]. De Silva et al. combine the central row's angular and positional errors in a proportional visual-servoing controller for simulation [2, Section 4.3].

3.6.2 Scope of reported evaluations

The studies evaluate different stages of navigation. Kneip et al. assess crop-edge detection on recorded harvesting data [5, Section 8]. Chen et al. report angular errors and processing times for image-based path fitting [7, Sections 3.6, Tables 6–7]. De Silva et al. evaluate row detection on real images and visual servoing in simulation, with preliminary real-world row following also reported [2, Sections 5.2–5.3 and 6, Table 5]. Mendez et al. report closed-loop vineyard trials, measuring centering error from the distances to the two sides of the row [4, Sections 3.6 and 4]. Navone et al. test in simulation and crop fields, using LiDAR and attitude measurements to assess displacement and heading [3, Sections 4.3–4.4].

Segmentation IoU, image-space line error, field tracking error, and processing time measure different parts of the system. Results from these studies therefore provide context, not a common performance benchmark.

3.7 Positioning of this thesis

The system developed in this thesis addresses autonomous navigation in GNSS-denied crop rows using monocular RGB vision and embedded computation. Its technical positioning relative to the state of the art rests on three architectural decisions:

1. **Perceptual representation (Soil corridor versus canopy):** Rather than detecting crop stalks or foliage masks, the segmentation stage targets the navigable soil corridor (and plastic mulch). This makes the navigation reference independent of crop growth stage, canopy height, and missing plants, providing a continuous geometric guide directly at ground level.

2. **Geometric extraction (Decoupled deterministic boundaries versus centroid reduction):** Rather than collapsing strip intervals into midpoints prior to fitting (as in Li et al. [6]) or employing stochastic RANSAC over dense edge contours, the proposed pipeline decouples the left and right physical corridor boundaries. Each boundary is fitted independently by iteratively reweighted least squares with a fixed number of four iterations. A disturbance on one side (such as a leaf overhang or a shadow) enters only the fit of that boundary. The centerline, as the mean of the two edge lines, receives half of that boundary's deviation.

3. **Embedded deployment feasibility:** The architecture decouples offline automatic dataset generation from onboard execution. Large foundation models (SAM 3) are restricted to offline zero-shot concept annotation, while real-time inference on the NVIDIA Jetson AGX Orin is handled by a lightweight LR-ASPP network (using a MobileNetV3 backbone) paired with the deterministic IRLS geometric extractor. Without the debug video, the onboard callback takes 21.6 ms with Strip-IRLS, shorter than the 33.3 ms frame period of the 30 fps camera (Section 7.3.1), and the pipeline uses no second neural network, stereo elevation mapping, or LiDAR sensor.

The chapters that follow detail and evaluate each stage of this pipeline: Chapter 4 formulates the deep learning architectures, Chapter 5 analyzes the annotation protocols and comparative model training, Chapter 6 formalizes the geometric extractors, Chapter 7 describes the Demetra robotic platform and onboard ROS integration, and Chapter 8 reports offline benchmarks and field tests.

Chapter 4

Deep Learning Architectures for Semantic Segmentation

In computer vision, semantic segmentation is the task of classifying every pixel in an image according to the category of the surface or object it belongs to [9], [18, Chapter 12]. Unlike image-level classification, which predicts a single global label for the entire scene, or object detection, which localizes entities with coarse bounding boxes, semantic segmentation generates a dense, pixel-level partition of the scene.

In this thesis, autonomous row navigation relies on a monocular vision pipeline in which semantic segmentation is the only perception stage. Rather than relying on satellite positioning (RTK-GPS) or active range sensors (LiDAR), the navigation system extracts the vehicle guidance reference directly from single RGB images by segmenting the navigable soil corridor from surrounding crop canopies, weed patches, and plastic mulch films under variable outdoor illumination.

To investigate the trade-off between segmentation accuracy, model complexity, and real-time execution on the mobile robot, this chapter analyzes three distinct deep learning architectures:

- **Fully Convolutional Networks (FCN):** pairing a dilated ResNet-50 backbone with a convolutional segmentation head (`FCNHead`);
- **DeepLabV3:** pairing a dilated ResNet-50 backbone with an Atrous Spatial Pyramid Pooling (ASPP) head;
- **MobileNetV3 with LR-ASPP:** pairing a MobileNetV3-Large backbone with a Lite Reduced ASPP head.

In addition, the Segment Anything Model 3 (SAM 3) is introduced as an offline foundation model for semi-automated training dataset annotation.

The remainder of this chapter is structured as follows. Section 4.1 establishes the mathematical formulation and encoder-decoder principles. Sections 4.2, 4.3, and 4.4 detail the three segmentation networks, while Section 4.5 describes the SAM 3 annotation pipeline.

4.1 Problem formulation and encoder-decoder principles

Let $I \in \mathbb{R}^{H\times W\times 3}$ denote an input color image with height H and width W. Semantic segmentation aims to estimate a dense label map $Y \in \{1,\dots,C\}^{H\times W}$, where each pixel (i,j) is assigned to one of C mutually exclusive semantic classes [9], [18, Chapter 12]. Deep segmentation networks approximate this mapping through a parameterized function $f_\theta(I)$ that outputs an unnormalized score tensor $Z \in \mathbb{R}^{H\times W\times C}$ containing the class logits for every pixel location. The predicted probability of class c at pixel (i,j) is obtained by applying a pixel-wise softmax over the class dimension:

$$p_{i,j,c} = \frac{\exp(Z_{i,j,c})}{\sum_{k=1}^{C}\exp(Z_{i,j,k})}. \tag{4.1}$$

During supervised training, network parameters θ are optimized by minimizing a loss function $\mathcal{L}(Y^*,p)$ that quantifies the discrepancy between the predicted probability map p and the ground-truth annotations $Y^* \in \{0,1\}^{H\times W\times C}$, represented in one-hot form [9]. Through backpropagation, the gradients of this loss with respect to θ guide stochastic optimization algorithms (such as SGD or AdamW) to iteratively update the network weights. Because semantic segmentation operates densely across the spatial grid, the overall loss is evaluated by aggregating individual prediction errors over all pixel locations (i,j) [9], [18, Chapter 12].

A widely used training objective for semantic segmentation is the pixel-wise categorical cross-entropy loss, averaged across all pixels:

$$\mathcal{L}_{\mathrm{CE}}(Y^*,p) = -\frac{1}{HW}\sum_{i=1}^{H}\sum_{j=1}^{W}\sum_{c=1}^{C} Y^*_{i,j,c}\log(p_{i,j,c}). \tag{4.2}$$

For each pixel, this formulation penalizes the negative logarithm of the probability assigned to the ground-truth category: the penalty approaches zero when the network predicts the correct class with high confidence ($p_{i,j,c} \approx 1$), and grows large when the assigned probability is low ($p_{i,j,c} \to 0$).

In practical segmentation tasks, loss functions can be adapted to handle specific domain challenges. When target classes occupy highly disproportionate image areas, cross-entropy can be extended with class-specific penalty weights w_c to mitigate the dominance of majority classes during optimization [9], as formulated for our

agricultural dataset in Chapter 5. Other common alternatives in the literature include region-based losses (such as the Dice loss) that optimize spatial segment overlap, and boundary-aware losses designed to refine object contours.

The output stride S denotes the ratio of the input image spatial resolution to the spatial dimensions of the extracted feature map [10]. Many classification backbones progressively reduce spatial resolution through consecutive strided convolutions and pooling operations, reaching an output stride of up to $S = 2^5 = 32$ [18, Chapter 12]. While spatial downsampling expands the effective receptive field and enables deeper layers to extract abstract semantic features (determining *what* is depicted in the scene), it discards fine-grained spatial coordinates (obscuring *where* object boundaries lie at the pixel level). Segmentation backbones may instead employ smaller output strides, such as $S = 8$ or $S = 16$, using atrous convolutions to preserve denser feature representations [10].

To resolve this trade-off between high-level semantic abstraction and pixel-level spatial precision, modern semantic segmentation models commonly adopt an encoder-decoder architecture:

- **Encoder (Backbone):** A sequence of feature-extraction stages that produces high-level semantic representations, yielding a feature tensor $F_{\text{enc}} \in \mathbb{R}^{\frac{H}{S} \times \frac{W}{S} \times C_{\text{enc}}}$, where S denotes the encoder output stride. Depending on the architecture, intermediate feature maps at different resolutions may also be supplied to the decoder.
- **Decoder and segmentation head:** The decoder processes the encoded features and progressively recovers spatial resolution, optionally combining them with higher-resolution encoder features through skip connections [9]. A final segmentation head maps the resulting features to class logits, which are upsampled to the input image resolution when necessary.

4.2 Fully Convolutional Networks (FCN)

Traditional Convolutional Neural Networks are designed for image-level classification, producing a single label for the entire input. Fully Convolutional Networks (FCN), introduced by Long et al. [9], adapt these architectures for semantic segmentation by predicting a class label for every individual pixel in an end-to-end manner.

This is achieved through convolutionalization: replacing the final dense (fully connected) layers with convolutional filters allows the network to process images of arbitrary size and output dense spatial score maps. In this study, the FCN architecture is instantiated with a ResNet-50 [19] residual backbone and an `FCNHead` segmentation head, matching the standard TorchVision implementation (`fcn_resnet50`).

4.2.1 Convolutionalization of classification networks

In standard classification networks, the feature map produced by the final convolutional stage is flattened into a one-dimensional vector and fed into dense (fully connected) layers. Because a dense layer contains a fixed number of weights dedicated to each flattened input position, the network is restricted to a rigid, predetermined input image resolution.

As detailed by Géron [18, Chapter 12], a dense layer with C units connected to an input feature map of spatial dimension $h \times w$ with C_{in} channels can be replaced by a convolutional layer with C filters, a filter size equal to $h \times w$, and *valid* padding. The convolutional layer then outputs a $1 \times 1 \times C$ tensor containing the same values produced by the dense layer.

Because a convolution is applied at every spatial position, the convolutionalized network no longer requires a fixed input size. When the input feature map is larger than $h \times w$, the same filters slide over it and output a spatial grid of class scores in a single forward pass. This result is equivalent to sliding the original classification network across the image, but each intermediate activation is computed only once instead of once per overlapping window [18, Chapter 12].

4.2.2 Skip architecture and upsampling

The classification networks adapted by Long et al. (AlexNet, VGG-16, and GoogLeNet [9]) apply an overall stride of $S = 32$, so the final score map is 32 times smaller than the input image [18, Chapter 12]. Bilinear interpolation works reasonably well only for upsampling factors up to $4\times$ or $8\times$ [18, Chapter 12]; a single $32\times$ interpolation step therefore cannot restore boundary detail.

For in-network upsampling, Long et al. [9] adopted transposed convolutions (also referred to as fractionally strided convolutions or deconvolutions, available in PyTorch as `nn.ConvTranspose2d`). A transposed convolution with stride s can be interpreted as expanding the input feature map by inserting $s - 1$ rows and columns of zeros between adjacent elements, followed by a standard convolution with filter weights $w \in \mathbb{R}^{K \times K}$ [18, Chapter 12]. Algebraically, the operation reverses the forward and backward passes of a convolution [9]. In the original FCN, the intermediate $2\times$ upsampling layers are initialized to bilinear interpolation and then learned, whereas the final upsampling layer is kept fixed to bilinear interpolation [9].

Upsampling directly from stride 32 in a single step (FCN-32s) yields coarse segmentation boundaries. To refine them, Long et al. introduced skip connections that fuse high-level semantic features from deep layers with fine-grained appearance information from shallower layers [9]:

- **FCN-32s:** Upsamples the final stride-32 score map directly to image resolution in a single $32\times$ step.

- **FCN-16s:** Upsamples the stride-32 score map by a factor of 2×, adds it element-wise to predictions computed from the stride-16 layer (`pool4` in VGG-16), and upsamples the combined map by 16×.
- **FCN-8s:** Upsamples the fused FCN-16s score map by 2×, sums it with predictions from the stride-8 layer (`pool3`), and upsamples the result by 8× back to $H \times W$.

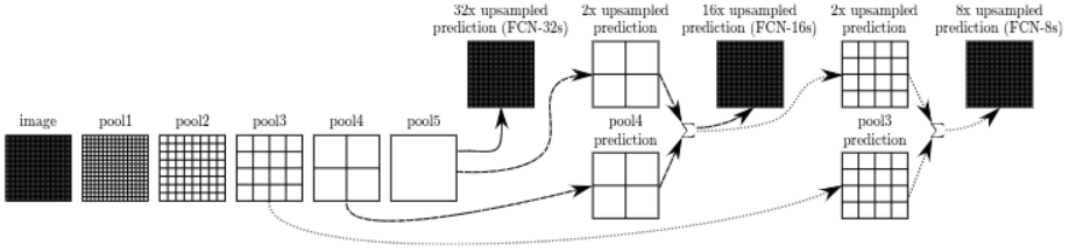


Figure 4.1: Fully Convolutional Network (FCN) architecture with progressive skip connections combining deep semantic features with shallow appearance representations (adapted from Long et al. [9]).

Implementation note: The experimental evaluation in this thesis employs the TorchVision `fcn_resnet50` implementation [20]. Unlike the original FCN-8s architecture, this model does not fuse predictions from multiple backbone stages. Instead, its ResNet-50 backbone replaces the spatial downsampling in `layer3` and `layer4` with dilated convolutions, yielding an output stride of 8. The main segmentation head (`FCNHead`) processes the 2048-channel features from `layer4` using a 3×3 convolution that reduces the channel dimension to 512, followed by batch normalization, ReLU activation, dropout with probability 0.1, and a 1×1 convolution that produces C class logits. The logits are then resized directly to the input image dimensions $H \times W$ using bilinear interpolation with `align_corners=False`. When enabled, an auxiliary classifier applies the same head structure to the 1024-channel features from `layer3`, also at output stride 8, using 256 intermediate channels. Its output can provide an additional supervision signal during training. The primary segmentation prediction is obtained from the main output (`out`), without fusion with the auxiliary output.

4.3 DeepLabV3 and atrous convolutions

DeepLabV3, developed by Chen et al. [10], addresses the resolution loss caused by consecutive pooling operations by replacing standard downsampling with atrous (dilated) convolutions and multi-scale pooling modules.

4.3.1 Atrous convolution formulation

Chen et al. [10] define atrous convolution in one dimension as $y[i] = \sum_k x[i+r\cdot k]\, w[k]$ (Eq. 1 in [10]), where the atrous rate r is the stride with which the input signal is sampled. Equivalently, the input is convolved with an upsampled filter obtained by inserting $r-1$ zeros between two consecutive filter values along each spatial dimension [10]. Applied along both spatial dimensions to a 2D feature map $x[i,j]$ with a $K \times K$ filter $w[p,q]$, where $p, q \in \{-M, \ldots, M\}$ and $M = \frac{K-1}{2}$ for odd K, the operation becomes:

$$y[i,j] = \sum_{p=-M}^{M} \sum_{q=-M}^{M} x[i + r \cdot p,\ j + r \cdot q] \cdot w[p,q]. \tag{4.3}$$

Standard convolution is the special case $r = 1$ [10]. Because $r-1$ zeros are inserted in each of the $K-1$ gaps between filter values, the effective filter size K' is:

$$K' = K + (K-1)(r-1). \tag{4.4}$$

Géron gives the one-dimensional case: the filter $[1,2,3]$ dilated with rate 4 becomes $[1,0,0,0,2,0,0,0,3]$, of length $3 + 2 \times 3 = 9$ [18, Chapter 12]. Likewise, a 3×3 filter with rate $r = 6$ covers a 13×13 region ($3 + 2 \times 5 = 13$) with 9 learnable parameters. Atrous convolution therefore enlarges the receptive field without extra parameters [18, Chapter 12] and without reducing the spatial resolution of the feature maps [10].

4.3.2 Atrous Spatial Pyramid Pooling (ASPP)

Objects and agricultural corridors appear at varying visual scales depending on camera mounting angle and distance from the vehicle. DeepLabV3 captures contextual information across multiple scales using the Atrous Spatial Pyramid Pooling (ASPP) module.

The ASPP module operates in parallel on the high-level feature map extracted by the backbone and consists of five branches:

1. One 1×1 standard convolution with 256 filters and batch normalization.
2. Three 3×3 atrous convolutions with 256 filters each, using dilation rates $r \in \{6, 12, 18\}$ for an output stride of 16 (or $r \in \{12, 24, 36\}$ for stride 8).
3. One Image-Level Pooling branch: to incorporate global image context, this branch computes global average pooling across the spatial dimensions of the input tensor, processes the resulting $1 \times 1 \times C_{\text{in}}$ vector with a 1×1 convolution (256 filters) and batch normalization, and upsamples the output bilinearly to the original ASPP spatial resolution.

Implementation note: The TorchVision `deeplabv3_resnet50` implementation [21] employs a dilated ResNet-50 backbone with output stride 8, achieved by removing downsampling in `layer3` and `layer4` and introducing atrous convolutions. The ASPP module correspondingly uses dilation rates $r \in \{12, 24, 36\}$ for the three parallel atrous branches.

The outputs of all five branches are concatenated along the channel dimension, resulting in a tensor with $5 \times 256 = 1280$ channels:

$$F_{\text{concat}} = [F_{1\times1},\ F_{\text{atrous},r_1},\ F_{\text{atrous},r_2},\ F_{\text{atrous},r_3},\ F_{\text{image_pool}}]\,. \tag{4.5}$$

This concatenated representation is projected back to 256 channels via a 1×1 convolution with batch normalization [10]; TorchVision follows it with ReLU activation and dropout ($p = 0.5$) [21]. The TorchVision implementation then applies a 3×3 convolution ($256 \to 256$ channels) with batch normalization and ReLU before the final 1×1 classification layer maps to C class logits. The output is upsampled via bilinear interpolation to image resolution $H \times W$. The ASPP parallel branch layout is depicted in Figure 4.2.

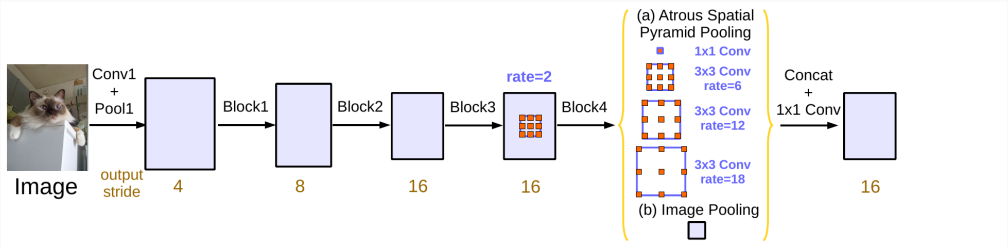


Figure 4.2: DeepLabV3 architecture featuring the Atrous Spatial Pyramid Pooling (ASPP) module with parallel atrous convolutions and image-level pooling (adapted from Chen et al. [10]).

4.4 MobileNetV3 and the Lite Reduced ASPP (LR-ASPP) head

DeepLabV3 with a ResNet-50 backbone provides high segmentation accuracy, but it is the slowest of the three networks in the offline comparison of Chapter 5 (86.90 ms mean recorded inference time). For mobile devices, Howard et al. [11] developed MobileNetV3, tuned to mobile phone CPUs, together with the Lite Reduced ASPP (LR-ASPP) segmentation head.

4.4.1 Depthwise separable convolutions

The building block that MobileNet architectures inherit from MobileNetV1 is the depthwise separable convolution [11], which factors a standard convolution into two separate steps: a spatial depthwise convolution and a channelwise pointwise convolution.

Consider an input feature map $X \in \mathbb{R}^{H \times W \times C_{\text{in}}}$ and an output feature map $Y \in \mathbb{R}^{H \times W \times C_{\text{out}}}$, processed with spatial filter size $D_K \times D_K$.

- **Standard convolution:** Each filter channel interacts with every input channel simultaneously. The computational cost, measured in Multiply-Accumulate (MAC) operations, is:

$$\text{Cost}_{\text{standard}} = H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot D_K^2. \tag{4.6}$$

- **Depthwise convolution:** Applies a single $D_K \times D_K$ spatial filter per input channel independently, without channel cross-talk:

$$\text{Cost}_{\text{depthwise}} = H \cdot W \cdot C_{\text{in}} \cdot D_K^2. \tag{4.7}$$

- **Pointwise convolution:** Applies C_{out} filters of size 1×1 across all channels to compute linear combinations of the depthwise outputs:

$$\text{Cost}_{\text{pointwise}} = H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}. \tag{4.8}$$

The total cost of a depthwise separable convolution is the sum of Eqs. (4.7) and (4.8):

$$\text{Cost}_{\text{sep}} = H \cdot W \cdot C_{\text{in}} \cdot D_K^2 + H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}. \tag{4.9}$$

Comparing Eq. (4.9) to Eq. (4.6) yields the computational reduction ratio:

$$\frac{\text{Cost}_{\text{sep}}}{\text{Cost}_{\text{standard}}} = \frac{H \cdot W \cdot C_{\text{in}} \cdot D_K^2 + H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}}}{H \cdot W \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot D_K^2} = \frac{1}{C_{\text{out}}} + \frac{1}{D_K^2}. \tag{4.10}$$

For 3×3 filters ($D_K = 3$), the ratio is $\frac{1}{64} + \frac{1}{9} \approx 0.13$ for $C_{\text{out}} = 64$ and approaches $\frac{1}{9} \approx 0.11$ as C_{out} grows. The parameter count follows the same ratio, since it equals the cost without the factor $H \cdot W$. Depthwise separable convolutions thus reduce arithmetic operations and parameters by about 87–89% relative to standard convolutions.

4.4.2 Inverted residual blocks with linear bottlenecks

In classical ResNet-50 bottleneck blocks, the input channels are first compressed via a 1×1 convolution, processed with a 3×3 convolution at reduced dimensionality, and

expanded back to the original high channel count (wide → narrow → wide). This structure reduces computational cost while maintaining representational capacity.

MobileNetV3 adopts the inverted residual block with linear bottleneck introduced by MobileNetV2, which reverses this design (narrow → wide → narrow) [11]:

1. **Expansion layer:** A 1×1 pointwise convolution expands the low-dimensional input C_{in} to a higher-dimensional space $e \cdot C_{\text{in}}$, where the expansion factor e varies between blocks (from 1 to 6 in MobileNetV3-Large [11, Table 1]).

2. **Depthwise layer:** A 3×3 or 5×5 depthwise convolution performs spatial filtering in the expanded channel space.

3. **Linear bottleneck:** A 1×1 projection layer compresses the channels back to C_{out}. No non-linear activation follows this projection: the narrow input and output of the block remain linear, and the non-linearities act only in the expanded space [11].

When the input and output have the same number of channels and the stride is 1, a residual shortcut connection adds the input directly to the block output, as illustrated in Figure 4.3a.

4.4.3 Squeeze-and-Excitation (SE) channel attention

To recalibrate channel responses, MobileNetV3 inserts Squeeze-and-Excitation (SE) modules in selected inverted residual blocks (8 of the 15 blocks of MobileNetV3-Large [11, Table 1]). The module is placed after the depthwise convolution, so that it acts on the expanded representation:

1. **Squeeze:** Global spatial average pooling aggregates the spatial dimensions of feature map $U \in \mathbb{R}^{H' \times W' \times C'}$ into a channel descriptor vector $z \in \mathbb{R}^{C'}$:

$$z_c = \frac{1}{H'W'} \sum_{i=1}^{H'} \sum_{j=1}^{W'} U_{i,j,c}. \tag{4.11}$$

2. **Excitation:** Two fully connected layers parameterized by weights $W_1 \in \mathbb{R}^{\frac{C'}{4} \times C'}$ and $W_2 \in \mathbb{R}^{C' \times \frac{C'}{4}}$ model non-linear channel relationships; the SE bottleneck is fixed to $1/4$ of the channels of the expansion layer [11]:

$$s = \sigma\left(W_2 \cdot \text{ReLU}(W_1 z)\right), \tag{4.12}$$

   where $\sigma(\cdot)$ denotes the hard-sigmoid activation function.

3. **Scale:** The input tensor U is rescaled channel-wise by the activation vector s:

$$\tilde{U}_{i,j,c} = s_c \cdot U_{i,j,c}, \qquad \forall c \in \{1, \ldots, C'\}. \tag{4.13}$$

4.4.4 Efficient non-linearities: hard-swish and hard-sigmoid

The swish non-linearity, defined as $\text{swish}(x) = x \cdot \sigma(x)$, improves the accuracy of neural networks when used as a drop-in replacement for ReLU. However, the sigmoid $\sigma(x) = (1 + e^{-x})^{-1}$ is much more expensive to compute on mobile devices and makes accuracy difficult to maintain in fixed-point arithmetic [11].

MobileNetV3 replaces standard sigmoid and swish with piece-wise linear hard approximations based on $\text{ReLU6}(x) = \min(\max(x, 0), 6)$:

$$\text{h-sigmoid}(x) = \frac{\text{ReLU6}(x+3)}{6}, \tag{4.14}$$

$$\text{h-swish}(x) = x \cdot \text{h-sigmoid}(x) = x \cdot \frac{\text{ReLU6}(x+3)}{6}. \tag{4.15}$$

Eqs. (4.14) and (4.15) contain no exponential. Howard et al. found no discernible difference in accuracy between the hard and smooth versions, while the piece-wise form reduces latency on mobile CPUs [11]. Because the cost of a non-linearity decreases as activation maps shrink, MobileNetV3 uses h-swish only in the second half of the network and ReLU in the earlier layers [11].

4.4.5 The Lite Reduced ASPP (LR-ASPP) architecture

The DeepLabV3 ASPP head applies four parallel convolutional branches with 256 channels each plus an image-pooling branch. Howard et al. proposed the Lite Reduced ASPP (LR-ASPP) head as a lighter alternative: it deploys global average pooling in a fashion similar to the Squeeze-and-Excitation module and uses only one 1×1 convolution in the pooling branch [11].

The LR-ASPP architecture uses features from two distinct stages of the MobileNetV3-Large backbone:

- A low-resolution, high-semantic feature map $F_{16} \in \mathbb{R}^{\frac{H}{16} \times \frac{W}{16} \times 960}$ extracted at output stride 16.
- A high-resolution, low-semantic feature map $F_8 \in \mathbb{R}^{\frac{H}{8} \times \frac{W}{8} \times 40}$ extracted at output stride 8.

The TorchVision LR-ASPP implementation (`lraspp_mobilenet_v3_large`) [22] processes the two feature streams as follows:

1. The stride-16 feature map F_{16} passes through two parallel branches:
   - **CBR path:** A 1×1 convolution followed by batch normalization and ReLU activation projects F_{16} to an intermediate representation $X \in \mathbb{R}^{\frac{H}{16} \times \frac{W}{16} \times C_{\text{inter}}}$, with $C_{\text{inter}} = 128$:

$$X = \text{ReLU}\left(\text{BN}\left(\text{Conv}_{1\times1}(F_{16})\right)\right). \tag{4.16}$$

- **Global context branch:** Applies global average pooling followed by a 1×1 convolution and a standard sigmoid activation to produce a channel-wise gating vector $G \in \mathbb{R}^{1\times1\times C_{\text{inter}}}$:

$$G = \sigma\left(\text{Conv}_{1\times1}\left(\text{GAP}(F_{16})\right)\right). \tag{4.17}$$

2. The CBR output X is modulated element-wise by the gating vector G and then upsampled by a factor of 2 via bilinear interpolation:

$$M_{16} = \text{BilinearUpsample}_{2\times}(X \odot G). \tag{4.18}$$

3. The upsampled context features M_{16} and the stride-8 features F_8 pass through **separate** 1×1 classification layers to produce class logits:

$$Z_{16} = \text{Conv}^{(16)}_{1\times1}(M_{16}), \qquad Z_8 = \text{Conv}^{(8)}_{1\times1}(F_8). \tag{4.19}$$

4. The two logit maps are **summed in logit space** to produce the final stride-8 prediction:

$$Z_{\text{fused}} = Z_{16} + Z_8. \tag{4.20}$$

5. A final $8\times$ bilinear upsampling step restores the prediction to the original image dimensions $H \times W$.

Implementation note: Unlike architectures that fuse feature representations before classification, LR-ASPP fuses predictions in logit space. This design allows each branch to learn independent classification boundaries while combining their complementary spatial resolutions. The global context gating uses a standard sigmoid activation (Eq. (4.17)), not the hard-sigmoid approximation employed in the MobileNetV3 inverted residual blocks.

By replacing the multi-branch parallel ASPP structure with a single global pooling gate and logit-space fusion, LR-ASPP reduces computational operations while retaining global scene context and boundary localization. The inverted residual backbone block and the complete LR-ASPP segmentation topology are illustrated in Figure 4.3.

4.5 Segment Anything Model 3 (SAM 3) for dataset annotation

Manually annotating agricultural images at pixel precision requires drawing polygons around irregular plant boundaries, inter-row soil, and covering material. The time required for this labeling motivated the use of SAM 3 to generate training

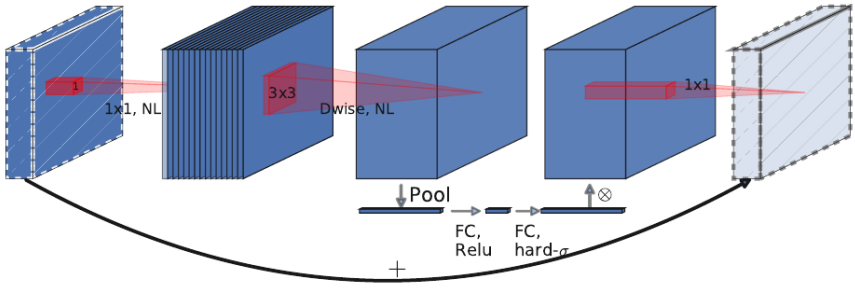


Figure 4. MobileNetV2 + Squeeze-and-Excite [20]. In contrast with [20] we apply the squeeze and excite in the residual layer. We use different nonlinearity depending on the layer, see section 5.2 for details.

(a) MobileNetV3 Inverted Residual block with Squeeze-and-Excitation channel attention.

V3-Small†	16.1	43	1.77	0.16

Table 6. Object detection results of SSDLite with different backbones on COCO test set. †: Channels in the blocks between $C4$ and $C5$ are reduced by a factor of 2.

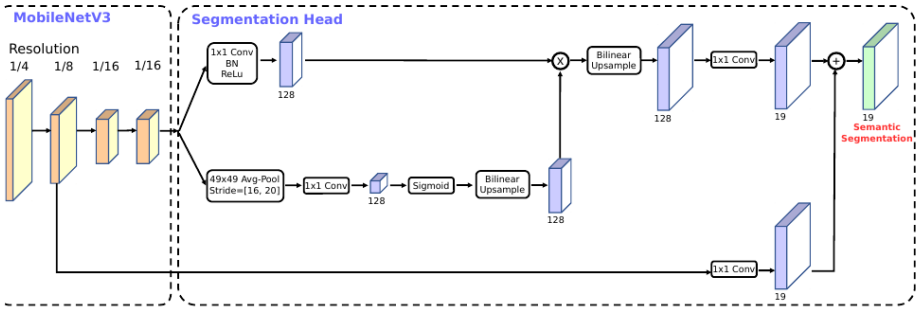


Figure 10. Building on MobileNetV3, the proposed segmentation head, Lite R-ASPP, delivers fast semantic segmentation results while mixing features from multiple resolutions.

(b) Lite Reduced ASPP (LR-ASPP) segmentation head mixing low- and high-resolution features.

Figure 4.3: MobileNetV3 and Lite Reduced ASPP (LR-ASPP) architecture: (a) inverted residual block combining depthwise separable convolution, linear bottleneck, and Squeeze-and-Excitation channel attention; (b) segmentation head combining stride-16 features modulated by global context gating with stride-8 fine features (adapted from Howard et al. [11]).

labels automatically (Chapter 5). Foundation models make it possible to generate candidate masks from prompts without training on the target crops.

Kirillov et al. [12] established the promptable segmentation paradigm with SAM, using Vision Transformers to generate binary segmentation masks from spatial prompts such as foreground/background points, bounding boxes, or coarse masks. Carion et al. [13] extended this paradigm in SAM 3 by introducing open-vocabulary concept segmentation driven by textual descriptions and visual exemplars.

4.5.1 Architecture overview

Unlike the original SAM architecture, which processes images through a single ViT encoder followed by a prompt-conditioned mask decoder, SAM 3 adopts a dual-architecture design comprising a **detector** for image-level segmentation and a **tracker** for video sequences. Both components share a unified **Perception Encoder (PE)** backbone that processes visual and linguistic inputs.

The SAM 3 detector follows a DETR-based paradigm with the following key components:

1. **Perception Encoder:** A shared vision-language backbone that encodes images into spatial feature representations. Unlike SAM's standalone ViT encoder, the PE is designed to handle both visual data and textual concept embeddings.

2. **Fusion Encoder:** Accepts unconditioned image embeddings from the PE and conditions them by cross-attending to prompt tokens derived from text descriptions (e.g., the prompts `"plants"`, `"central farm path"`, and `"plastic mulch film"` used in Chapter 5) or visual exemplars (bounding boxes around reference instances).

3. **Presence Token:** A learned global token, processed in the decoder together with the object queries, that decouples recognition from localization. It predicts only whether the prompted concept is present in the image, while each object query solves the localization problem; the final score of each query is the product of its own score and the presence score. Carion et al. report that this decomposition reduces the detector's sensitivity to false positives [13].

4. **Decoder:** Learned object queries cross-attend to the conditioned image embeddings from the fusion encoder, producing a set of binary segmentation masks and associated confidence scores for all instances of the target concept.

For video input, SAM 3 extends this architecture with a memory-augmented tracker that shares the frozen PE backbone with the detector. The tracker incorporates a memory encoder and memory bank that store object appearance across frames, enabling consistent temporal segmentation.

Figure 4.4 illustrates the original SAM architecture, which established the foundation for prompt-based segmentation; SAM 3 extends this paradigm with text-driven concept detection and video tracking capabilities.

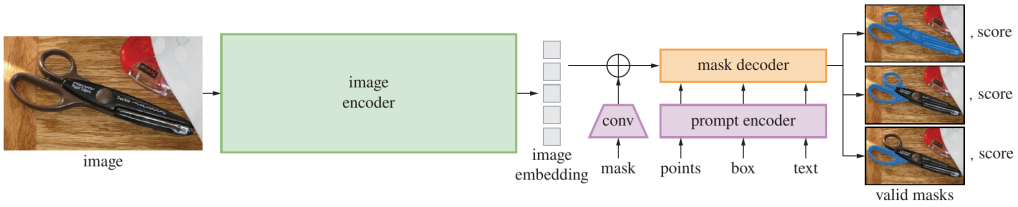


Figure 4.4: Original Segment Anything Model (SAM) architecture with ViT image encoder, prompt encoder, and mask decoder (adapted from Kirillov et al. [12]). SAM 3 extends this foundation with a dual detector-tracker architecture and open-vocabulary concept segmentation.

4.5.2 Operational distinction from onboard models

A key architectural distinction must be emphasized between the foundation model and the robotic segmentation network:

- SAM 3 has about 850 million parameters, of which about 450 million belong to the vision encoder and 300 million to the text encoder [13]. Its reported speed of 30 ms per image refers to an NVIDIA H200 data-center GPU [13], a hardware class not available on the robot, and the model was not evaluated on the onboard computer in this work.
- In this thesis, SAM 3 functions strictly as an **offline supervision generator**. Given text prompts representing crop and corridor concepts, SAM 3 generates dense candidate masks across lettuce, tomato, and cucumber image collections.
- The resulting masks are used without manual correction as training targets for LR-ASPP, the network selected for onboard perception after the comparison on the manually annotated lettuce dataset (Chapter 5). LR-ASPP recorded a mean offline inference time of 8.21 ms in that comparison.

4.6 Summary of architectural characteristics

Table 4.1 summarizes the characteristics of the three segmentation networks and of SAM 3 analyzed in this chapter, establishing the foundation for the dataset experiments and comparative evaluations presented in Chapter 5.

Table 4.1: Comparison of deep learning architectures used in this thesis. Recorded inference times are reported in Chapter 5.

Model	Backbone	Context mechanism	Target role	Primary characteristics
FCN	Dilated ResNet-50	Single-scale `FCNHead` on stride-8 features, auxiliary classifier during training	Benchmark segmentation	High parameter count, no multi-scale context module
DeepLabV3	Dilated ResNet-50	Multi-scale ASPP on stride-8 features	High-accuracy benchmark	Longest recorded inference time, multi-scale context
LR-ASPP	MobileNetV3-Large	Gated global pooling with logit-space fusion	Onboard segmentation for navigation	Compact architecture designed for mobile devices
SAM 3	Perception Encoder (vision-language)	DETR-based detector with presence token and text/visual prompts	Offline automatic annotation	Foundation model for open-vocabulary concept segmentation; not deployed onboard

Chapter 5

Dataset Preparation and Comparative Model Training

Following the theoretical analysis of deep learning architectures in Chapter 4, this chapter describes the datasets, annotation pipelines, and experimental model evaluations that guide the configuration for robotic integration.

The experimental study began by fine-tuning FCN, DeepLabV3, and LR-ASPP from pretrained weights on a manually annotated lettuce dataset for an empirical comparison. Because manual segmentation was time-consuming, this initial training was limited to lettuce rather than repeated across every target crop. Its outcome led to the adoption of LR-ASPP for embedded deployment.

The second experiment, carried out after the selection of LR-ASPP, tests whether networks trained on masks generated automatically by SAM 3, without any manual correction, can replace manual annotation. Manual masks were available for lettuce and were more accurate, but the aim was to evaluate labels that require no human work.

The experiment uses only lettuce images, because lettuce is the only crop with manual ground truth. SAM 3 generated three label sets for these images, with four, three, and two classes (Section 5.1.2), and a separate LR-ASPP network was trained on each. The predictions of each network are compared with the manual lettuce masks on the classes the two label sets share. The path class matters most, because it is the only one used for row following. The SAM 3 masks themselves are not compared with the manual masks.

The binary labeling was then extended to the complete image collection, labeled entirely by SAM 3, and a network trained on this collection is the one deployed on the robot. The results of this configuration are reported in Table 5.7.

This chapter details image acquisition, dataset partitioning, training parameters, and offline benchmark evaluations. Subsequent chapters detail geometric navigation

extraction and closed-loop robot validation.

5.1 Image acquisition and class definition

5.1.1 Image sources and acquisition conditions

[SPAZIO FIGURA / SCHEMA]

Sample field images across crops and corresponding training masks

Esempi visivi delle acquisizioni di campo per lattuga, pomodoro e cetriolo, con le corrispondenti maschere di annotazione impiegate per l'addestramento (ground truth manuale per la lattuga, maschere binarie del path autogenerate da SAM 3 per pomodoro e cetriolo).

Figure 5.1: Sample field images across crops and corresponding training masks

▶ **DA COMPLETARE: Origine delle immagini e condizioni di acquisizione**

Specificare la sorgente dei dati e le modalità di raccolta:

- Dispositivo di ripresa utilizzato (risoluzione e ottica), quota di acquisizione e angolazione.
- Condizioni operative nei campi (aziende agricole o serre sperimentali, condizioni meteo e fasce orarie).
- Numero complessivo di immagini raccolte per lattuga, pomodoro e cetriolo.

5.1.2 Semantic classes and labeling targets

The segmentation labels in this thesis use up to four semantic classes: background, plants, path (the traversable corridor between two crop rows), and covering material (plastic mulch film). The manual lettuce annotations use all four classes (Section 5.2.1).

The labels generated with SAM 3 (Section 5.2.2) follow three configurations, each used to train a separate network:

- **Four-class configuration:** background, plants, path, and covering material, matching the four classes of the manual lettuce annotations.

- **Three-class configuration:** background, plants, and path. No prompt targets the covering material.
- **Binary configuration:** background and path. This configuration is used both for the lettuce-only binary network and for the multi-crop dataset.

For row following, the path mask is the input of the geometric extraction in Chapter 6.

5.2 Dataset preparation

5.2.1 Lettuce dataset with manual annotation

The manually annotated lettuce dataset, identified as `Data`, contains 762 image–mask pairs distributed across eight folders. Its masks were prepared from the manual annotations stored in `annotations.json`. The same manual reference is used for the initial architecture comparison and for evaluating the models trained with SAM 3-generated labels. These manual masks are the ground truth for all evaluations in this chapter. They contain four classes (background, plants, path, and covering material), and the path class marks only the corridor the robot should follow, not other visible paths. The three lettuce autolabel datasets (`Data_insalataSAM_4class`, `Data_insalataSAM_3class`, and `Data_insalataSAM_2class`) contain images from the same eight lettuce folders, with masks generated by SAM 3 instead of the manual annotations.

5.2.2 Automated zero-shot annotation via SAM 3

Manual segmentation requires substantial human effort, which motivated testing whether SAM 3 can replace it. The experiment was carried out without any manual work: SAM 3 was run offline with text prompts, and its masks were used for training without inspection, filtering, or correction. Manual masks were available for lettuce and were more accurate, but keeping the unedited masks was an intentional choice, to measure how well a network performs when the annotation step is fully automatic.

All autolabel masks were generated using `SAM3SemanticPredictor` with the local `sam3.pt` checkpoint and a confidence threshold of 0.25. For each text prompt, SAM 3 returned zero or more candidate masks, and pixels with mask values greater than 0.5 were assigned to the corresponding class. When masks of different prompts overlapped, the class was assigned according to a fixed prompt order, listed below for each dataset: later prompts were written only on pixels still labeled as background, so earlier classes were never overwritten. Pixels not assigned by any prompt remained background. The prompts and their order are:

- `Data_insalataSAM_4class` (lettuce images): `plants` (class 1), then `plastic mulch film` (class 3), then `central farm path` (class 2)
- `Data_insalataSAM_3class` (lettuce images): `plants` (class 1), then `central farm path` (class 2)
- `Data_insalataSAM_2class` (lettuce images): `central farm path` only (class 1)
- `Data_allPlantsSAM2` (images of lettuce, tomato, and cucumber crops): `central path` only (class 1).

The SAM 3 masks are not compared directly with the manual annotations. Their quality enters the evaluation only through the networks trained on them (Section 5.4.1).



Figure 5.2: SAM 3 zero-shot autolabeling and mask composition workflow

5.2.3 Tomato and cucumber images

In addition to lettuce, the image collection includes tomato and cucumber crops. These images were annotated exclusively with SAM 3 and are included only in the multi-crop dataset (`Data_allPlantsSAM2`) used to train the binary network deployed on the robot. They were not included in the training sets of the lettuce-specific models.

Because no manual ground truth was created for tomato and cucumber, these crops are not part of the quantitative evaluations in Section 5.4. Similarly, the initial comparison among architectures (FCN, DeepLabV3, and LR-ASPP) was carried out exclusively on the lettuce dataset.

5.2.4 Dataset partitioning

The lettuce training runs use an 80% training, 10% validation, and 10% test partition. For the 762 image–mask pairs, this yields 609 training, 76 validation, and 77 test samples.

The training split is used to update the network weights. The validation split is used to compute the loss at the end of each epoch and to select the best checkpoint. The test split is not used during training and is used only afterwards, to compute the final scores. The three lettuce autolabel datasets contain the same 762 images as `Data`, in the same order, and the three autolabel projects use a split file identical to that of the manual model. The 77 test images are therefore the same physical images for all four lettuce networks, and none of them belongs to the 609 training images of any of these networks. The multi-crop binary network evaluated in Section 5.4 is trained on the complete image collection (`Data_allPlantsSAM2`, with images of lettuce, tomato, and cucumber crops). Before splitting this collection, the 77 lettuce test images were removed from it, so that none of them is used for training. The remaining 1882 of 1959 image–mask pairs were then split 80/10/10.

The split is random at the level of individual images: no test image is used for training, but each lettuce folder, which corresponds to one acquisition video recorded along a different path, contributes images to both the training and the test split. During training, the networks therefore see other photos taken along the same paths as the test images. The reported IoU values may overestimate performance on paths or fields not seen during training. This limitation applies to all networks of the comparison.

The realistic scenario uses only the test images from `insalata_grande1`, `insalata_grande2`, and `insalata_media1`: 42 of the 77 test images. These folders were recorded between parallel rows, in the area where the robot operates, from viewpoints the robot can reach. The other five folders were recorded from viewpoints the robot cannot reach and can show more than one path, while the manual masks label only the path the robot should follow. The realistic scenario is therefore a subset of the test split, not a separate test set, and its results describe only the robot's operating conditions.

5.3 Architecture comparison: FCN, DeepLabV3, and LR-ASPP

This section selects the segmentation architecture used in the rest of the thesis. FCN, DeepLabV3, and LR-ASPP were trained and compared on the manually annotated lettuce dataset.

5.3.1 Model selection and pretrained weights

FCN, DeepLabV3, and LR-ASPP were chosen from the implementations available in TorchVision. Their selection was practical, rather than the outcome of a comparison with every available architecture.

The implementation separates each network into a backbone, which extracts image features, and a segmentation head, which produces class scores at each output location. Transfer learning is implemented by loading pretrained parameters and adapting the output layers to the target labeling task, rather than initializing every network parameter from scratch.

For FCN and DeepLabV3, the selected implementations use a ResNet-50 backbone with pretrained segmentation weights. After these weights are loaded, the final convolution of the main classifier is replaced by a new 1×1 convolution with one output channel per target class (4 channels). The final convolution of the auxiliary classifier is replaced in the same way.

LR-ASPP is initialized in the same way. The implementation loads the default pretrained weights of the complete LR-ASPP model with a MobileNetV3-Large backbone. Its segmentation head ends with two 1×1 convolutions, one applied to the low-level features and one to the high-level features [22]. Both are replaced by new convolutions with 4 output channels. LR-ASPP has no auxiliary classifier.

The default weights of the three models were trained on a subset of COCO restricted to the 20 categories of Pascal VOC [20, 21, 22]. All three networks therefore start from pretrained segmentation weights of the same origin, and only their final output convolutions are newly initialized.

5.3.2 Training configuration and loss formulation

The optimization pipeline addresses class imbalance and follows a two-stage transfer learning schedule for all evaluated architectures.

Loss function and class weighting

Agricultural scenes exhibit a strong pixel imbalance between the navigable soil corridor, the plant canopy, and background areas. To prevent the optimization from biasing toward dominant categories, the models are trained with a weighted cross-entropy loss, denoted $\mathcal{L}_{\mathrm{WCE}}$, in which the contribution of each pixel is multiplied by the weight w_c of its ground-truth class c, defined in Eq. (5.1).

Class weights are computed exclusively on the training split $\mathcal{D}_{\mathrm{train}}$ to avoid information leakage from validation and test partitions. For each class c, the total pixel count $N_c = \sum_{k \in \mathcal{D}_{\mathrm{train}}} \sum_{i,j} \mathbb{I}(Y_{i,j}^{(k)} = c)$ is accumulated across all training masks. The balanced inverse frequency inv_c is then normalized by the mean of all inverse

frequencies:

$$\mathrm{inv}_c = \frac{\sum_{k=0}^{C-1} N_k}{C \cdot N_c}, \qquad w_c = \frac{\mathrm{inv}_c}{\frac{1}{C}\sum_{k=0}^{C-1} \mathrm{inv}_k}. \tag{5.1}$$

where C is the number of classes (four in this comparison), k indexes the training masks, $Y_{i,j}^{(k)}$ is the class of pixel (i, j) in mask k, and $\mathbb{I}(\cdot)$ equals 1 when its argument is true and 0 otherwise. A rare class therefore receives a large weight and a frequent class a small one. This normalization enforces $\frac{1}{C}\sum_{c=0}^{C-1} w_c = 1.0$.

For FCN and DeepLabV3, which incorporate an auxiliary classifier on the `layer3` features (output stride 8 in the dilated TorchVision backbone), the total training objective combines the primary and auxiliary losses with an auxiliary weighting factor of 0.4:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{WCE}}(\text{main}) + 0.4 \cdot \mathcal{L}_{\text{WCE}}(\text{aux}). \tag{5.2}$$

For LR-ASPP, which lacks an auxiliary head, training relies solely on the primary loss $\mathcal{L}_{\text{WCE}}(\text{main})$.

Data augmentation and two-stage optimization

Data augmentation is applied only to the training subset: random horizontal flips with probability 0.5, random rotations between $-10°$ and $10°$, and, with probability 0.8, a color jitter of brightness and contrast (factors in $[0.8, 1.2]$) and saturation (factors in $[0.85, 1.15]$) to handle variable outdoor illumination. The validation and test sets are not augmented. Images are not resized, and all images are normalized with the ImageNet mean and standard deviation.

The networks were trained on an NVIDIA GeForce RTX 2080 SUPER with 8 GB of memory, with a batch size of 2.

Optimization proceeds across 200 epochs in two distinct phases:

1. **Stage 1 (Warmup, Epochs 1–10):** Backbone parameters are frozen by setting `requires_grad = False`. Only the segmentation heads (and auxiliary classifiers when present) are optimized using AdamW with an initial learning rate of 10^{-3} and weight decay of 10^{-4}. A cosine-annealing scheduler with $T_{\max} = 200$ and $\eta_{\min} = 10^{-6}$ updates the learning rate after each epoch. In the first epochs, the newly initialized output layers make large errors, and the resulting large gradients could damage the pretrained weights. Freezing the reused layers during these epochs gives the new layers time to learn reasonable weights [18, Chapter 11].

2. **Stage 2 (Differential Fine-Tuning, Epochs 11–200):** At epoch 10, the backbone is unfrozen. A new AdamW optimizer is instantiated with decoupled parameter groups: the backbone is assigned a learning rate of 10^{-5} and the

segmentation heads a learning rate of 10^{-4}. The lower learning rate of the unfrozen backbone limits changes to its pretrained weights [18, Chapter 11]. The cosine-annealing scheduler is re-initialized across the remaining 190 epochs ($T_{\max} = 190, \eta_{\min} = 10^{-6}$).

Early stopping was not used: all three networks were trained for the full 200 epochs, so that the complete training and validation loss curves could be recorded. After each epoch, the model is evaluated on the unaugmented validation set. The weights are saved whenever the validation loss falls below its previous minimum, and the weights of the final epoch are saved separately.

5.3.3 Evaluation metrics and results

The three networks are evaluated on the 77 test images. The IoU is computed for each of the four classes, and the mIoU is their arithmetic mean, including background. The inference time is measured on the NVIDIA GeForce RTX 2080 SUPER used for training, one image at a time at its original resolution, and covers the forward pass and the argmax. The epoch column of Table 5.1 gives the epoch with the lowest validation loss, whose weights are evaluated.

FCN and DeepLabV3 reach the same mIoU (0.9253), 1.08 percentage points above LR-ASPP, and LR-ASPP has the lowest IoU in every class (Table 5.2). LR-ASPP is 7.2 times faster than FCN and 10.6 times faster than DeepLabV3.

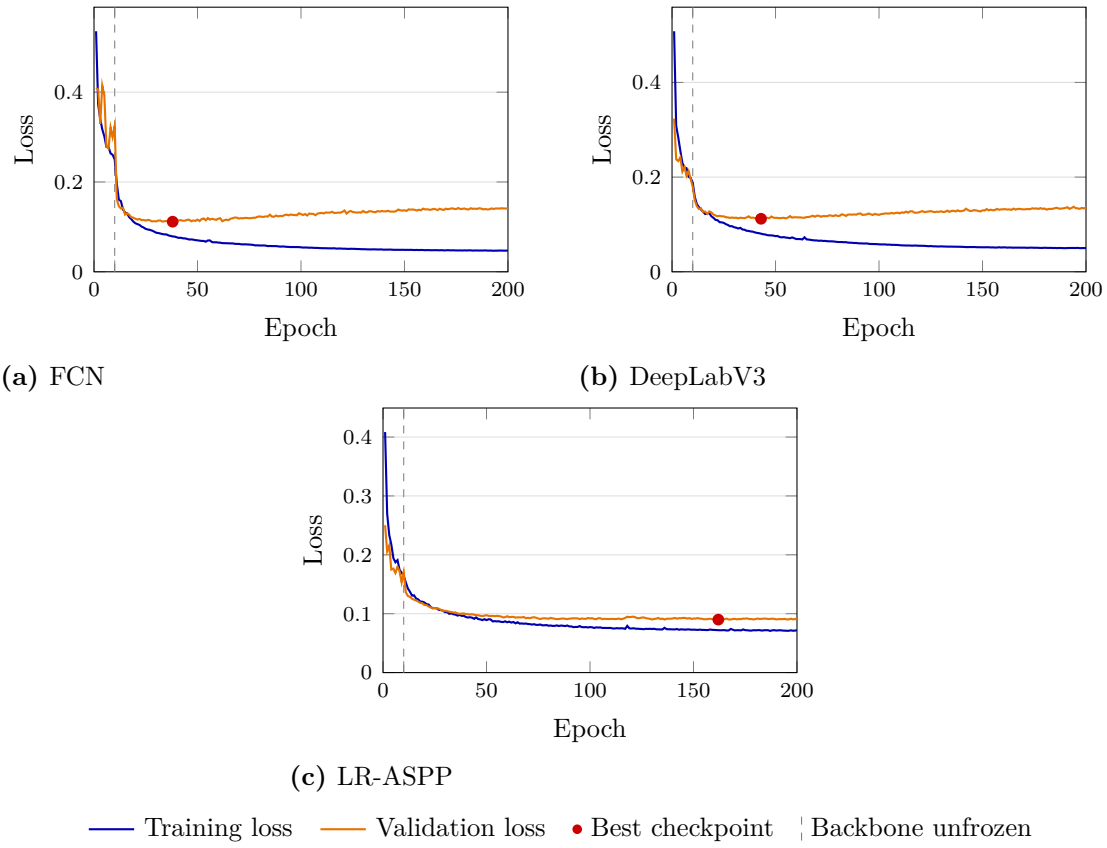
For FCN and DeepLabV3, the validation loss reaches its minimum early (Figure 5.3). After that, both networks start to overfit, and their validation loss rises while the training loss keeps decreasing. For LR-ASPP, the validation loss reaches a plateau around epoch 75 and does not rise until epoch 200. The losses of FCN and DeepLabV3 include the auxiliary term of Eq. (5.2), while those of LR-ASPP do not, so the absolute values are not comparable across networks.

Table 5.1: Architecture comparison on the 77 test images of the manual lettuce dataset. Parameter counts refer to the trained four-class models and include the auxiliary classifier of FCN and DeepLabV3, which is used only during training.

Model	Parameters (M)	Epoch	mIoU	Path IoU	Mean time (ms)	FPS
FCN	35.3	38	0.9253	0.9654	59.10	16.92
DeepLabV3	42.0	43	0.9253	0.9686	86.90	11.51
LR-ASPP	3.2	162	0.9145	0.9532	8.21	121.87

Table 5.2: Per-class IoU on the same 77 test images.

Model	Background	Plants	Path	Covering material
FCN	0.9493	0.9385	0.9654	0.8479
DeepLabV3	0.9494	0.9384	0.9686	0.8448
LR-ASPP	0.9439	0.9262	0.9532	0.8346



(a) FCN **(b)** DeepLabV3

(c) LR-ASPP

Figure 5.3: Training and validation loss per epoch of FCN, DeepLabV3, and LR-ASPP on the manual lettuce dataset. The best checkpoint is the epoch with the minimum validation loss. The backbone is unfrozen after epoch 10.

5.3.4 Selection of LR-ASPP

LR-ASPP was selected mainly for its speed. It was not the most accurate model, but its much shorter inference time offered the accuracy–latency compromise sought for robot perception. The subsequent SAM-supervised training runs therefore used LR-ASPP while varying the output label set.

These times were measured on the training PC, not on the robot. The timing of the complete perception and control pipeline on the robot is reported in Section 7.3.1.

5.4 Manual versus SAM 3 labels

5.4.1 Training configurations and evaluation

Table 5.3 summarizes the four LR-ASPP networks considered in this comparison. The manual baseline is trained on the manual labels. The other three networks are trained on SAM 3 labels and are called SAM 3-supervised networks below. This study is separate from the initial comparison of the three candidate architectures. All four networks are trained on lettuce images only, so that their predictions can be evaluated against the manual ground truth of the same crop. The extension of the binary configuration to the multi-crop dataset is described in Section 5.4.3.

Table 5.3: LR-ASPP training configurations in the supervision comparison. All networks are trained on the same lettuce images and evaluated against the manual lettuce ground truth. The epoch is that of the evaluated weights, those with the lowest validation loss.

Configuration	Labels	Output classes	Epoch
Manual baseline	Manual	Background, plants, path, covering material	162
SAM 3-supervised four-class	SAM 3	Background, plants, path, covering material	30
SAM 3-supervised three-class	SAM 3	Background, plants, path	38
SAM 3-supervised binary	SAM 3	Background, path	18

The SAM 3-supervised networks reach their minimum validation loss much earlier than the manual baseline, between epochs 18 and 38 against epoch 162. A possible explanation, not verified in this work, is the noise in the SAM 3 labels, which are also the labels of the validation split of these networks. Arpit et al. [23] observed that, when part of the training labels is random, a network first learns the patterns shared by the real data and only then fits the noisy labels, so its validation

accuracy reaches its maximum before the training accuracy becomes high. Their experiments concern image classification with randomly corrupted labels, while the SAM 3 label errors observed in this work are not random but depend on the prompts and on the scene, so their result supports this explanation only as an analogy.

Each network is evaluated with the IoU between its predictions and the manual lettuce labels of the test images:

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \tag{5.3}$$

where TP_c, FP_c, and FN_c are the true positive, false positive, and false negative pixels of class c. The mIoU is the arithmetic mean of the IoU of the evaluated classes.

Performance is evaluated on two subsets of the 77-image test partition:

- **Complete test set (77 images):** Spans all eight lettuce acquisition folders, including viewpoints that the robot cannot reach.
- **Realistic operational scenario (42 images):** Restricts evaluation to the test images of the three folders recorded between parallel rows, described in Section 5.2.4.

5.4.2 Results on the lettuce test images

All results are computed on the test split, both on the complete test set and in the realistic scenario. Each SAM 3-supervised network is compared with the manual baseline on the classes the two networks share. The background is not compared for the three-class and binary networks. In the three-class labels it also contains the covering material, and in the binary labels it also contains the plants and the covering material, so it does not match the background class of the manual masks. Table 5.4 compares the four-class networks on all classes, Table 5.5 compares the plants and path classes of the three-class network, and Table 5.6 compares the path class of the binary network.

Table 5.4: IoU of the four-class networks on the test set.

Scenario	Model	Background	Plants	Path	Covering material	mIoU
Complete test set	Manual baseline	0.9439	0.9262	0.9532	0.8346	0.9145
	SAM 3-supervised	0.4631	0.8081	0.4124	0.1719	0.4639
Realistic scenario	Manual baseline	0.9203	0.9485	0.9587	0.8178	0.9113
	SAM 3-supervised	0.3449	0.8757	0.8997	0.2495	0.5925

Table 5.5: IoU of the three-class network on plants and path. The mean is computed over these two classes only.

Scenario	Model	Plants IoU	Path IoU	Mean IoU
Complete test set	Manual baseline	0.9262	0.9532	0.9397
	SAM 3-supervised	0.8130	0.4124	0.6127
Realistic scenario	Manual baseline	0.9485	0.9587	0.9536
	SAM 3-supervised	0.8779	0.8966	0.8873

Table 5.6: Path IoU of the binary network trained on lettuce only.

Scenario	Model	Path IoU
Complete test set	Manual baseline	0.9532
	SAM 3-supervised	0.3917
Realistic scenario	Manual baseline	0.9587
	SAM 3-supervised	0.9013

In the four-class network, the IoU of the covering material and of the background is low. Visual inspection of the SAM 3 masks showed that SAM 3 segmented the covering material poorly in general and also assigned the greenhouse plastic cover to this class, which is background in the manual masks. This explanation comes from visual inspection only. With an mIoU of 0.5925 in the realistic scenario, against 0.9113 for the manual baseline, the four-class SAM 3 labels do not replace manual annotation when all four classes are needed.

All SAM 3-supervised networks lose much of their path accuracy on the complete test set (path IoU 0.39–0.41), while the manual model does not (0.9532). The drop comes from the five folders outside the operational scenario, whose images were taken from viewpoints the robot cannot reach. In these images the manual masks label only the path the robot should follow. Visual inspection showed that the SAM 3 path labels also cover other parts of the ground, such as adjacent paths and soil outside the corridor, especially where the path between the rows is not clearly visible. The networks learn this wider path class, and the IoU counts the additional ground pixels as false positives. The complete-test values therefore combine segmentation errors and this difference between the SAM 3 and manual labels. The two effects were not separated quantitatively.

In the realistic scenario, the path IoU of the three SAM 3-supervised networks is between 0.8966 and 0.9013, 5.7 to 6.2 percentage points below the manual baseline (0.9587). The three SAM 3-supervised networks differ from each other by less than 0.5 percentage points, so these data do not show that removing the plant and

covering material classes improves path segmentation. In any case, the geometric extraction uses only the path class (Chapter 6), so the binary configuration was chosen for the robot. Whether its accuracy is sufficient for row following is assessed in simulation and field tests (Chapter 8).

5.4.3 Extension to the multi-crop dataset and deployed network

After the lettuce study, the binary labeling was extended to the complete image collection, with images of lettuce, tomato, and cucumber crops, labeled entirely by SAM 3 with the `central path` prompt and without manual correction. The binary LR-ASPP network trained on this collection is the one deployed on the robot and used in the following chapters (Chapters 7 and 8).

The training data of the deployed network include the lettuce test images, so its IoU on them would not be an independent measure. A second network with the same configuration was therefore trained on the collection without these images (Table 5.7). Its results estimate the accuracy of the configuration, not of the exact deployed weights. On the complete test set it is clearly better than the lettuce-only binary network (0.6369 against 0.3917).

Table 5.7: Path IoU of the multi-crop binary network, trained without the lettuce test images.

Scenario	Model	Path IoU
Complete test set	Manual baseline	0.9532
	SAM 3-supervised binary, multi-crop	0.6369
Realistic scenario	Manual baseline	0.9587
	SAM 3-supervised binary, multi-crop	0.9071

Chapter 6

Extraction of the Local Navigation Reference

This chapter describes five path extractors, two of which were developed in this work. The first, **Strip-IRLS**, is the main one. It extracts the path intervals in horizontal strips of the predicted mask. Going upward from the bottom of the image, it links in each strip the interval that continues the corridor found below, and it keeps the chain that gives the most edge samples. It then fits the left and right edges of the corridor separately with iteratively reweighted least squares. **EdgeNet**, still at an early stage, is a small network that takes the same mask and regresses the two corridor edges directly. The other three are geometric baselines: **Center points**, which fits one line to the midpoints of the intervals, **Column histogram**, which takes the column with the most path pixels, and **RANSAC edges**, which fits the two edges with RANSAC on the row extremes of the corridor. All five extractors take the same input mask and return the same outputs, a straight centerline, a confidence and a status, `ok`, `degraded` or `lost`, so the controller node can run any of them. The controller does not use the centerline of a `lost` result (Chapter 7). The extractor is chosen at start-up with a ROS parameter. All five were tested on the robot. Section 6.1 defines the input mask and the line models, Sections 6.2 to 6.4 describe the extractors, and Section 6.5 compares their structure and lists the limitations of **Strip-IRLS**.

6.1 Input mask and line models

All five extractors take the mask M predicted by the deployed binary network (Chapter 5), of height H and width W, with x increasing to the right and y downward. On the robot, the 640×480 camera image is resized to 224×224 pixels before the segmentation network, so $H = W = 224$ (Chapter 7). Path pixels have

value 1 and all other pixels have value 0. Lines are written as functions of y, which avoids infinite slopes for vertical lines. All distances are in pixels of the input mask.

Strip-IRLS, *EdgeNet* and *RANSAC edges* model the two corridor edges as separate straight lines, each with its own slope and intercept,

$$x_L(y) = m_L y + q_L, \qquad x_R(y) = m_R y + q_R, \tag{6.1}$$

and take the centerline as their mean,

$$x_C(y) = \frac{x_L(y) + x_R(y)}{2} = m_C y + q_C, \qquad m_C = \frac{m_L + m_R}{2}, \quad q_C = \frac{q_L + q_R}{2}. \tag{6.2}$$

The edges need not be parallel, so the modeled width can decrease toward the top of the image because of perspective. The slopes and intercepts are not constrained, so the lines can follow the corridor for any position and heading of the robot inside the row. *Strip-IRLS* then rejects the edges that do not converge upward (Section 6.2.5). *Center points* fits the centerline $x_C(y)$ directly to the midpoints of the corridor in each strip, without the edges. *Column histogram* returns a vertical centerline, with $m_C = 0$.

6.2 Strip-IRLS

Strip-IRLS works in five stages:

- Select a lower region of interest and clean the path mask (Section 6.2.1).
- Extract the path intervals in horizontal strips (Section 6.2.2).
- Associate intervals upward from each candidate seed and keep the best chain (Section 6.2.3).
- Fit the two edges separately (Section 6.2.4).
- Validate the edge geometry, assign a status from a confidence score, and return the centerline (Section 6.2.5).

6.2.1 Region of interest and mask cleaning

Like the three baselines of Section 6.4, *Strip-IRLS* uses only the lower part of the mask, the region of interest (ROI). The upper rows, where the path is not visible or cannot be distinguished, are excluded (Figure 6.1). The ROI starts at row $\lfloor \rho H \rfloor$ and ends at the bottom of the image. The ratio ρ must be chosen based on the position of the camera and on the path, which together set the highest row where the path appears in the image. The ROI should contain the path and as few other

rows as possible (Section 6.2.6). On the robot, $\rho = 0.30$, chosen on the development data (Section 8.1.2), gives row 67. When needed, *Strip-IRLS* extends the ROI slightly upward, so that it is filled exactly by strips of equal height (Section 6.2.2). On the robot, the ROI is extended from row 67 to row 64.

The path pixels inside the ROI are cleaned before the intervals are extracted. An opening with a 5×5 elliptical kernel removes thin protrusions and small isolated regions, then a closing with a 9×9 rectangular kernel fills small holes and gaps. The three baselines apply the same opening and closing. Finally, the connected components with fewer than 500 pixels are removed. In Figure 6.1, the dark gray regions inside the ROI are the pixels removed by the cleaning. Only the remaining path pixels, the cleaned mask $\widetilde{M}$, are passed to the interval extraction.

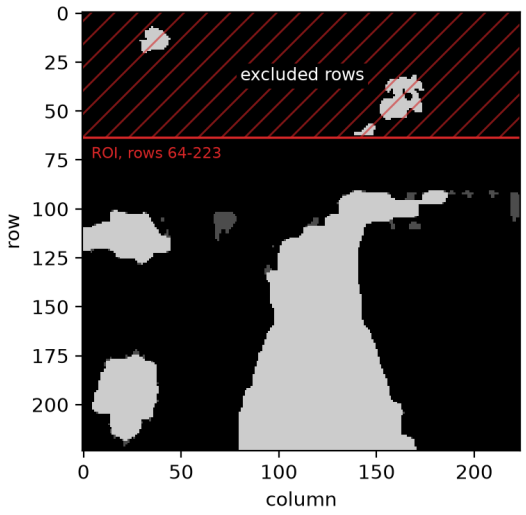


Figure 6.1: Example of the ROI and of the mask cleaning of Strip-IRLS on an example mask, with the parameters used on the robot. The rows above row 64 are excluded, and the dark gray regions inside the ROI are removed by the cleaning.

6.2.2 Horizontal strips and intervals

The ROI is divided into N strips of equal height, indexed from the bottom. N is a parameter of the extractor, set to 16 on the robot. The strip height is $h = \lceil (H - \lfloor \rho H \rfloor)/N \rceil$, which is the smallest height for which N strips cover the rows from $\lfloor \rho H \rfloor$ to the bottom of the image. When the number of these rows is not a multiple of N, the N strips cover a few more rows, so the ROI is extended upward and starts at $y_0 = H - Nh$. On the robot, the 157 rows from 67 to 223 are not a multiple of 16, so $h = 10$ px and the ROI starts at row 64. The extension

depends only on H, ρ and N, so the ROI is the same for every frame. Strip s covers the image rows from $H-(s+1)h$ to $H-sh-1$, and its center is at row $H-(s+0.5)h$. On the robot, strip 0 covers rows 214 to 223.

In each strip, a column is active if it contains at least one path pixel of $\widetilde{M}$. Runs of active columns separated by at most 30 inactive columns are merged into one interval $[\ell_i, r_i]$ of width $d_i = r_i - \ell_i + 1$, so that small holes and shadows inside the corridor do not split it.

An endpoint within 2 px of the image border is flagged as clipped. There the corridor leaves the image, so the endpoint is the image border and not the edge of the corridor. Two kinds of intervals are discarded: those narrower than 8 px, and those clipped on both sides when path pixels cover less than 70% of the strip.

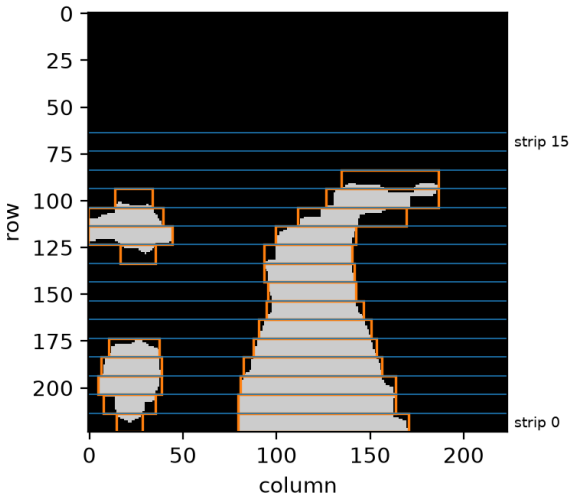


Figure 6.2: Strips and intervals on the mask of Figure 6.1, after the cleaning of Section 6.2.1, with the parameters used on the robot ($N = 16$, $h = 10$ px, ROI from row 64). The blue lines are the strip boundaries, and the orange boxes are the intervals found in each strip.

6.2.3 Bottom-up association and chain selection

The association decides which intervals belong to the corridor. The endpoints of these intervals are the samples of the two edge fits (Section 6.2.4). It works in two steps: one or more chains of intervals are grown upward from the bottom of the ROI, then one chain is selected.

Growing the chains. A chain starts from a seed, an interval of the bottom six strips, where the corridor is closest to the robot. On the robot, these six strips cover 60 image rows. From the seed, the chain is grown upward, one strip at a time. In each strip, each interval is compared with the last accepted interval $[\ell_p, r_p]$, of width d_p, through the boundary displacement $\delta_i = \max(|\ell_i - \ell_p|, |r_i - r_p|)$, which is the larger shift of its two endpoints. An interval is a candidate only if this shift is within the jump limit $\Delta_{\max}$. The limit is 28 px, or half the width of the last accepted interval when this is wider than 56 px. Among the candidates, the accepted interval is the one with the lowest cost J_i, which adds to the shift a penalty on the change of width:

$$\Delta_{\max} = \max(28,\, 0.5\, d_p), \qquad J_i = \delta_i + 0.4\, |d_i - d_p|. \tag{6.3}$$

The accepted interval becomes the reference for the next strip. The choice is greedy: an accepted interval is never revised later. A strip without candidates is skipped, and the reference does not change. The chain ends at the top of the ROI or at the fourth consecutive skipped strip, so on the robot it can cross a gap of up to 30 rows. No information is carried from one frame to the next.

The seeds are tried from strip 0 upward, and a chain is grown from each seed that does not already belong to an earlier chain. A chain grown from an interval of an earlier chain would start from the same reference and accept the same intervals, so it would only repeat the upper part of that chain. When the first chain contains all the intervals of the bottom six strips, there is only one chain. Other chains are grown when some of these intervals are not in the first chain, for example the intervals of a blob or of an adjacent corridor next to the path.

Selecting a chain. The complete chains are compared. The selected chain is the one that gives the most samples to the edge fits. The samples are the endpoints not clipped at the image border. An interval inside the image gives two samples, an interval clipped on one side gives one, and an interval clipped on both sides gives none. A chain along the image border therefore has fewer samples than a chain of the same length inside the corridor. Among chains with the same number of samples, the one with the lowest sum of the costs J_i is selected. If the bottom six strips contain no interval, or the selected chain has fewer than three intervals, the result is `lost`.

Figure 6.3 shows the association on the mask of Figure 6.2. The blob at the bottom left has more than 500 pixels, so the cleaning does not remove it, and it gives one interval in each of strips 0 to 4. Strip 0 contains two intervals, and the one on the left, in the blob, is tried first as a seed. Its chain accepts the blob intervals up to strip 4. The only interval of strip 5 belongs to the corridor, and it is shifted by 113 px from the interval accepted in strip 4, well beyond the jump limit of 28 px. The same happens in the next strips, so the chain finds no other candidate and ends at strip 4. The corridor interval of strip 0 is not in this chain,

so a second chain is grown from it. This chain accepts one interval in each strip up to strip 13. Strips 14 and 15 contain no interval, and the chain ends at the top of the ROI. In the bottom six strips, no interval is left outside the two chains, so no other seed is available and no other chain is grown (Figure 6.3a). The corridor chain has 28 samples and the blob chain has 10, so the corridor chain is selected, although its total cost is higher, 122 against 26.8 (Figure 6.3b). The mask also contains a second region separate from the corridor, at the top left, in strips 9 to 12. Its intervals are above the bottom six strips, so they are not seeds. In each of these strips, the corridor chain compares them with its last accepted interval: they are shifted by 96 to 136 px, well beyond the jump limit of 28–29 px, and they are rejected. They are not in any chain, and their endpoints are not used in the edge fits.

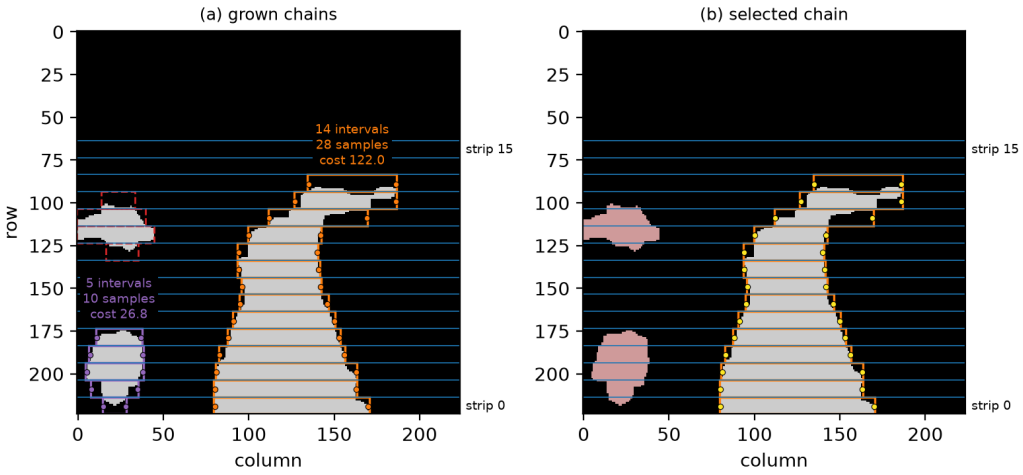


Figure 6.3: Chains built from the intervals of Figure 6.2. (a) The two chains grown from the seeds of strip 0, in purple from the blob and in orange from the corridor, with their number of intervals, samples and total cost. The dots are the samples, the endpoints that are not clipped. The dashed red boxes are intervals that are not in any chain. (b) The selected chain, the one with more samples. The yellow dots are its samples, used in the edge fits. The regions in light red are not in the selected chain and are not used in the fits.

6.2.4 Separate fitting of the corridor edges

The two edges are fitted separately. The left edge uses the left endpoints ℓ_i of the accepted intervals, and the right edge uses the right endpoints r_i. Clipped endpoints are not used, so an interval clipped on one side gives a sample only to the other edge, and the two edges can have samples in different strips. The row of

a sample is the center of its strip. In the equations below, a sample of one edge is the point (y_i, x_i), where y_i is its row and x_i is the endpoint, ℓ for the left edge and r for the right edge. In these equations i indexes the samples of one edge, not the intervals. Each edge needs at least two samples. If one edge has fewer, the result is `lost`, because the centerline is never estimated from a single edge. The edges are first fitted inside a crossing check, described at the end of this section, and then once more on the intervals that remain.

Each edge is fitted with iteratively reweighted least squares. The first fit uses uniform weights, and ten reweighting iterations follow. At each iteration, the distance of each sample from the current line gives its new weight, and the line is fitted again. A sample far from the line gets a low weight. How fast the weight decreases is set by the residual scale σ, a parameter of the extractor. A sample at distance σ from the line gets weight 0.5. On the robot, $\sigma = 18$ px (Section 6.2.6). At iteration t, the residuals and the new weights are

$$e_i^{(t)} = x_i - \left(m^{(t)} y_i + q^{(t)}\right), \qquad w_i^{(t+1)} = \frac{1}{1 + (e_i^{(t)}/\sigma)^2}, \tag{6.4}$$

and the line is fitted again by weighted least squares, with each weight applied to the residual before it is squared:

$$(m, q) = \arg\min_{m,q} \sum_i \left[w_i \Big(x_i - (m y_i + q) \Big) \right]^2 . \tag{6.5}$$

Each squared residual is therefore weighted by w_i^2. The weights are not accumulated across iterations, since they are computed again from the current line only. The reweighting reduces the effect of isolated outliers, but not of many samples from outside the corridor, because they lie on a consistent line and the fit follows them. Figure 6.4 shows the reweighting on the samples of the selected chain of Figure 6.3b.

The two edges of a corridor converge upward in the image, but in the rows where the chain has samples they are as far apart as the corridor is wide. Every accepted interval is at least $d_{\min} = 8$ px wide, the minimum interval width of Section 6.2.2. If the fitted edges come closer than $d_{\min}$ in these rows, the intervals above the row where this happens do not follow the corridor. They can come, for example, from a region attached to the corridor, and they bend the fits. The edges are therefore checked before the final fit. With $\Delta m = m_R - m_L$ and $\Delta q = q_R - q_L$, the modeled width is

$$D(y) = x_R(y) - x_L(y) = \Delta m\, y + \Delta q, \tag{6.6}$$

the signed horizontal distance between the two fitted lines at row y. It is negative where the lines have crossed. $D(y)$ is not the width of an interval, and it is defined at every row, also in the rows where the chain has no interval. The lines are fitted on the left and right endpoints. At the row of an interval $[\ell, r]$ not clipped on either

side, a good fit therefore gives D close to $r - \ell = d - 1$, not to the interval width d. When $\Delta m \neq 0$, $D(y)$ equals $d_{\min}$ at one value of y only,

$$y_{\times} = \frac{d_{\min} - \Delta q}{\Delta m}. \tag{6.7}$$

In general $y_{\times}$ is not an integer row. The edges converge upward when $\Delta m > 0$, in the code when $\Delta m > 10^{-6}$, and then $D(y) < d_{\min}$ above $y_{\times}$. If the edges converge upward and $y_{\times}$ is below the highest sample, the intervals whose strip center is above $y_{\times}$ are discarded, together with their samples on both edges, and both edges are fitted again. This crossing check is made at most three times. Each pass fits the edges, checks them, and makes at most one discard. After a discard in the third pass, the edges fitted on the remaining intervals are not checked again here. The check stops earlier in three cases:

- The check finds no crossing to treat: the edges do not converge upward, or $y_{\times}$ is not below the highest sample.
- The discard would leave fewer than three intervals. The intervals are then kept, and the check stops with the crossing still present.
- An edge has fewer than two samples, and the result is `lost`. The previous rule counts only the intervals, so this can also happen after a discard, when the remaining intervals are clipped on that side.

If the result is not `lost`, the final fit uses the remaining intervals. A crossing in the rows where the chain has samples can therefore still be present in three cases: when too few intervals would remain, after a discard in the third pass, or when the edges do not converge upward, which the check does not treat. These crossings are found by the width check of Section 6.2.5.

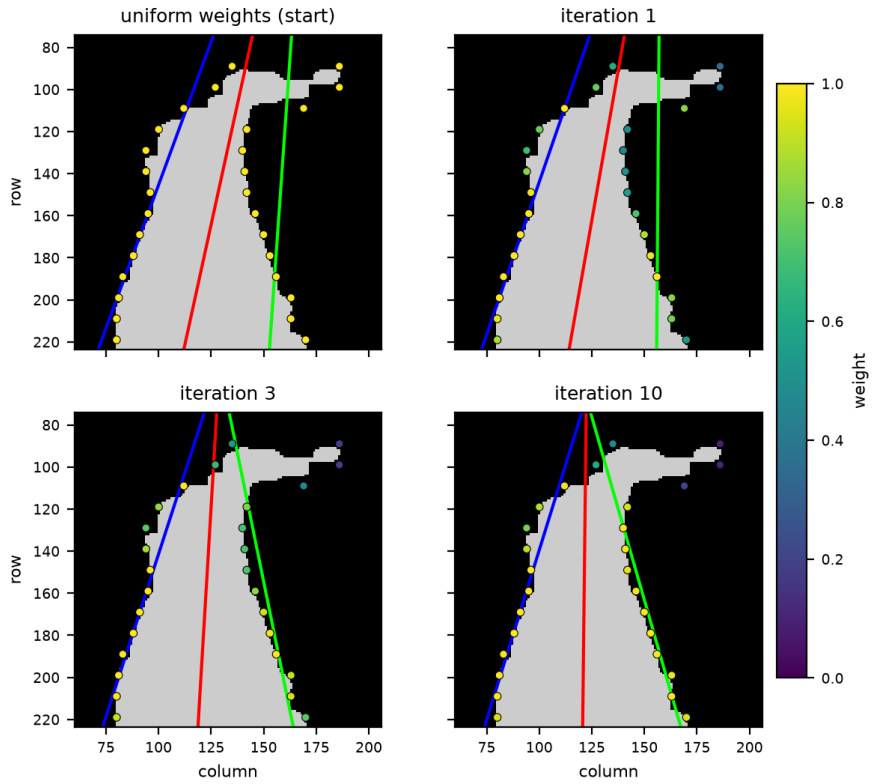


Figure 6.4: Reweighted fit of the two edges of Strip-IRLS on the samples of the selected chain of Figure 6.3b, at the start and after iterations 1, 3 and 10. The points are the samples of the chain, colored by their weight, the blue and green lines are the fits of the left and right edge, and the red line is the centerline, their mean. The three upper samples of the right edge, on the region attached to the corridor, pull the first fit toward the vertical. Their weights fall to 0.21, 0.10 and 0.09 only after several iterations, and after the tenth the line follows the edge.

6.2.5 Validation, confidence, and output

After the final fit, the width check tests the final edges. It finds the crossings that are still present in the three cases listed at the end of Section 6.2.4. It also defines the valid range, and it rejects edges that do not converge upward and a valid range that is too short. The check uses the modeled width $D(y)$ of Eq. (6.6) and $y_\times$ of Eq. (6.7), both computed on the final edges. It looks at the rows from the highest sample used in the final fit down to the bottom row of the image, row $H-1$. The valid range is the part of these rows where $D(y) \geq d_{\min}$. It starts at the highest sample, or at $y_\times$ when $y_\times$ lies just below it, within the tolerance described below, and it always ends at the bottom row of the image.

The robot moves between parallel rows, so because of perspective the edges of the corridor converge upward, with $\Delta m > 0$. Edges that are parallel or converge downward indicate a wrong fit, and the check fails.

When the edges converge upward, $D(y)$ is below $d_{\min}$ in the rows above $y_\times$, so these rows are not in the valid range. The check then works as follows:

- If $y_\times$ is at the highest sample or above it, no row is removed, and the valid range starts at the highest sample.
- If $y_\times$ is below the highest sample by at most $0.05H$, that is 11.2 px on the robot, a little more than the strip height of 10 px, the valid range starts at $y_\times$. This tolerance accepts a crossing just below the highest sample.
- If $y_\times$ is more than $0.05H$ below the highest sample, the check fails. The edges are then closer than $d_{\min}$ in rows where the chain has samples, which indicates a chain that joins different regions or a wrong fit.
- The check also fails if the distance from the top of the valid range to the bottom row of the image is less than 10% of the ROI height $H - y_0$. On the robot the ROI has 160 rows, so the limit is 16 px. The corridor is then seen in too few rows.

The confidence C combines four scores between 0 and 1. They use the seed and the accepted intervals of the selected chain that were not discarded by the crossing check of Section 6.2.4. Below they are called the intervals of the chain.

- **Tracking T.** The fraction of the strips of the valid range that contain an interval of the chain. It is the number of intervals of the chain whose strip center lies in the valid range, divided by the number of strips whose center lies in the valid range. The intervals of the chain above the valid range are not counted, for example the interval of the highest sample when the valid range starts at $y_\times$ within the tolerance. T is 1 when every strip of the valid range has an interval of the chain, and it is low when the chain skips many of them. For example, a chain from strip 0 to strip 13 that skips two strips below strip 11 has 12 intervals. If the crossing check discards the intervals of strips 12 and 13, 10 intervals remain. If the interval of strip 11 gives at least one sample, and the final edges converge upward and reach $d_{\min}$ at or above its center, the valid range starts at the center of strip 11, the highest remaining sample. Then $T = 10/12 \approx 0.83$: the two skipped strips lower T, while the two discarded intervals lie above the valid range and do not.
- **Fit F.** How close the samples are to the fitted edge lines. With $\bar{e}$ the mean horizontal distance of the samples of both edges from their lines, $F = \text{clip}(1 - \bar{e}/\sigma,0,1)$. F is 1 when the samples lie on the lines and falls to 0 when

$\bar{e}$ reaches $\sigma = 18$ px. It is low when the edges of the mask are irregular or when some samples come from outside the corridor. The distances are not weighted, so a sample with a low weight in the fit still counts fully in $\bar{e}$.

- **Width regularity** A**.** With straight edges, the width of the corridor changes by the same amount from one strip to the next. A measures how far the widths are from this. It uses the widths $r_i - \ell_i$ of the intervals of the chain that are not clipped on either side, from the bottom up, since the width of a clipped interval is not the corridor width. The error E_D is computed in three steps:

  1. The changes of width between consecutive intervals are computed, and then the changes between consecutive changes, the second differences. A second difference is zero when the width changes by the same amount twice.
  2. The 90th percentile of the absolute second differences is taken. With few intervals it is close to the largest one.
  3. It is divided by the median width, so that E_D is a jump relative to the width of the corridor.

  Then $A = \mathrm{clip}(1 - E_D/0.30,0,1)$, which is 1 when the width changes by the same amount at every step and 0 when E_D reaches 0.30. For example, the widths 120, 114, 108, 102 and 96 px change by -6 px at every step, so the second differences are zero, $E_D = 0$ and $A = 1$. If a region attached to the corridor widens the third interval to 140 px, the changes are -6, $+26$, -38 and -6 px, and the absolute second differences are 32, 64 and 32 px. Their 90th percentile is 57.6 px and the median width is 114 px, so $E_D = 0.51$ and $A = 0$. A skipped strip or an excluded clipped interval between two widths also gives a nonzero second difference when the width changes along the corridor. With fewer than three unclipped intervals there is no second difference and the width cannot be checked, so A is 0.

- **Gaps** G**.** The longest hole in the chain. With $g_{\max}$ the longest run of strips skipped between two intervals of the chain, $G = \mathrm{clip}(1 - g_{\max}/3,0,1)$. G is 1 when the chain skips no strip and 0 when it skips three strips in a row, the longest gap the chain can cross. Every strip of the valid range without an interval of the chain lowers T, also below the seed. G depends only on the longest run of skipped strips between two intervals of the chain, so one long gap lowers G more than several short ones.

The confidence is their weighted sum:

$$C = 0.3125\,T + 0.3125\,F + 0.25\,A + 0.125\,G. \tag{6.8}$$

The result is `lost` if the width check fails, if $\bar{e} > 2\sigma$, or if $C < 0.38$. Otherwise it is `degraded` for $C < 0.60$ and `ok` for $C \geq 0.60$. C is a heuristic score, not a calibrated probability.

For a `lost` result, no centerline model is returned. Otherwise, the centerline model $x_C(y)$ is returned. The extractor also returns a mask of the size of the input. The centerline is drawn in it with one pixel per row, from the top of the valid range, rounded up to an integer row, to the bottom row of the image, and then dilated with a 3×3 elliptical kernel. For a `lost` result this mask is empty. The controller reads the lateral error from the model, not from the mask. It evaluates the model at the reference row $y_{\text{ref}} = 156$ (Section 7.2.3), also when this row is above the valid range.

6.2.6 Setting the parameters

Table 6.1 lists the parameters of *Strip-IRLS* and their values on the robot. The upper part lists the arguments of the extractor, and the lower part the constants written in the code. The distances and areas are in pixels of the 224×224 mask. The 640×480 camera image is resized to this size (Section 6.1), so one pixel of the mask is about 2.9 image pixels horizontally and 2.1 vertically. The values therefore depend on the camera mounting and on this resizing. Chapter 8 reports tests on the development data for the ROI start, the merging gap, the exact tiling of the ROI with strips of 10 rows and the ten reweighting iterations (Sections 8.1.2 and 8.1.2). The other values were set by hand during the development, from the results on the synthetic development masks (seed 0), and no systematic test of them is reported. The list below explains the main parameters: what each one is, how to set it and how it changes the result.

Table 6.1: Parameters of Strip-IRLS used on the robot. Distances and areas are in pixels of the 224×224 mask.

Parameter	Value
Arguments of the extractor	
Path class	1
ROI start ratio ρ	0.30
Number of strips N	16
Minimum component area	500
Minimum interval width $d_{\min}$	8
Merging gap	30
Image-border margin	2
Minimum jump limit	28
Residual scale σ	18
Reweighting iterations	10
Seed window	6 strips
Consecutive skipped strips allowed	3
Crossing tolerance	$0.05H$
Minimum valid-range length	$0.10\,(H - y_0)$
Lost / ok thresholds on C	0.38 / 0.60
Constants in the code	
Opening / closing kernels	5×5 elliptical / 9×9 rectangular
Connectivity of the components	8
Occupancy for intervals clipped on both sides	0.70
Jump-limit factor on d_p	0.5
Width-change cost coefficient	0.4
Minimum intervals in the chain	3
Minimum samples per edge	2
Crossing-check passes	3
Convergence threshold on Δm	10^{-6}
Residual rejection factor	2 ($2\sigma = 36$)
Percentile of the second differences	90
Width-regularity scale	0.30
Weights of T, F, A, G in C	0.3125, 0.3125, 0.25, 0.125

- **ROI start ρ.** It sets the requested first row of the ROI, $\lfloor \rho H \rfloor$, which is then extended up to $y_0 = H - Nh$ (Section 6.2.2). It should be the highest row where the path is visible. A smaller ρ adds rows above the path, whose regions can enter the chain and pull the fits. With a larger ρ, the edges are fitted on fewer rows of the corridor. With the same N, ρ can also change the strip height h. On the robot, $\rho = 0.30$, chosen on the development data (Section 8.1.2).

- **Number of strips N.** The ROI is divided into N strips of height h. More strips give more samples to the edge fits, but a thin strip is emptied or split more easily by a hole in the mask. The endpoints of an interval are the outermost path columns over all the h rows of its strip. In a thick strip, noise on the edge in any one of these rows moves an endpoint outward, and so does the inclination of the edge, by up to half the shift of the edge over the strip. The seed window, the bottom six strips (Section 6.2.3), and the number of consecutive skipped strips allowed, three, are set in strips and not in rows. On the robot, with $N = 16$ and $h = 10$, they cover 60 and 30 rows. With $N = 32$ and $h = 5$ they would cover 30 and 15 rows, so they have to be changed together with N to keep the same rows.

- **Merging gap.** The largest number of inactive columns between two runs merged into one interval. It should be larger than the holes and shadows inside the corridor and smaller than the crop row between two adjacent corridors, which is narrowest at the top of the ROI. If it is too large, a blob or an adjacent corridor is merged into the interval and moves its endpoint. On the robot it is 30 px, chosen on the development data (Section 8.1.2).

- **Minimum component area and minimum interval width $d_{\min}$.** The cleaning removes the components with fewer pixels than the minimum area, and the intervals narrower than $d_{\min}$ are discarded. The area should be below that of the parts of the corridor separated from the rest, and $d_{\min}$ below the narrowest width of the corridor, at the top of the ROI. Smaller values keep blobs and narrow intervals of noise, which can start or enter a chain. $d_{\min}$ is also the threshold on $D(y)$ in the crossing check and in the width check, so a larger $d_{\min}$ discards more intervals and shortens the valid range. On the robot they are 500 pixels and 8 px.

- **Jump limit $\Delta_{\max}$.** The largest allowed shift of each endpoint between the last accepted interval and a candidate, $\max(28,\, 0.5\, d_p)$ px (Section 6.2.3). It should be above the shift of the corridor edges, also over four strips when three strips are skipped, and below the shift to the regions next to the corridor. If it is too small, the chain stops on inclined edges or where the interval widens. If it is too large, the regions next to the corridor become candidates. The shift of an edge is its slope times the number of rows. The slope of an edge depends on the heading and the lateral offset of the robot in the row and on the pitch and roll of the camera.

- **Residual scale σ.** A sample at distance σ from the edge line gets weight 0.5 (Eq. (6.4)). It should be above the distance of the correct samples from the line, which comes from the irregular edges of the mask, and below the distance of the outliers. A smaller σ lowers the weight of correct samples and the fit

score F, so more results are `degraded` or `lost`. A larger σ keeps the outliers in the fit and accepts more wrong results, because F and the threshold 2σ grow. On the robot, $\sigma = 18$ px.

- **Status thresholds.** The result is `lost` if the width check fails, if $\bar{e} > 2\sigma$ or if $C < 0.38$. Otherwise it is `degraded` for $C < 0.60$ and `ok` for $C \geq 0.60$ (Section 6.2.5). A higher lost threshold rejects more frames, and a higher ok threshold makes more results `degraded`, which the controller filters differently (Section 7.2.3).

These values are set for the camera, its mounting and the fields of this work. With another camera, mounting or crop, the parameters have to be set again for the new scenario. This is a limitation of *Strip-IRLS*, and one of the reasons for *EdgeNet* (Section 6.3).

6.3 EdgeNet, an edge regression network

EdgeNet is a first test of replacing the geometric steps of *Strip-IRLS* with a learned model. Like the second network of Müller et al. [16] (Section 3.5.5), it takes a segmentation mask and not the image, but it regresses the two corridor edges instead of waypoints. Its aim is to avoid the geometric parameters of *Strip-IRLS*, which have to be set by hand for each scenario (Section 6.2.6). The network has no geometric parameters in the extraction. To adapt it to a new scenario, the generator of its training masks is changed and the network is trained again, instead of tuning thresholds. It still has the validity and status thresholds described below.

The network is a small CNN of about 0.8 million parameters, shown in Figure 6.5. Its input is the 224×224 mask, and it has five outputs: the x positions of the left and right edges at rows 156 and 223, and the logit of the probability p_{valid} that the mask contains a corridor. Row 156 is the row read by the controller and row 223 is the last row of the image. Each edge line is the line through its predicted points at rows 156 and 223, and the centerline is the mean of the two lines, as in Eq. (6.2).

The confidence C combines two values. The first is p_{valid}, given by the network: its estimate of the probability that the mask contains a corridor. The network learns it from the validity label of the training masks, defined below. The second is computed by the extractor after the network: the intersection over union (IoU) between the region enclosed by the two edge lines and the path pixels of the mask, in the rows from 64 to the bottom of the image. The mask is not cleaned as in *Strip-IRLS*. With R the set of pixels between the two lines and P the set of path pixels, both in the rows from 64 to 223,

$$\text{IoU} = \frac{|R \cap P|}{|R \cup P|} = \frac{|R \cap P|}{|R| + |P| - |R \cap P|}, \tag{6.9}$$

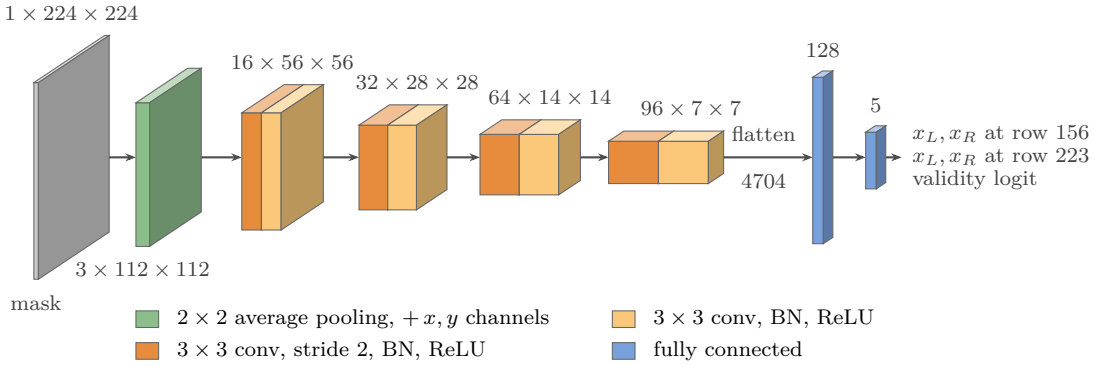


Figure 6.5: Architecture of *EdgeNet*. Each box is a feature map, colored by the layer that produces it and labeled with channels × height × width. The two maps of a block have the same size and share one label. The thickness of a box grows with the channels and its height with the size of the map, not to scale. The mask is averaged over 2×2 blocks, and two channels with the x and y coordinates of each pixel, scaled to $[-1,1]$, are added, so the convolutions also see where each pixel is. Each of the four blocks halves the size with a stride-2 convolution and then applies a second convolution. The $96 \times 7 \times 7$ map is flattened and passed to two fully connected layers, the first followed by ReLU and dropout 0.1.

where $|\cdot|$ is the number of pixels of a set. It is the IoU of Eq. (8.1), with the region between the lines in place of the prediction and the path pixels of the mask in place of the reference. The numerator counts the path pixels between the lines. The denominator also counts the pixels between the lines that are not path and the path pixels outside the lines. The IoU is 1 when the region between the lines covers exactly the path of the mask. It is lower when the lines are wrong, or when the mask also contains path outside them, for example an adjacent corridor. The confidence is their product, $C = p_{\text{valid}} \cdot \text{IoU}$. The status is `lost` if $p_{\text{valid}} < 0.5$, if the distance between the two predicted edges at row 223 is below 8 px, or if $C < 0.38$. Otherwise it is `degraded` for $C < 0.60$ and `ok` for $C \geq 0.60$. The width is checked at row 223 and not at row 156 because of the labels of the training masks, described below.

The network is trained only on synthetic masks, with labels computed from the corridor model and not from an extractor, so that it does not learn the errors of an algorithm. Müller et al. obtain realistic errors by applying a real segmentation network to simulated images. Here the errors of the segmentation are modeled by hand in the generator. The generator draws perspective corridors whose apex, the row where the two edges meet, goes from above the image down to row 190. A quarter of the corridors are curved. The generator adds disturbances found in predicted masks, such as irregular edges, adjacent corridors, holes and blobs.

Above the apex the labels extend the edge lines, so when the apex is below row 156 the labeled edges are swapped at that row. For this reason the width is checked at row 223. Each training mask also has a validity label. It is 1 when the corridor drawn before the disturbances has at least 800 pixels and at least one of its two edge lines lies inside the image, more than 2 px from the border, in at least 15 rows from row 64. It is 0 for the other masks and for the masks without a corridor. The label does not depend on the disturbances, so a corridor partly erased by them is still valid. About 14% of the 300 000 training masks have no valid corridor. The generator does not use the synthetic test sets of Chapter 8, but it was written by the same author, who knew their scenarios. The loss is a smooth L1 loss on the four edge positions, plus twice the same loss on the centerline at row 156, both only on masks with a valid corridor, plus 0.2 times the binary cross-entropy on the validity. On the 17 266 masks with a valid corridor among 20 000 synthetic validation masks, the median centerline error at row 156 is 1.7 px and the 90th percentile 13.0 px. These masks were also used to choose the checkpoint with the lowest 90th percentile. These values say nothing about real masks. The network is exported to ONNX and run on the robot with ONNX Runtime.

6.4 Baseline extractors

Strip-IRLS and *EdgeNet* are compared with the three geometric baselines of this section.

6.4.1 Center points

The *Center points* extractor uses the same mask cleaning and interval extraction as *Strip-IRLS*, with the same merging gap of 30 columns. Its ROI is built as in *Strip-IRLS* (Section 6.2.2), with $\rho = 0.30$ and $N = 24$ strips. The strips are $h = \lceil 157/24 \rceil = 7$ px high, so the ROI is extended upward from row 67 to row $y_0 = H - Nh = 56$. Each interval is reduced to its midpoint $c = (\ell + r)/2$ and its width $d = r - \ell + 1$. The seed is the interval of the bottom six strips with the most path pixels, with ties broken by width. Intervals are associated upward with the cost

$$J_i = |c_i - c_p| + 0.4\,|d_i - d_p|, \tag{6.10}$$

with c_p and d_p the midpoint and width of the last accepted interval, under the jump limit $|c_i - c_p| \le \max(28, 0.5\,d_p)$, with the same stopping rule and the same minimum of three intervals as *Strip-IRLS*.

One line is fitted to the midpoints with four iterations of the weights of Eq. (6.4). Unlike *Strip-IRLS*, the initial weights are proportional to the pixel count of each interval (normalized to mean one), the weights multiply across iterations, and

clipped intervals are not excluded. The confidence combines the same four scores as *Strip-IRLS* (Section 6.2.5), tracking T, fit F, regularity of the width A and gaps G, with weights 0.30, 0.35, 0.25 and 0.10. T is the number of accepted intervals divided by the number of all the strips, so the strips above the end of the corridor also lower it. A uses the widths of all the accepted intervals, and G also counts the skipped strips that end the association. The result is `lost` if the mean residual exceeds 2σ or if the confidence is below 0.38, and `degraded` if it is below 0.60. Because the midpoints are formed before the fit, a disturbance on one edge moves the midpoint and enters the centerline.

6.4.2 Column histogram

The *Column histogram* extractor is a simplified version of the column projection of SegMin. SegMin sums a plant mask, accumulated over recent frames and refined with depth, along the image columns, smooths the profile, and takes its minimum [3]. *Column histogram* uses the path mask of a single frame, without depth or smoothing, and takes the maximum. Its ROI starts at $y_0 = \lfloor 0.30H \rfloor = 67$, and the mask is cleaned with the same opening and closing as *Strip-IRLS*. The connected components with fewer than 300 pixels are then removed. In each column, the extractor counts the path pixels of the cleaned ROI, and the target column x_{target} is the column with the most path pixels. If several columns have the maximum count, as when the corridor fills the whole height of the ROI, x_{target} is the center of the widest group of adjacent ones. If two or more groups have the same largest width, no column is chosen and the result is `lost`. The confidence is the maximum count divided by the ROI height $H - y_0$, with the same thresholds as *Strip-IRLS* (0.38 and 0.60). The output is the vertical line $x = x_{\text{target}}$, so the method gives a lateral position but no heading.

6.4.3 RANSAC edges

RANSAC line fitting is used in agricultural row detection on points of the crop rows [1, 14]. The RANSAC extractor applies it to the edges of the segmented corridor, without strips or bottom-up association.

The ROI starts at $y_0 = \lfloor 0.30H \rfloor = 67$ and the mask is cleaned with the same opening and closing as *Strip-IRLS*. The extractor then keeps one connected component, a large one that starts low in the image. In each ROI row of this component, its leftmost and rightmost pixels are edge points. Rows where they are less than 10 px apart are skipped. As in *Strip-IRLS*, the endpoints at the image border are not used.

Each edge is fitted with RANSAC [15]: in 100 trials, two random points define a line, and the points within 18 px of it are inliers. The line with the most inliers

is refitted by least squares on its inliers, and the centerline is the mean of the two edge lines, as in Eq. (6.2). The random draws use a fixed seed, so the same mask always gives the same line.

The result is `lost` if one edge has fewer than 10 points or its points span fewer than 20 rows, for example when the mask is a band of path across the bottom of the image at the end of a row. It is also `lost` if the component has fewer than 1000 pixels in the ROI, or if the mean fraction of inliers of the two edges is below 0.30. Otherwise a score combines the fraction of inliers, their residual, the vertical extent of the component and the regularity of its width. Below 0.38 the result is `lost`, below 0.60 `degraded`, and from 0.60 `ok`.

6.5 Structural differences and limitations

Table 6.2 summarizes how the five extractors differ in their code. It does not describe measured performance, which is reported in Chapter 8.

Table 6.2: Structural comparison of the five extractors. Row extremes are the leftmost and rightmost pixels of the selected component in each ROI row. LS is a least-squares refit on the inliers.

Characteristic	Strip-IRLS	EdgeNet	Center points	Column histogram	RANSAC edges
Output	Line	Line	Line	Vertical line	Line
Heading estimate	Yes	Yes	Yes	No	Yes
First row used	64	0 (network), 64 (IoU)	56	67	67
Mask cleaning	Open, close, ≥500 px	None	Open, close, ≥500 px	Open, close, ≥300 px	Open, close, one component ≥500 px
Edges fitted separately	Yes	Regressed	No	N/A	Yes
Edge points	Chain endpoints	N/A	N/A	N/A	Row extremes
Clipped endpoints excluded	Yes	N/A	No	N/A	Yes
Bottom-up seed and jump limit	Yes	No	Yes	No	No
Line estimator	IRLS	CNN regression	IRLS	N/A	RANSAC, LS

Strip-IRLS has three structural limitations:

- **Seed dependence.** The seeds are searched only in the bottom six strips. If these contain no interval, the result is `lost` even if the upper part of the corridor is segmented. If a hole wider than the merging gap and taller than these strips splits the bottom of the corridor, the chain follows only one part (Section 8.1.2). At a fork above these strips, the chain takes the branch with the lowest cost and never revises it.
- **Number of parameters.** *Strip-IRLS* has 16 arguments and 13 constants (Table 6.1), most of them in pixels of the mask, and several act together. They were set for the camera, its mounting and the fields of this work, and have to be set again for a new scenario (Section 6.2.6).

- **Missing plants on one side.** The interval widens on that side. Within the jump limit it is accepted, and its endpoint on that side is a wrong sample that only the reweighting can reduce. Beyond the limit the strip is skipped. If the interval reaches the image border, its endpoint on that side gives no sample.

Chapter 7

Robot Platform and Integration

This chapter describes the robot used in the field tests, the software that runs on it, and the deployment of the segmentation network.

7.1 Robot hardware and sensors

7.1.1 Robot platform

The experimental platform is Demetra, a mobile robot prototype developed by the Roboto student team of the Politecnico di Torino. It has four-wheel steering (4WS) and four-wheel drive (4WD).

Each wheel has a DJI RoboMaster M3508 brushless motor with a DJI RoboMaster C620 speed controller, and its own CubeMars AK70-10 actuator for steering. A DJI RoboMaster Development Board Type C (STM32F407) controls the speed controllers and the steering actuators. It receives the linear velocity and the steering angle from the onboard computer and sends back the measured platform speed and steering angle. The wheelbase is $L = 40$ cm, the track width is $T = 34$ cm, the wheel diameter is 19 cm ($r_w = 9.5$ cm) and the chassis is 15 cm above the ground.

Steering is described by a kinematic bicycle model with opposite-phase steering (Figure 7.2), where the equivalent front and rear steering angles are $\delta_f = \delta$ and $\delta_r = -\delta$. With pure rolling, the turning radius of the mid-axle point M is

$$R = \frac{L}{2 \tan \delta}. \tag{7.1}$$

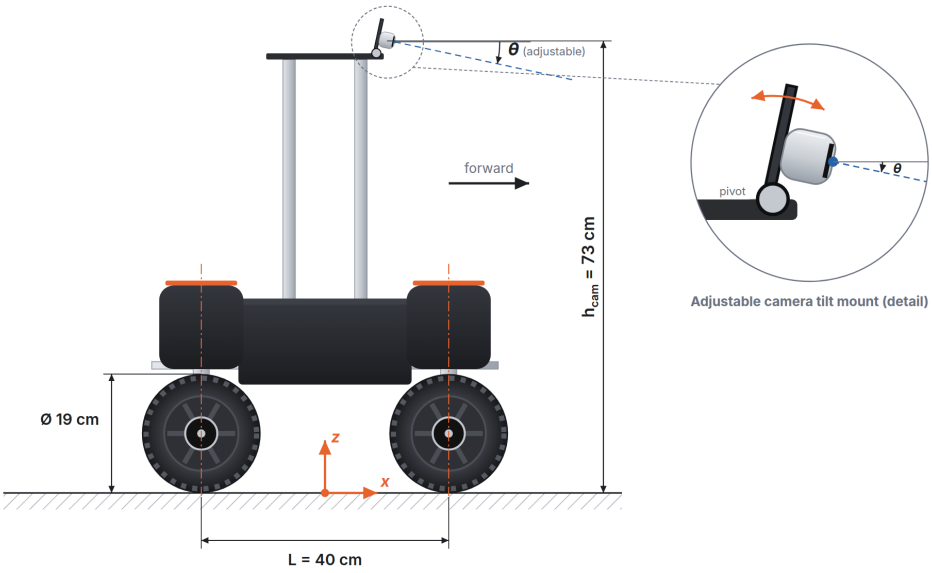


Figure 7.1: Side view of Demetra, with the wheel diameter, the wheelbase L, the camera height h_{cam} and the adjustable tilt θ of the camera.

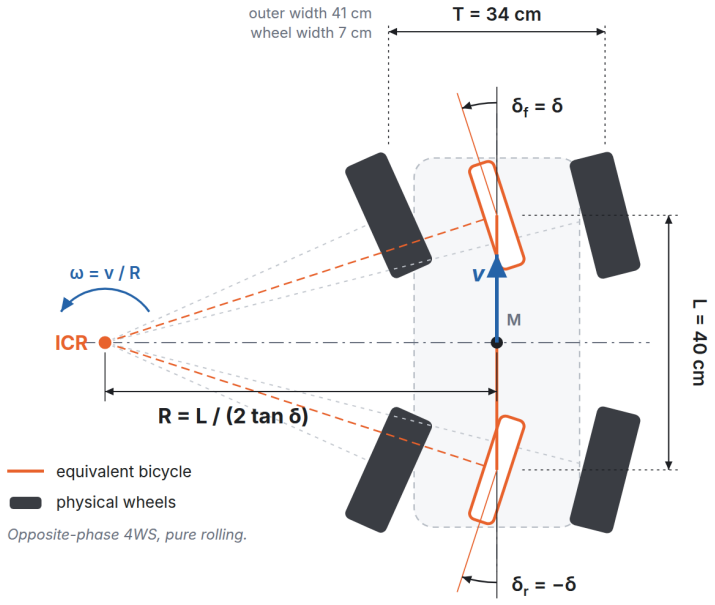


Figure 7.2: Top view of the opposite-phase 4WS kinematics of Demetra, with the equivalent bicycle in orange, the ICR and the turning radius R of the mid-axle point M (Equation (7.1)).

7.1.2 Camera configuration

The camera is an Intel RealSense D435, mounted on the front of the robot frame on a mount with adjustable tilt (Figure 7.1). The color stream is used at 640×480 pixels and 30 frames per second. The camera is tilted downward by a few degrees, so that the image shows mainly the path, which covers about the lower 70% of the image, as the region of interest of the extractors. The tilt is adjusted to the field.

7.1.3 Computing platform

The onboard computer is an NVIDIA Jetson AGX Orin Developer Kit (64 GB), with a 12-core ARM Cortex-A78AE CPU and an NVIDIA Ampere GPU with 2048 CUDA cores. It runs Ubuntu 22.04 with L4T R36.4.4. The software runs in a Docker container with access to the GPU and to the USB devices. Inside the container, ROS 2 Humble runs the camera driver, the controller node, and the serial driver.

7.2 Software architecture

7.2.1 ROS integration

The software pipeline is implemented in ROS 2. Figure 7.3 shows the node graph of the robot. Three nodes are active. The RealSense node (`/camera/camera`) publishes the compressed color images. The controller node (`/controller_node`) subscribes to these images. In the same callback, it runs the segmentation network and the path extractor, computes the velocity command (Section 7.2.3), and publishes it on `/cmd_vel`. The serial driver node (`/demetra_node`) receives the commands and sends them to the microcontroller. The controller node also publishes debug topics under `/debug`, which are used during configuration and are omitted from the figure. When the debug video is enabled, it also publishes three debug images (segmentation, region of interest and path), shown in Figure 7.4. The debug video makes the node skip camera frames (Section 7.3.1), so it should be disabled when the robot operates. The text log on `/debug/text` is always active. It carries important information on each frame and its cost is small, so the node still processes every camera frame.

The image subscription keeps only the last frame, so frames that arrive during the processing are not queued and the controller always processes the most recent image seen by the robot.

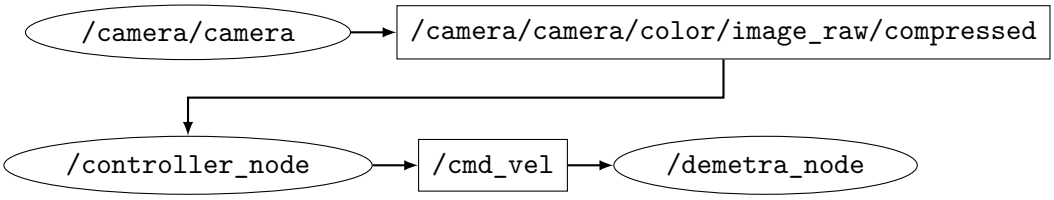


Figure 7.3: ROS 2 node graph on the robot, as shown by `rqt_graph`. Ellipses are nodes and rectangles are topics.

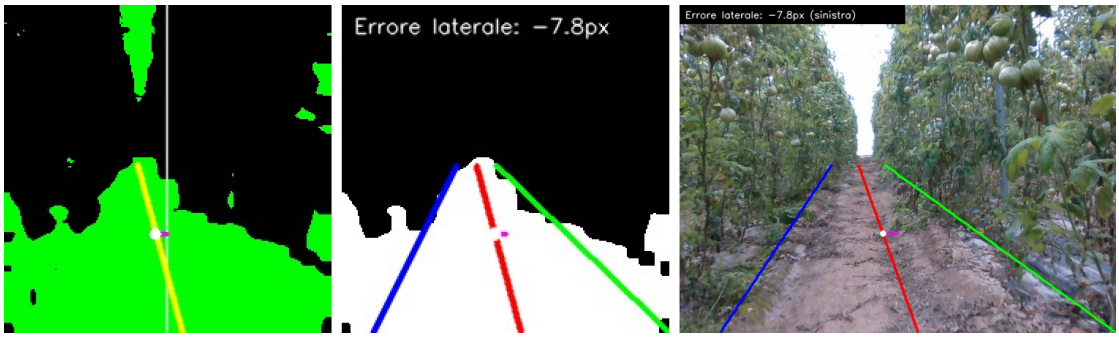


Figure 7.4: Frame of the debug video recorded during a field run with Strip-IRLS. From left to right: segmentation (the segmentation before the morphological cleaning), region of interest (the cleaned segmentation, on which the path extractor operates) and path (the extracted path drawn on the camera image).

7.2.2 Control interface

The serial driver node (`/demetra_node`) converts each command (v, ω) into the steering angle

$$\delta = \arctan\left(\frac{L\,\omega}{2\,v}\right), \tag{7.2}$$

limited to $\pm 30°$. Before the conversion, the node limits v to ± 1.5 m/s and ω to ± 6.0 rad/s. Every 10 ms it sends the last v and δ to the microcontroller. If no command arrives for 500 ms, it sets the speed to zero.

The microcontroller receives v and δ from the driver and actuates them with the steering actuators and the traction motors of the four wheels.

7.2.3 Controller node

For each frame, the controller node decodes the image, resizes it to 224×224 pixels and normalizes it with the ImageNet mean and standard deviation. The

segmentation network turns the image into the label mask, and the path extractor (Chapter 6) turns the mask into a centerline.

The lateral error e is the horizontal offset, in pixels, of the centerline from the image center at row $y_{\text{ref}} = 156$ of the $W \times H = 224 \times 224$ input. It is positive when the centerline is on the right. The error then passes through a filter that depends on the status returned by the path extractor (Section 6.2.5), a deadband and the steering law, which gives the angular velocity command.

Error filter. The error is smoothed with an exponential moving average (EMA). At each frame k, the filtered error is a weighted average of the previous filtered error $\hat{e}_{k-1}$ and of the new measurement e_k:

$$\hat{e}_k = (1 - \alpha)\, \hat{e}_{k-1} + \alpha\, e_k. \tag{7.3}$$

The smoothing factor α is the weight of the new value. With $\alpha = 0.20$ the filter keeps 80% of the previous value at each frame. If the measured error jumps to a new value and then stays constant, the filter covers about half of the step after three frames and 95.6% of it after 14 frames, which at 30 frames per second take 0.1 s and 0.47 s.

In practice the measured error changes at every frame, because of the noise of the segmentation and of the path extractor and because the robot moves. Replacing $\hat{e}_{k-1}$ with Equation (7.3) written for the previous frame, and repeating this for the older frames, gives

$$\hat{e}_k = \alpha\, e_k + \alpha(1 - \alpha)\, e_{k-1} + \alpha(1 - \alpha)^2\, e_{k-2} + \dots \tag{7.4}$$

Each measurement enters the filtered error with weight α, and its weight is multiplied by $1 - \alpha$ at every following frame. With $\alpha = 0.20$ the newest measurement weighs 0.20, the previous one $0.20 \cdot 0.8 = 0.16$ and the one before $0.20 \cdot 0.8^2 = 0.128$. The weights of the last n measurements add up to $1 - 0.8^n$, so more than 95% of the filtered value depends on the last 14 measurements. Within them, the most recent measurements count the most, while the oldest weigh almost nothing, for example $0.20 \cdot 0.8^{13} \approx 0.011$ for the fourteenth. A change that lasts one frame therefore enters the filtered error only with weight 0.20, while a change that lasts several frames enters all the recent measurements and is followed with a short delay. Figure 7.5 shows an example from a field run. In the 1.5 s shown, the mean difference between two consecutive frames is 10.8 px for the measured error and 4.2 px for the filtered error.

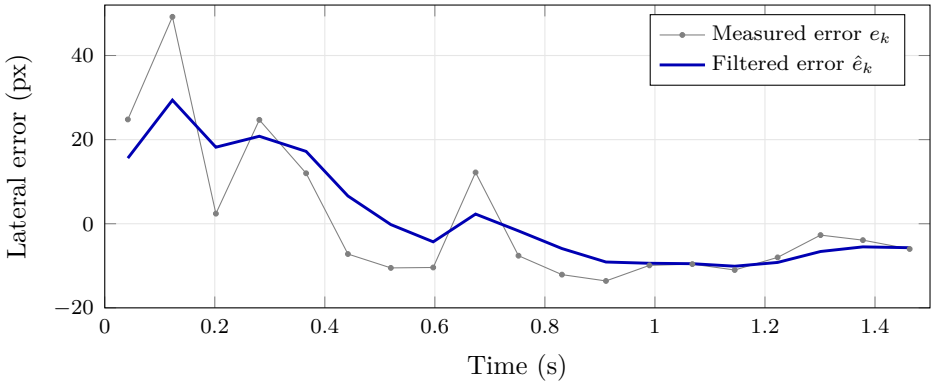


Figure 7.5: Measured lateral error e_k and filtered error $\hat{e}_k$ during 1.5 s of a field run with Strip-IRLS. The filtered error was computed from the recorded measurements with Equation (7.3) and the smoothing factor of Equation (7.5) for the recorded time between frames.

Smoothing independent of the frame rate. If the node runs slower, the same three frames last longer and the filter reacts later. With the debug video enabled, for example, the node processes 17 to 24 frames per second (Section 7.3.1). The smoothing factors are therefore defined as reference values α_{ref} for 30 frames per second. At each frame, the node measures the time Δt since the previous frame, limited to the interval from 1/120 to 0.25 s, and converts each reference factor into

$$\alpha = 1 - (1 - \alpha_{\text{ref}})^{30\,\Delta t}. \tag{7.5}$$

With $\Delta t = 1/30$ s, $\alpha = \alpha_{\text{ref}}$. At 15 frames per second, for example, $\alpha_{\text{ref}} = 0.20$ gives $\alpha = 1 - 0.8^2 = 0.36$, so the filter keeps 64% of the previous value at each frame, as in two frames at 30 frames per second. The filter therefore follows the same response in time, only with fewer updates (Table 7.1). Figure 7.6 shows the two responses.

Table 7.1: Fraction of a step of the input covered by the filter of Equation (7.3) with $\alpha_{\text{ref}} = 0.20$, at 30 and 15 frames per second. A dash marks a time with no frame at 15 frames per second.

Time (s)	0.033	0.067	0.100	0.133	0.167	0.200	0.233	0.467
30 fps, $\alpha = 0.20$	20.0%	36.0%	48.8%	59.0%	67.2%	73.8%	79.0%	95.6%
15 fps, $\alpha = 0.36$	–	36.0%	–	59.0%	–	73.8%	–	95.6%

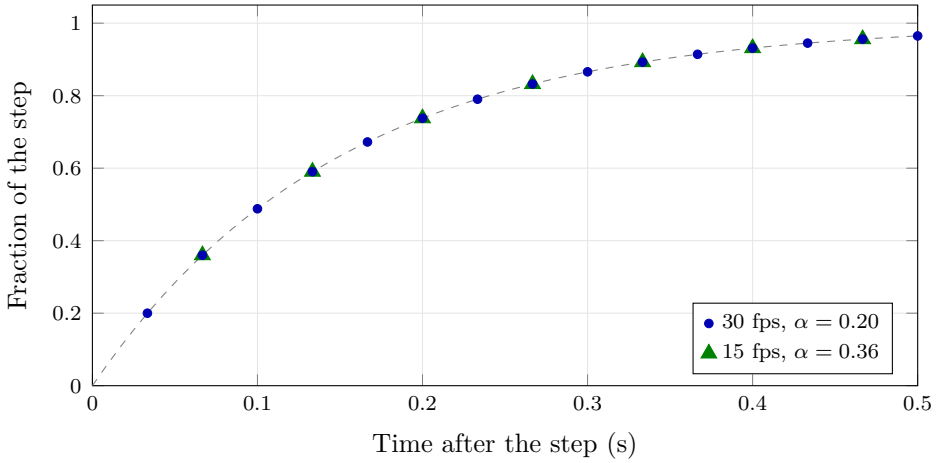


Figure 7.6: Response of the filter of Equation (7.3) to a step of the input, with $\alpha_{\text{ref}} = 0.20$. At 15 frames per second, the factor converted with Equation (7.5) gives the same response in time as at 30 frames per second (dashed line).

Parameters per status and jumps. When the status is `ok` or `degraded`, the filter updates the error with a reference smoothing factor α_{ref} and a jump threshold J that depend on the status:

$$(\alpha_{\text{ref}}, J) = \begin{cases} (0.20,\ 40 \text{ px}) & \text{if the status is } \texttt{ok}, \\ (0.18,\ 38 \text{ px}) & \text{if the status is } \texttt{degraded}. \end{cases} \tag{7.6}$$

In our case the values for the two statuses are close to each other and worked well. The filter keeps them separate, so that the values for `degraded` can be changed if the segmentation is worse. When the status is `lost`, the error is not updated and the robot stops after a short hold, described at the end of this section.

The jump is the distance $\Delta_k = |e_k - \hat{e}_{k-1}|$ between the new measurement and the previous filtered error. The filtered error is

$$\hat{e}_k = \begin{cases} (1-\alpha)\,\hat{e}_{k-1} + \alpha\, e_k & \text{if } \Delta_k \le J, \\ e_k & \text{if } \Delta_k > J,\ \Delta_{k-1} > J \text{ and } |e_k - e_{k-1}| \le J_c, \\ \hat{e}_{k-1} & \text{otherwise.} \end{cases} \tag{7.7}$$

A measurement within the threshold is smoothed with Equation (7.3). A jump on an isolated frame is ignored, so a segmentation error on a single frame does not reach the command. A jump present in two consecutive frames is taken directly only if the two measurements agree, that is, if they differ by at most the tolerance J_c. Otherwise both frames are ignored. Taking the measurement directly lets the filter follow a sudden real change of the centerline position, for example when an

obstacle under a wheel suddenly rotates the robot, without waiting for the slower response of Equation (7.3).

We chose J_c in offline tests, by applying the filter to recorded errors of Strip-IRLS. When two consecutive jumps agreed on the new position of the centerline, their measurements differed by at most 18 px, while in the other cases they differed by more than 50 px. $J_c = 25$ px accepts the first case with some margin and rejects the second.

Steering law. The steering law uses the command error

$$e_{\text{cmd}} = \begin{cases} 0 & \text{if } |\hat{e}_k| < 4 \text{ px}, \\ \hat{e}_k & \text{otherwise.} \end{cases} \tag{7.8}$$

This deadband avoids small oscillations of the steering around the center of the row. It acts only on the command, so the filter keeps its internal value $\hat{e}_k$. The angular velocity target is

$$\omega^* = \text{sat}_{\omega_{\max}}\left(-K_p\,(0.6 + 0.4\,C)\,\frac{e_{\text{cmd}}}{W/2}\right), \tag{7.9}$$

with $K_p = 1.2$, where $\text{sat}_{\omega_{\max}}$ limits the value to $\pm\omega_{\max} = \pm 0.58$ rad/s. Dividing by the half-width $W/2 = 112$ px gives the error as a fraction of the image, from 0 at the center to ± 1 at the edges, so K_p does not depend on the input resolution. The confidence $C \in [0, 1]$ of the path extractor reduces the gain when the result is less reliable. The effective proportional gain is $K_p(0.6 + 0.4\,C)$, which equals K_p with $C = 1$ and $0.75\,K_p$ with $C = 0.38$, the lowest value before `lost`. The minus sign makes the robot turn towards the centerline.

The limit $\omega_{\max}$ is chosen from v so that it matches the steering limit of 30° (Equation (7.2)): a larger ω would not turn the wheels further. The command therefore saturates for errors above

$$e_{\text{sat}} = \frac{\omega_{\max}\,(W/2)}{K_p\,(0.6 + 0.4\,C)}, \tag{7.10}$$

which is about 54 px with $C = 1$ (Figure 7.7a). The gain K_p sets how strongly the robot reacts to the error (Figure 7.7b). A larger K_p gives a larger command for the same error and reaches 30° at a smaller error. A smaller K_p gives a smoother response and reaches the limit later.

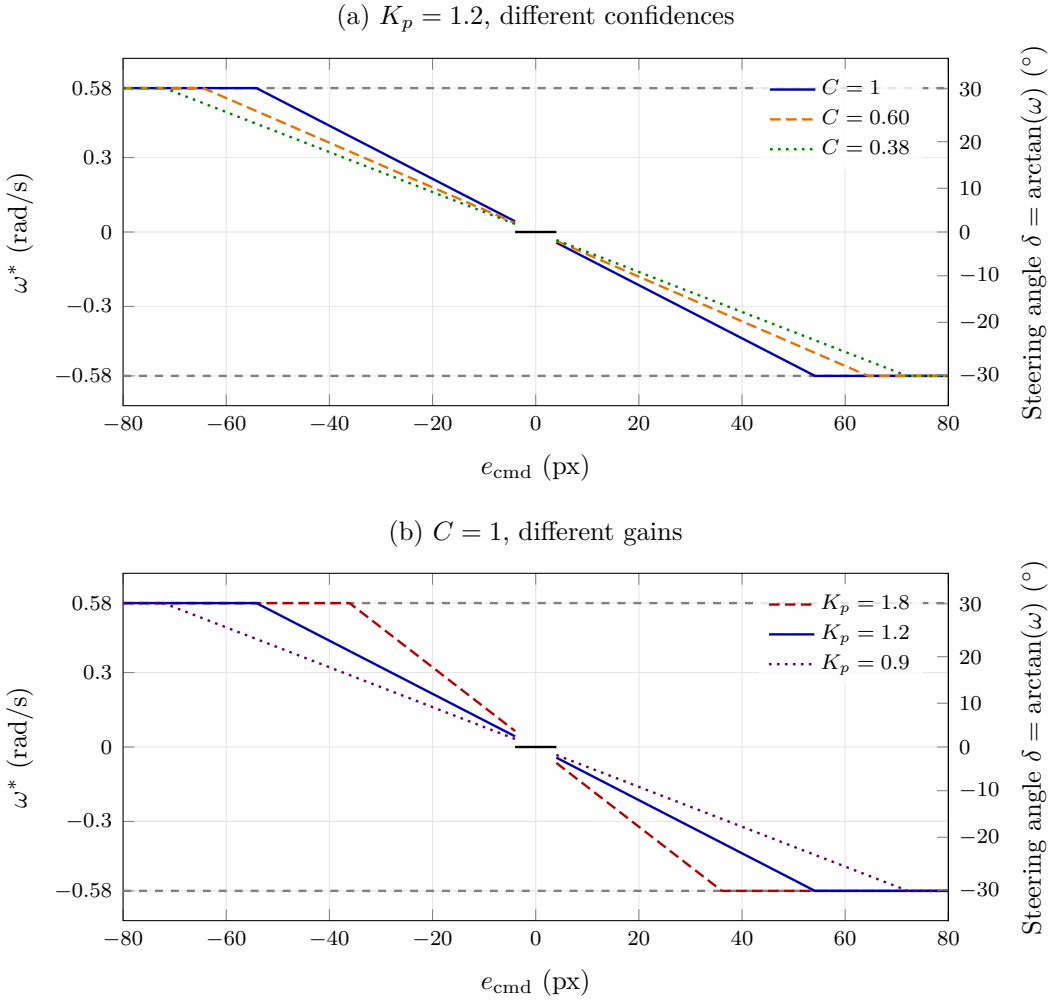


Figure 7.7: Angular velocity target ω^* of Equation (7.9) as a function of the command error e_{cmd}. (a) Effect of the confidence, with the gain used on the robot, $K_p = 1.2$. The curves are for $C = 1$, $C = 0.60$ (threshold between `ok` and `degraded`) and $C = 0.38$ (threshold below which the status is `lost`). (b) Effect of the gain, with $C = 1$. The command saturates at 36 px with $K_p = 1.8$, at 54 px with $K_p = 1.2$ and at 72 px with $K_p = 0.9$. In both plots the command is zero inside the deadband ($|e_{\text{cmd}}| < 4$ px) and is limited to ± 0.58 rad/s. The right axis gives the corresponding steering angle δ at $v = 0.20$ m/s, where Equation (7.2) becomes $\delta = \arctan(\omega)$, so 0.58 rad/s corresponds to 30°.

A second EMA smooths the angular velocity sent to the robot, $\omega_k = (1 - \alpha_w)\,\omega_{k-1} + \alpha_w\,\omega^*$, with $\alpha_w = 0.30$ at 30 frames per second (converted with Equation (7.5), so this filter is also independent of the frame rate). The linear velocity v is constant.

The values of K_p and $\omega_{\max}$ above hold for $v = 0.20$ m/s. The steering angle of Equation (7.2) depends on the ratio ω/v, so at a higher speed the same ω turns the wheels less. If only v is increased, the robot steers less for the same error and can no longer reach the maximum steering angle of 30°. At $v = 0.50$ m/s, for example, $\omega_{\max} = 0.58$ rad/s gives at most 13°.

This can be a desired behavior. The same steering angle gives the same turning radius R at any speed (Equation (7.1)), so the lateral acceleration v^2/R grows with the square of the speed, and steering less at a higher speed keeps it lower.

If instead the same steering angle is wanted for each error, $\omega_{\max}$ and K_p can be multiplied by the same factor as v:

$$\omega_{\max} = \frac{2\,v\tan 30^\circ}{L} \approx 2.89\,v, \qquad K_p = 1.2\,\frac{v}{0.20\ \text{m/s}}. \tag{7.11}$$

In this way $\omega_{\max}$ still corresponds to 30° and each error gives the same steering angle as at 0.20 m/s. At $v = 0.30$ m/s, for example, $\omega_{\max} = 0.87$ rad/s and $K_p = 1.8$.

Missing centerline and stop. When the status is `lost` or no centerline is available, the node keeps the last filtered error for up to $t_{\text{hold}} = 0.5$ s. After this time it publishes a zero velocity command. After a stop, the error filter restarts from the first new measurement and the filter of the angular velocity from zero. Table 7.2 lists the parameters used on the robot.

Table 7.2: Parameters of the error filter and of the steering law used on Demetra.

Parameter	Symbol	Value
Input resolution ($W \times H$)	–	224×224 px
Reference row	y_{ref}	156 px
Reference frame rate of the smoothing factors	–	30 fps
Smoothing factor, status `ok`	α_{ok}	0.20
Smoothing factor, status `degraded`	α_{deg}	0.18
Jump threshold, status `ok`	J_{ok}	40 px
Jump threshold, status `degraded`	J_{deg}	38 px
Frames that confirm a jump	–	2
Coherence tolerance of two jumps	J_c	25 px
Deadband	–	±4 px
Proportional gain	K_p	1.2
Maximum angular velocity	$\omega_{\max}$	0.58 rad/s
Smoothing factor of the angular velocity	α_w	0.30
Linear velocity	v	0.20 m/s
Hold time	t_{hold}	0.5 s

7.3 Deployment and testing

7.3.1 Timing and latency

The inference times in Table 5.1 were measured on the training GPU at the original image resolution, so they do not describe the onboard model. The times in this section were measured on the Jetson AGX Orin in the MAXN power mode, with the clock frequencies fixed at their maximum by `jetson_clocks`, so that the runs are comparable. For each path extractor, the controller node processed the same recording of a row, played at its original speed, once with the debug video disabled and once with it enabled. For each run, 100 callbacks were measured. Table 7.3 lists the mean and standard deviation of the time of the path extractor and of the whole callback.

Table 7.3: Time of the path extractor and of the whole callback on the Jetson AGX Orin (mean ± standard deviation, ms). The inference takes about 5.8 ms, and the other steps of the callback (decoding, resizing, filter, command and log) about 4.7 ms without the debug video.

Extractor	Path extractor	Callback, debug off	Callback, debug on
Strip-IRLS	11.07 ± 0.21	21.62 ± 0.24	53.73 ± 0.69
EdgeNet	4.68 ± 0.18	14.72 ± 0.22	47.64 ± 0.71
RANSAC edges	15.52 ± 0.24	26.24 ± 0.28	57.75 ± 0.77
Center points	7.27 ± 0.27	17.97 ± 0.30	47.40 ± 0.72
Column histogram	0.91 ± 0.04	11.30 ± 0.14	41.45 ± 0.31

Without the debug video, the callback takes from 11.3 ms with Column histogram to 26.2 ms with RANSAC edges, less than the 33.3 ms frame period of the camera. The node therefore processes every camera frame, and at $v = 0.20$ m/s the robot moves about 0.67 cm between two processed frames. The debug video adds 29 to 33 ms, and every callback becomes longer than the frame period. The node then processes 17 to 24 frames per second and skips the others, so the robot moves 0.8 to 1.2 cm between two processed frames. The smoothing factors are corrected for the time between frames (Equation (7.5)), so the response of the filters in time does not change. The difference between the two cases is small, so the field tests of Chapter 8 were run with the debug video enabled, to record the debug images. In normal operation it should still be disabled, so that every frame is used.

7.3.2 Processor-in-the-Loop test

Before the field tests, the complete pipeline was tested in the laboratory with the robot raised, so that the wheels could steer without moving the robot. The purpose

of the test was to check that the whole chain, from the image to the steering of the wheels, gives steering commands coherent with the position of the row in the image. A simulation node replaces the camera. It publishes a recorded frame of a row on the camera topic at 30 frames per second, and the controller node runs on these frames without changes. Both the simulation node and the controller node ran on the Jetson AGX Orin of the robot, and the driver sent the commands to the real steering actuators. The simulation node keeps the lateral offset d and the heading ψ of a simulated robot with respect to the row. The offset d is positive when the robot is to the right of the center of the row, and ψ is positive when the robot points to the left. At each step of $\Delta t = 1/30$ s, the node converts the velocity command into the steering angle δ with Equation (7.2), limited to $\pm 30°$, and updates the pose with the kinematic model of the robot:

$$\psi_k = \psi_{k-1} + \frac{2\, v \tan \delta}{L}\, \Delta t, \qquad d_k = d_{k-1} - v \sin \psi_k\, \Delta t. \tag{7.12}$$

It then shifts the frame horizontally by

$$s = -\frac{f}{D}\, d + f\, \psi \tag{7.13}$$

pixels, positive to the right, where $f = 465$ px is the nominal focal length of the camera and $D = 1.25$ m is the assumed distance of the image row where the error is measured. A lateral offset therefore moves the row in the image by $f/D \approx 372$ px per meter, and a rotation by 465 px per radian. During the test, a lateral offset was added from the keyboard. The simulated pose was computed from the velocity command, assuming that the wheels follow the commanded steering angle, and the wheels visibly reached the commanded angle. Figure 7.8 shows three frames of the test.

The test checked the integration of the perception, control and actuation chain. The wheels turned forward, and after each displacement they steered towards the center of the row while the simulated offset returned to the center without oscillating. However, the test is not a realistic simulation of row navigation and does not validate physical path tracking, because the scene is a single shifted frame, the projection is approximate and the robot does not move on the ground. The network segments every frame, but always the same image shifted laterally. The segmentation and the path extraction along a real row were instead tested by playing recordings of the rows to the controller node, as in Section 7.3.1. A visual check of the debug video showed that most of the extracted centerlines were correct.

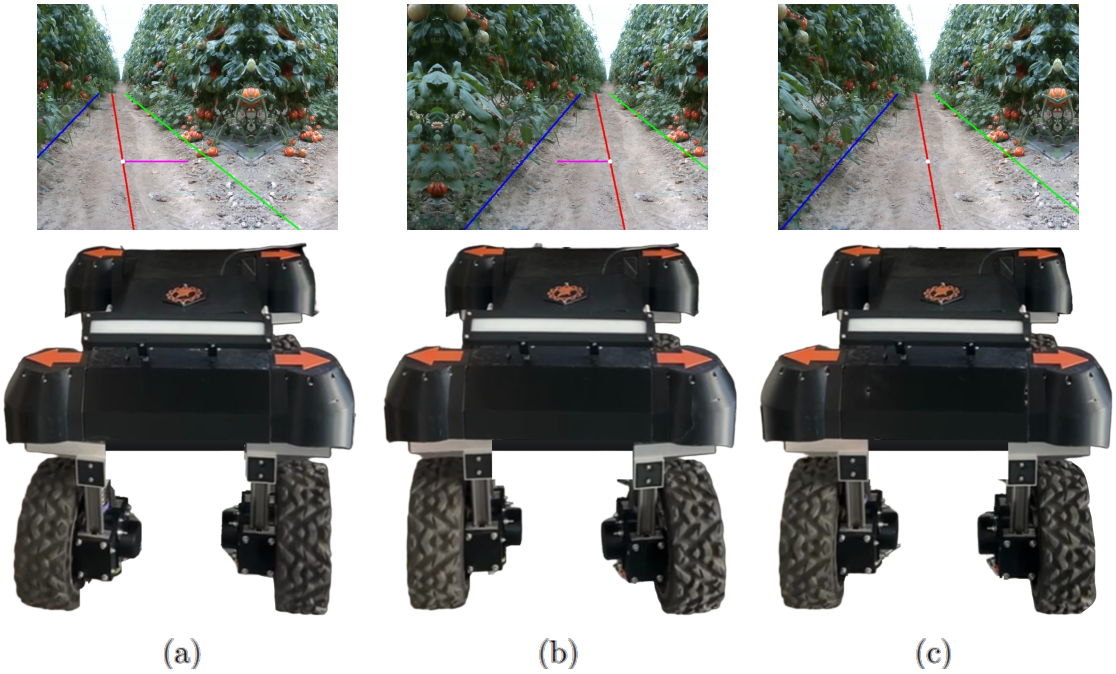


Figure 7.8: Processor-in-the-Loop test. Top: frame of the simulation node shifted with Equation (7.13), with the edges (blue and green), the centerline (red) and the lateral error at y_{ref} (magenta) extracted offline by the network and Strip-IRLS. Bottom: the raised robot during the test, at similar lateral errors. (a) and (b) Lateral error of -48 px and $+39$ px: the wheels steer in opposite directions to bring the simulated robot back to the center. (c) Error below 1 px: the simulated robot is back at the center and the wheels are straight. The mirrored vegetation comes from the border that the simulation node adds before the shift.

Chapter 8

Experimental Validation

This chapter reports the experimental evaluation of the segmentation models, geometric extractors, and integrated navigation system. It covers offline evaluation using recorded images and, if completed, closed-loop field trials.

8.1 Offline evaluation

8.1.1 Segmentation metrics

Segmentation performance is measured using intersection over union (IoU) for each semantic class. For a class c, IoU is defined as:

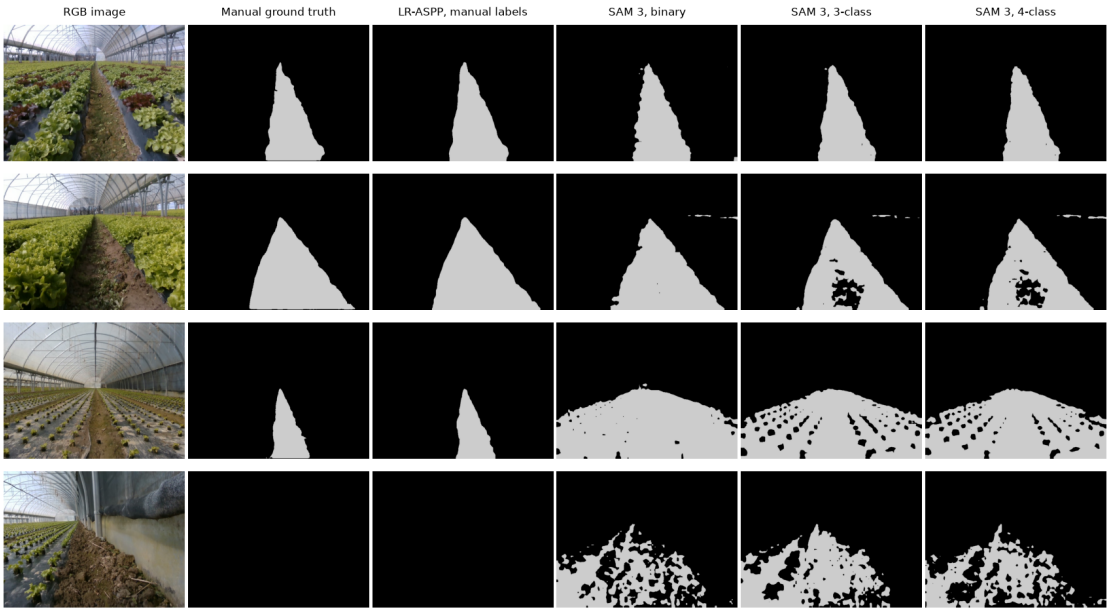
$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \tag{8.1}$$

where true positives, false positives, and false negatives are computed against the manual reference masks.

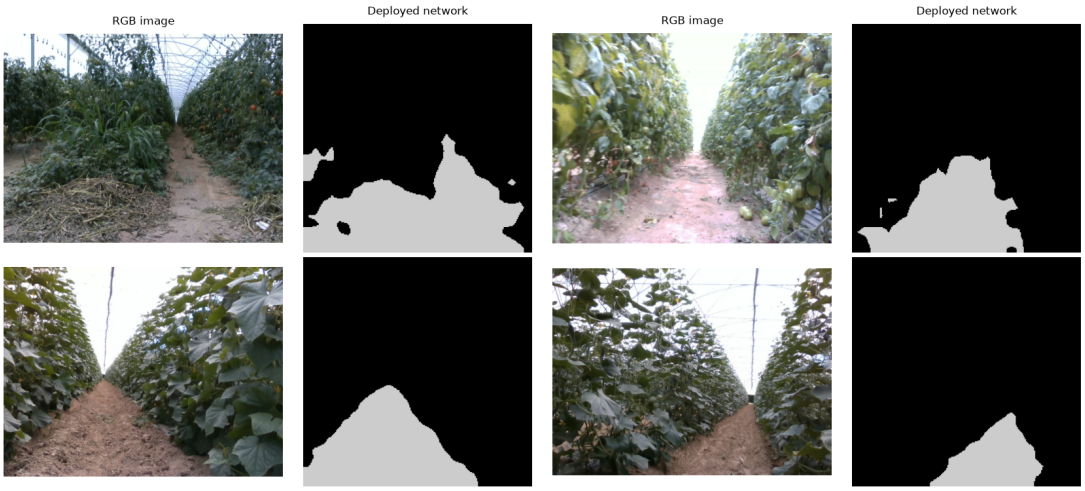
Mean IoU (mIoU) averages the per-class IoU values. For the four-class configuration, mIoU includes background, plants, path, and covering material. Path IoU is the task-specific metric most relevant for navigation.

Architecture comparison results

The architecture comparison in Section 5.3.3 reports, on the 77 test images of the manual lettuce dataset, a path IoU of 0.9532 for LR-ASPP, against 0.9654 for FCN and 0.9686 for DeepLabV3. The recorded GPU inference times are 8.21 ms, 59.10 ms, and 86.90 ms. LR-ASPP accepts an mIoU about 1.1 percentage points lower in exchange for 7.2 to 10.6 times shorter recorded times.



(a) Lettuce test images



(b) Tomato (top) and cucumber (bottom)

Figure 8.1: Visual comparison of segmentation outputs. (a) Four of ten test images of the manual lettuce dataset, taken at evenly spaced positions of the test split: RGB image, manual ground truth, and the predictions of the LR-ASPP network trained on the manual labels and of the three LR-ASPP networks trained only on SAM 3 lettuce labels. In the third image the SAM 3 networks mark the whole floor as path, including the strips beside the central corridor, and in the fourth, where the manual ground truth has no path, they mark the soil in front of the robot. (b) Tomato and cucumber images, which have no ground truth: RGB image and prediction of the binary network installed on the robot, as a qualitative comparison only.

Supervision strategy results

The supervision comparison is reported in Section 5.4.2. On the test split, the manual model reaches a path IoU of 0.9532 on the complete test set and 0.9587 in the realistic scenario. The three networks trained only on SAM 3 lettuce labels reach path IoU values between 0.8966 and 0.9013 in the realistic scenario and between 0.3917 and 0.4124 on the complete test set. The 4-class network reaches an mIoU of 0.5925 in the realistic scenario. A network with the deployed multi-crop configuration, trained on the complete SAM 3 collection without the lettuce test images, reaches 0.9071 in the realistic scenario and 0.6369 on the complete test set. The deployed weights themselves have no independent IoU (Section 5.4.3).

8.1.2 Geometric reference accuracy

The two extractors developed in this work, Strip-IRLS and EdgeNet, and the three baselines of Chapter 6, which are the five extractors tested on the robot, are compared offline on the same masks with the parameters of that chapter. Table 8.1 lists the four data sets.

Table 8.1: Data sets of the offline comparison of the extractors.

Set	Masks	Ground truth	Use
Synthetic	2500 (25 scenarios)	Exact	Failure cases: straight corridors with one disturbance, adjacent corridors, curve
Synthetic, tomato shapes	1000 (10 scenarios)	Exact	Failure cases on the shapes of the real tomato masks
Annotated real frames	255 of 308	Edges marked by hand	Accuracy on real masks
Real masks	1604	None	Status and agreement between extractors

The synthetic masks have 224×224 pixels. Each one is a straight corridor in perspective, defined by its bottom center, vanishing point, bottom half-width and apex row, with irregular edges (a slow undulation of a few pixels and a fine grain of 1–2 px). The first set has 21 base scenarios with one disturbance each, two of them without a corridor (an empty mask and a mask that is all path), and 4 scenarios with adjacent corridors or a curved corridor, layouts that were not present in the test field. The second set takes its shapes from 100 masks predicted

on frames evenly spaced in the recording `pomodori2` (9465 compressed images of the topic `/camera/camera/color/image_raw/compressed`, taken in a greenhouse with tomato plants), segmented as in the controller node (Chapter 7): apex low in the image, wide corridor, large-scale wavy edges, bottom corners cut by the nearby plants, and a band of path across the bottom of the image with no corridor. Table 8.2 lists the scenarios. The masks generated with seed 0 were used during the development of the extractors, and the results reported here are computed on a separate set generated with seed 2027.

The real masks were predicted by the network deployed on the robot on the images of 13 sequences of the SAM 3 dataset (Chapter 5): 866 of tomato in 7 sequences, 268 of cucumber in 2 sequences and 470 of lettuce in 4 sequences. They are reduced from 640×480 to 224×224 pixels with nearest-neighbor interpolation. These images belong to the training data of the network, so the masks are probably cleaner than those of images not seen in training. 348 masks without path pixels and four lettuce sequences in which the network predicted almost no path (7 further masks) were removed. The annotated frames are described in Section 8.1.2.

The error is $|\hat{x} - x_{\text{true}}|$ at the row used by the controller, $y_{\text{ref}} = 156$ (Chapter 7), and is computed only on the frames not classified as `lost`. In the tomato-shaped set the apex can lie below y_{ref}, and the ground truth is the linear extension of the centerline, as the extractors extend their lines. A gross error is an error above 15 px (13% of the half-width of the image). For the masks without a corridor the expected status is `lost`. A failure is a frame that is `lost` or has a gross error. All five extractors are deterministic: RANSAC edges draws its samples from a generator with a fixed seed, and the others do not sample. Their times on the Jetson AGX Orin are in Section 7.3.1.

Results on the annotated real frames

The annotated frames give a ground truth on real data. From each of the 14 real sequences, 22 frames evenly spaced in the sequence were selected (308 frames). In a browser tool written for this purpose, the author marked two points on each corridor edge in the 640×480 photo, with the predicted mask shown as a semi-transparent overlay, along the line where the plants meet the ground. Each edge is the line through its two points, scaled to the 224×224 mask as in the controller node, and the true centerline is the mean of the two edge lines, as in Eq. (6.2). The error is measured at y_{ref} as for the synthetic masks.

The sequences were split before the evaluation into a development part (`cetrioli_grandi`, `insalata_grande1`, `insalata_media1`, `pomodori_maturi1`, `pomodori_maturi3`, `pomodori_maturi5` and `pomodori2`) and a test part (the other seven), each with frames of the three crops. `pomodori2` is in the development part because the thresholds of RANSAC edges were chosen on its frames. The

Table 8.2: Scenarios of the three synthetic sets (100 masks each). Unless stated otherwise, the corridors of the first two sets have a bottom half-width of 45–70 px and the apex at row 40–90, and those of the third set a bottom half-width of 55–110 px and the apex at row 130–170.

Scenario	Content
Base set (21 scenarios)	
`clean`	Clean corridor, robot almost centered
`offset`	Robot offset laterally by 20–55 px at the bottom
`heading`	Robot rotated, vanishing point moved by 30–90 px
`offset+heading`	Offset and rotated
`narrow`, `wide`	Bottom half-width 18–28 px or 85–105 px
`blobs`	5–14 false-path spots in the ROI
`holes`	4–11 holes inside the corridor
`hole_bottom`	Central hole at the bottom, 10–35 rows high
`hole_bottom_tall`	Central hole at the bottom, 45–60 rows high
`bulge_one_side`	Widening on one side, 20–60 px over 15–40 rows
`bite_one_side`	Intrusion on one side, 10–30 px over 15–40 rows
`jagged`	Jagged edges, 3–10 px per row
`shadow_gap`	Empty horizontal band 8–28 rows high
`bottom_full`	Last 10–40 rows all path
`row_end`	End of row, no path above row 112–145
`edge_out`	Robot far from the center, one edge leaves the image
`speckle`	Salt-and-pepper noise, 2–8% of the pixels inverted
`combined`	Spots and holes, robot offset and rotated
`empty`, `full`	Empty mask, mask all path (expected `lost`)
Adjacent corridors and curve (4 scenarios)	
`adjacent`	Two copies of the corridor on the left and right, with the same vanishing point, separated at the bottom by a row 40–110 px wide
`adjacent+offset`	As `adjacent`, robot offset by 25–50 px at the bottom
`adjacent_narrow_crop`	As `adjacent`, row 15–35 px wide
`curved`	Centerline bent by a quadratic term, 9–27 px at y_{ref}
Set based on the tomato masks (10 scenarios)	
`pom_pulito`	Clean corridor with low apex, wide at the bottom
`pom_ondulato`	As `pom_pulito`, edges wavy at large scale (5–12 px)
`pom_cupola`	Bottom corners cut by up to 35 px per side (nearby plants)
`pom_spostato`	Robot close to one row (offset 45–90 px), one edge leaves the image
`pom_ruotato`	Robot rotated (vanishing point moved by 40–100 px), half of the masks with the top cut by an oblique line
`pom_piccolo`	Small path, apex at row 165–195, below y_{ref}
`pom_grande`	Large path, apex at row 10–50, bottom wider than the image
`pom_macchie`	3–9 false-path spots in the upper part and 0–2 in the bottom corners
`pom_banda`	Band of path across the bottom, no corridor (expected `lost`)
`pom_combinato`	Offset and rotated, wavy edges, cut corners and spots

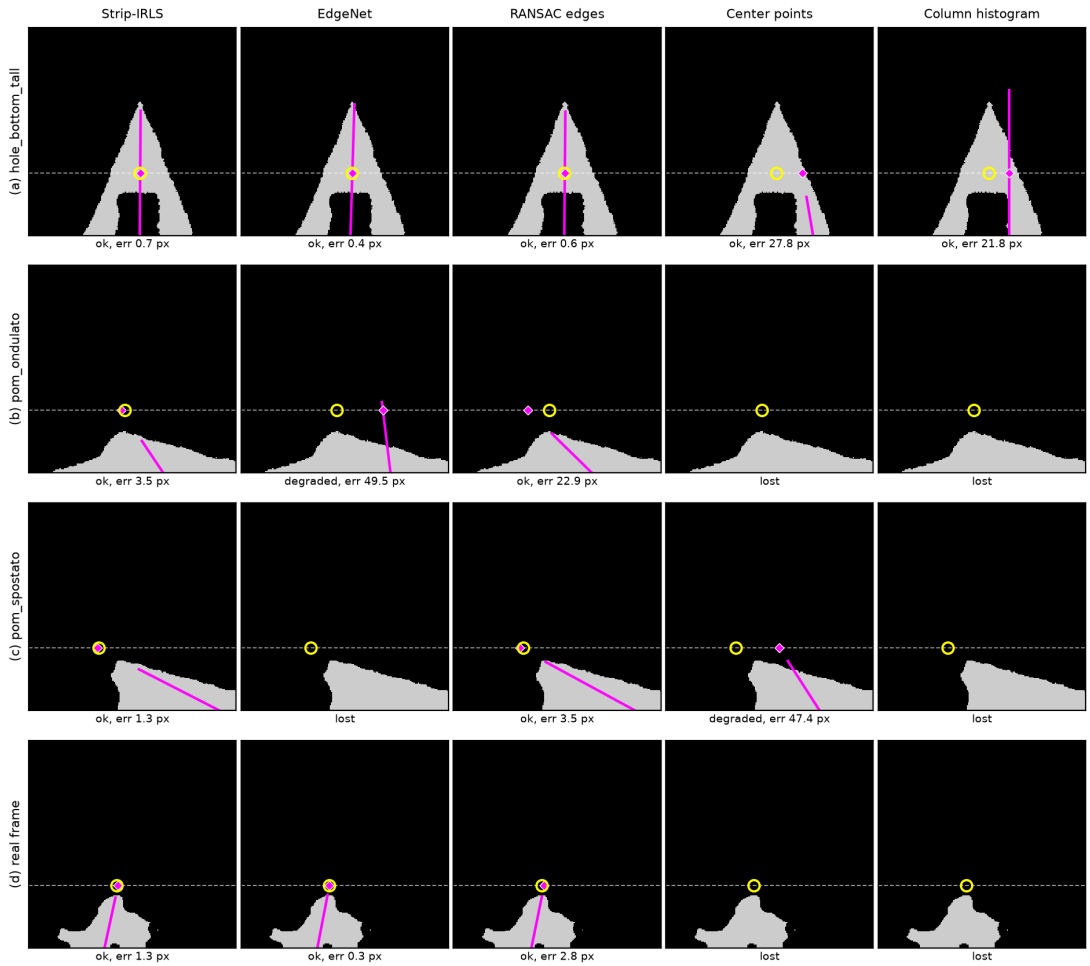


Figure 8.2: Lines estimated by the five extractors on four masks with disturbances, chosen among the masks where Strip-IRLS has an error below 4 px and at least two other extractors fail. (a) `hole_bottom_tall`, (b) `pom_ondulato` and (c) `pom_spostato` are masks of the synthetic test sets, and (d) is the real frame `pomodori2_01051`, whose true centerline comes from the hand annotation. The magenta line is the estimated centerline, the diamond its position at y_{ref} (dashed line), and the yellow circle the true centerline at y_{ref}. Below each panel are the status and the error at y_{ref}. A panel without a line is `lost`.

development frames were used to choose the version of Strip-IRLS. The test frames were evaluated once for the other extractors, after they had been fixed, and four times for Strip-IRLS: with $\rho = 0.50$ and gap 30 px (4.1%, 5 of 122 gross errors), with $\rho = 0.30$ and gap 30 px (2.5%, 3 of 122), with $\rho = 0.30$, gap 40 px and the strips of 9 rows of an earlier version (3.3%, 4 of 122), and with the final version, reported below. The final version tiles the ROI exactly with strips of 10 rows (Section 6.2.2) and uses ten reweighting iterations. Both changes were checked on the development data, where the gross errors on the annotated frames stayed at 3 frames, and the test frames were evaluated after them. After this evaluation the merging gap was changed from 40 to 30 px, the value used on the robot, and no result on the annotated frames changed. The parameters were chosen on the development data. The annotator skipped 12 development frames and 22 test frames, mostly of tomato, where the edges could not be read. The tomato sequences of the SAM 3 dataset were recorded with a handheld camera, and in many of their frames the image is shaken or the segmentation is poor. In 10 development frames and 9 test frames one annotated edge is inclined by more than 60° from the vertical. There the mean of the two lines in the form of Eq. (6.1) is not a usable centerline, and its value at y_{ref} can lie hundreds of pixels outside the image, so these frames are excluded from the error statistics. The threshold of 60° was chosen on the development frames. 132 development frames and 123 test frames remain. None of them was annotated as a frame without a corridor.

Table 8.3: Results on the annotated real frames. Gross errors are errors above 15 px at y_{ref} on the non-`lost` frames, given as a percentage and as a count. Failures are the frames that are `lost` or have a gross error.

	This work		Baselines		
	Strip-IRLS	EdgeNet	RANSAC edges	Center points	Column histogram
Development, 132 frames					
Gross errors (%)	2.3 (3/128)	3.0 (4/132)	2.3 (3/129)	5.5 (6/110)	12.9 (16/124)
`lost` (%)	3.0	0.0	2.3	16.7	6.1
Failures	7	4	6	28	24
Gross errors classified `ok`	3	3	3	1	15
Median error (px)	1.69	1.69	1.49	2.13	5.08
Test, 123 frames					
Gross errors (%)	2.5 (3/122)	6.6 (8/121)	0.8 (1/121)	7.6 (8/105)	18.8 (22/117)
`lost` (%)	0.8	1.6	1.6	14.6	4.9
Failures	4	10	3	26	28
Gross errors classified `ok`	3	5	1	0	22
Median error (px)	2.32	2.00	1.29	2.45	5.65

Strip-IRLS and RANSAC edges have the fewest failures on both parts. Their difference is a few frames and their order changes between the two parts, so these

frames do not separate them. Center points and Column histogram fail on 24 to 28 frames per part. The network has few failures on the development part and 10 on the test part. The three gross errors of Strip-IRLS on the test frames are all in the tomato sequence `pomodori_maturi6`, as is the one of RANSAC edges. Most gross errors of Strip-IRLS, RANSAC edges and EdgeNet are classified as `ok`, so their status does not warn the controller.

These results have four limits. The 22 frames of a sequence are not independent. On the test frames the gross errors of Strip-IRLS and RANSAC edges are all in one tomato sequence, and Strip-IRLS, RANSAC edges and EdgeNet have no gross errors on lettuce, so the comparison rests on about 100 frames. The annotation was made once by one person, with the predicted mask shown in overlay, which can draw the annotation toward the mask, and its error was not measured. The annotated centerline is therefore not exact like the synthetic one, and differences of a few pixels in the median error, which are 1.3 to 2.3 px for Strip-IRLS, RANSAC edges and EdgeNet, are not interpreted: the comparison relies on the gross errors. The ground truth is marked on the photo, so the measured error also contains the segmentation error, which is the same for all extractors. The excluded frames, those skipped and those with a nearly horizontal edge, are among the most difficult, so the reported rates are lower than on all frames.

Results on the synthetic masks

Table 8.4 reports the results on the synthetic test masks, and Table 8.5 the gross error rate in the scenarios where at least one extractor exceeds 5%.

Table 8.4: Results of the five extractors on the synthetic test masks (seed 2027). Gross errors are errors above 15 px at $y_{\text{ref}} = 156$ on the non-`lost` frames.

	This work		Baselines		
	Strip-IRLS	EdgeNet	RANSAC edges	Center points	Column histogram
Base scenarios, 1900 masks with a corridor					
Gross errors (%)	0.3	0.1	0.1	4.3	23.2
`lost` (%)	0.4	0.0	0.0	3.4	1.1
Gross errors classified `ok`	4/6	1/2	1/1	25/78	406/436
`empty` and `full` rejected (%)	100	100	100	50	50
Adjacent corridors and curve, 400 masks					
Gross errors (%)	2.3	0.0	18.5	0.5	42.6
`lost` (%)	1.0	6.2	0.0	0.2	0.2
Gross errors classified `ok`	5/9	–	74/74	1/2	170/170
Tomato shapes, 900 masks with a corridor					
Gross errors (%)	2.8	10.7	1.5	11.9	4.6
`lost` (%)	0.1	0.7	0.8	5.0	30.6
Gross errors classified `ok`	24/25	72/96	13/13	6/102	12/29
Band without corridor rejected (%)	87	91	98	82	70

Table 8.5: Gross error rate (%) on the synthetic test masks (seed 2027) in the scenarios where at least one extractor exceeds 5%. Scenarios with the prefix `pom_` belong to the tomato-shaped set. In `pom_piccolo`, Column histogram rejects all masks.

	This work		Baselines		
Scenario	Strip-IRLS	EdgeNet	RANSAC edges	Center points	Column histogram
`adjacent`	3.0	0.0	13.0	0.0	0.0
`adjacent+offset`	4.2	0.0	61.0	2.0	71.0
`curved`	0.0	0.0	0.0	0.0	99.0
`offset`	0.0	0.0	0.0	0.0	64.0
`heading`	0.0	0.0	0.0	0.0	92.0
`offset+heading`	0.0	0.0	0.0	0.0	59.0
`edge_out`	0.0	0.0	0.0	0.0	74.0
`row_end`	0.0	0.0	0.0	0.0	21.6
`combined`	0.0	0.0	1.0	0.0	44.3
`holes`	0.0	0.0	0.0	0.0	5.2
`hole_bottom`	0.0	0.0	0.0	0.0	9.2
`hole_bottom_tall`	2.0	0.0	0.0	71.9	70.4
`bulge_one_side`	3.1	2.0	0.0	11.5	0.0
`pom_ondulato`	7.0	9.0	6.0	26.1	6.8
`pom_combinato`	10.1	19.0	3.0	15.5	4.3
`pom_spostato`	1.0	22.9	0.0	52.1	5.3
`pom_piccolo`	4.0	7.1	2.1	8.2	—
`pom_ruotato`	3.0	38.0	2.0	6.0	7.5
`pom_grande`	0.0	0.0	0.0	0.0	12.0

Each extractor fails in different scenarios:

- **Strip-IRLS.** In `hole_bottom_tall` a central hole 33 to 50 px wide splits the bottom of the corridor. Strip-IRLS merges columns separated by at most 30 px, so it does not join the two parts where the hole is wider. A hole wider than the gap and taller than the seed window (six strips of 10 rows) puts every seed in one of the two parts. The chain that starts from it stops where the parts join, because there the interval widens beyond the jump limit, and its edges are the two edges of that part. This causes gross errors in 2.0% of the masks of this scenario. In `adjacent+offset` the merging gap joins the narrow upper part of the row between two corridors with the corridors beside it, and 4.2% of the masks have a gross error.
- **EdgeNet.** Its errors are in the tomato-shaped scenarios where the apex lies below y_{ref} and the true centerline is near the image border (`pom_ruotato`, `pom_spostato`, `pom_combinato`). There the value at y_{ref} is an extension of the two lines, and a small error on one edge becomes a large error. With adjacent corridors it has no gross errors, but it classifies only 31.5% of these masks as `ok`, because the mask also contains the adjacent corridors and the intersection over union in its confidence is low. A first model, trained with an earlier generator without adjacent corridors and with the apex only between rows 30 and 100, had 21.3% gross errors on the tomato-shaped set and 95.7% in `adjacent+offset`, so the accuracy of the network depends on the coverage of its generator.
- **RANSAC edges.** It takes the first and last path pixel of each row of the selected component, so when an adjacent corridor belongs to that component its edge replaces the edge of the central corridor in those rows.
- **Center points.** It forms the midpoints of the intervals before the fit, so a disturbance on one side or in the middle of the corridor moves the midpoints and enters the centerline (`bulge_one_side`, `hole_bottom_tall`, `pom_spostato`). It returns a line on the mask that is all path. With adjacent corridors it follows one interval per strip and has almost no gross errors.
- **Column histogram.** It returns a vertical line, so its error grows with the offset and the rotation of the robot and on the curved corridor. Its confidence is the maximum column count divided by the ROI height, which is low when the path starts below the middle of the image, so it rejects 30.6% of the tomato-shaped masks and all those of `pom_piccolo`.

On the curved corridor, the straight-line extractors keep the error below 15 px at y_{ref}, but their lines do not follow the curve. As on the annotated frames, most gross errors of Strip-IRLS, RANSAC edges and EdgeNet are classified as `ok`.

Choice of the parameters of Strip-IRLS

The ROI start and the merging gap of Strip-IRLS were chosen on the development data, the seed 0 masks and the development part of the annotated frames. Moving the requested ROI start from $\rho = 0.50$ to $\rho = 0.30$ at a gap of 30 px lowered the gross errors on the base scenarios from 6.0% to 0.5% and on `hole_bottom_tall` from 100% to 1.0%. On the development part of the annotated frames it moved them from 1.6% (2 of 126) to 2.3% (3 of 128). Table 8.6 shows the effect of the merging gap with $\rho = 0.30$: a larger gap joins more holes inside the corridor but also more rows between adjacent corridors, and on the annotated frames it makes no difference from 20 to 40 px. A gap of 40 px was chosen with the strip geometry of the earlier version. With the final geometry the table shows that 30 px has fewer errors on `adjacent+offset` (3.1% against 15.0%) and 1.0% instead of 0.0% on `hole_bottom_tall`, so the final version uses 30 px, the value used on the robot. Variants with an MSAC fit of the edges [24] and with parameters chosen on the synthetic errors had fewer synthetic errors, but they were not better on the development part of the annotated frames, and they were not kept.

Table 8.6: Effect of the merging gap of Strip-IRLS ($\rho = 0.30$) on the development data: gross error rate (%) on the seed 0 masks and on the development part of the annotated frames.

Gap (px)	Base scenarios	Adjacent and curve	`hole_bottom_tall`	`adjacent+offset`	Annotated frames
20	0.6	2.5	4.0	3.0	2.3
30	0.5	1.3	1.0	3.1	2.3
40	0.7	5.1	0.0	15.0	2.3
50	0.8	10.8	0.0	40.9	3.1
60	1.1	18.8	0.0	74.2	3.1

Results on the real masks

Table 8.7 reports the status of the five extractors on the 1604 real masks. Without ground truth, the status shows whether an extractor accepts the frame, not whether its line is correct.

On lettuce all five extractors classify more than 95% of the frames as `ok`. On tomato and cucumber, Strip-IRLS, RANSAC edges and EdgeNet classify most frames as `ok`, and Center points most frames as `degraded`.

On the frames where two extractors both return a line, their centerlines at y_{ref} differ by more than 15 px in 2.4% of the cases for Strip-IRLS and RANSAC edges (35 of 1429 frames), in 5.8% and 5.5% for EdgeNet with Strip-IRLS and with RANSAC edges, in 6.6 to 12.8% for the pairs that include Center points, and in 24.6 to 36.1% for the pairs that include Column histogram. Of the 35 frames

Table 8.7: Status of the five extractors on the real masks: share of `ok` / `degraded` / `lost` frames (%).

	This work		Baselines		
Masks	Strip-IRLS	EdgeNet	RANSAC edges	Center points	Column histogram
Tomato (866)	81.2 / 4.2 / 14.7	85.2 / 7.9 / 6.9	89.5 / 0.0 / 10.5	13.9 / 61.1 / 25.1	60.0 / 30.3 / 9.7
Cucumber (268)	88.1 / 0.7 / 11.2	85.4 / 3.4 / 11.2	91.4 / 0.0 / 8.6	16.0 / 69.4 / 14.6	42.5 / 45.5 / 11.9
Lettuce (470)	99.4 / 0.2 / 0.4	99.4 / 0.4 / 0.2	99.6 / 0.0 / 0.4	95.7 / 3.8 / 0.4	96.2 / 3.4 / 0.4
All (1604)	87.7 / 2.4 / 9.9	89.4 / 4.9 / 5.7	92.8 / 0.0 / 7.2	38.2 / 45.7 / 16.1	67.7 / 24.9 / 7.4

where Strip-IRLS and RANSAC edges disagree, 18 belong to one tomato sequence (`pomodori_maturi1`), in which the mask is a triangular corridor that rises from a band of path as wide as the image. In the same sequence Strip-IRLS returns `lost` on 56 frames, and RANSAC edges returns a line on 39 of them. The images with the lines were inspected visually only for an earlier evaluation of Strip-IRLS, with $\rho = 0.50$ and gap 30 px (34 frames), where the disagreements were dome-shaped masks with the apex near or below y_{ref} and the upper boundary of the band in `pomodori_maturi1`. The inspection was not repeated for the final parameters. Without ground truth, it cannot be decided which line is correct.

8.2 Field validation

Field validation evaluates the integrated navigation system on the physical robot Demetra operating autonomously inside crop rows. Unlike offline evaluation, which isolates the geometric extractor on recorded images, closed-loop field tests exercise the full chain from camera acquisition through segmentation, geometric fitting, error conditioning, and steering actuation in the presence of real-world disturbances.

8.2.1 Ground-truth reference: laser sensor

To assess the robot's physical centering independently of the onboard vision pipeline under test, closed-loop field validation integrates an auxiliary laser sensor.

▶ DA COMPLETARE: Dettagli del sensore laser utilizzato per i test in campo

Inserire le specifiche esatte del sensore laser impiegato nei test:

- Modello, costruttore e tipologia di misura (ad es. sensore di distanza laser, barriera ottica o profilometro).
- Configurazione di montaggio sul robot Demetra (posizione e orientamento).
- Interfaccia di comunicazione e frequenza di campionamento.

The laser measurement provides an independent reference signal to monitor the robot's lateral alignment along the crop row and evaluate whether the trajectory remains within the allowable tolerance corridor.

8.2.2 Field evaluation metrics

The following metrics are computed for each test run. All algorithms under comparison (Strip-IRLS, EdgeNet, RANSAC edges, Center points and Column histogram) are evaluated on the same physical row under equivalent conditions.

Laser centering rate

The laser centering rate P_{laser} quantifies the fraction of the autonomous run during which the robot remains centered within the laser reference corridor:

$$P_{\text{laser}} = \frac{T_{\text{on}}}{T_{\text{total}}} \times 100\ [\%], \tag{8.2}$$

where T_{on} is the cumulative time with positive laser detection and T_{total} is the total duration of the autonomous run. A higher P_{laser} indicates that the guidance algorithm maintains the robot closer to the physical corridor center for a larger fraction of the mission.

Number of centering losses

The number of centering loss events N_{exit} counts how many times the laser detector transitions from active (1) to inactive (0) during a run. Each event corresponds to the robot drifting outside the acceptable centering band. Frequent transitions indicate oscillatory tracking behavior, even if the overall centering rate P_{laser} remains acceptable.

Mission completion rate

The mission completion rate S_{mission} measures the fraction of the test row traversed autonomously without requiring manual intervention or emergency stop:

$$S_{\text{mission}} = \frac{L_{\text{auto}}}{L_{\text{row}}} \times 100\ [\%], \tag{8.3}$$

where L_{row} is the total length of the test row and L_{auto} is the distance covered before any intervention. A run terminated by collision with crop plants or by operator emergency stop records $S_{\text{mission}} < 100\%$.

Steering stability

Steering stability is evaluated from the angular velocity commands ω_k recorded on the `/cmd_vel` topic. Two indicators are computed:

- The standard deviation of angular velocity, $\sigma_\omega = \text{std}(\omega_k)$, which quantifies overall steering activity.
- The mean absolute jitter, $J_\omega = \frac{1}{N-1} \sum_{k=1}^{N-1} |\omega_{k+1} - \omega_k|$, which isolates high-frequency direction reversals independent of steady-state turning.

Lower values of both indicators correspond to smoother robot motion and reduced mechanical wear on the CubeMars AK70-10 steering actuators.

Effective forward speed

The effective forward speed v_{eff} is the ratio between the distance traversed and the elapsed time. If the error conditioning filter enters the temporal hold state and issues stop commands ($v = 0$), the effective speed drops below the nominal setpoint $v_{\text{nom}} = 0.20$ m/s. This metric captures how frequently each algorithm loses the guidance reference and forces the robot to halt.

8.2.3 Test conditions

▶ DA COMPLETARE: Descrizione delle condizioni sperimentali in campo

Dettagliare lo scenario delle prove sperimentali:

- Località del test, tipologia di coltura, sesto d'impianto (distanza tra i filari e altezza delle piante).
- Lunghezza del percorso navigato, tipologia del fondo (terreno lavorato, teli plastici pacciamanti, presenza di malerbe).
- Configurazione del sensore laser di riferimento (modello, posizionamento sul robot o nel filare, principio di misura).
- Condizioni meteorologiche e di luce (sole diretto, cielo coperto, ombre proiettate dalla vegetazione) e numero di ripetizioni effettuate per ciascun algoritmo.

8.2.4 Closed-loop navigation results

[SPAZIO FIGURA / SCHEMA]

Lateral tracking error and heading error profiles during inter-row autonomous navigation

Grafico dell'errore di inseguimento laterale e_y (cm) e dell'errore di orientamento e_θ (gradi) registrati nel tempo durante la navigazione del robot mobile all'interno dei filari coltivati.

Figure 8.3: Lateral tracking error and heading error profiles during inter-row autonomous navigation

▶ **DA COMPLETARE: Risultati quantitativi di navigazione ad anello chiuso**

Riportare i dati raccolti durante le prove in campo con il robot Demetra:

- Tabella comparativa tra i 5 algoritmi (Strip-IRLS, EdgeNet, RANSAC edges, Center points, Column histogram) con le metriche definite in Sezione 8.2.2: P_{laser}, N_{exit}, S_{mission}, σ_ω, J_ω e v_{eff}.
- Grafico temporale del segnale del sensore laser sovrapposto alla velocità angolare comandata $\omega(t)$ per ciascun algoritmo.
- Analisi degli eventi critici: corrispondenza tra le anomalie del segnale laser e i frame classificati come `lost` dal path finder.

8.2.5 Qualitative observations

▶ **DA COMPLETARE: Osservazioni qualitative sul comportamento del sistema**

Discutere gli aspetti qualitativi emersi durante i test:

- Differenze visibili nella fluidità di guida tra Strip-IRLS e Center points (oscillazioni laterali, scatti di sterzo).
- Comportamento in presenza di interruzioni (gaps) nella fila di piante e al variare dell'illuminazione.
- Casi limite riscontrati (es. testate del campo, biforcazioni del corridoio, erbacce invadenti).

8.3 Discussion

8.3.1 Segmentation performance

The architecture comparison supports LR-ASPP as an accuracy–speed compromise: on the manual lettuce test images it loses about 1.1 percentage points of mIoU with respect to FCN and DeepLabV3, with much shorter recorded inference times. IoU alone does not establish that a segmentation is sufficient for navigation. This is assessed through the geometric extraction and the robot tests.

The supervision comparison tested labels generated by SAM 3 and used without any manual correction. On lettuce, networks trained on these labels remain below the manual model (path IoU 0.9013 against 0.9587 for the binary network in the realistic scenario) and lose most of their accuracy on the complete test set. Part of this gap may come from a difference in label definition: the manual masks mark only the central traversable corridor as path, whereas networks trained on SAM 3 labels can also segment adjacent or non-traversable corridors, which the IoU counts

as false positives. This behavior was observed visually for the deployed network but was not quantified. The deployed network required no manual annotation of its training data, but the time saved with respect to manual labeling was not measured.

8.3.2 Geometric extraction trade-offs

No extractor is best on all the test sets. The results give the following picture of each geometric extractor:

- **Strip-IRLS** has few failures on the annotated real frames. On the synthetic masks it fails when a hole wider than the merging gap and taller than the seed window splits the bottom of the corridor, and when the merging gap joins a narrow row between adjacent corridors. When the corridor rises from a band of path as wide as the image, it returns `lost` more often than RANSAC edges.
- **RANSAC edges** has few failures on the annotated real frames and rejects the band without a corridor most often. It fails with adjacent corridors, because the row extremes of the selected component can belong to the adjacent corridor.
- **Center points** follows one interval per strip and is accurate with adjacent corridors, but the midpoints formed before the fit carry the disturbances on one side into the centerline, and it fails more often on the annotated frames.
- **Column histogram** is the fastest extractor on the Jetson, but it returns a vertical line without heading, its error grows with the offset and the rotation of the robot, and it has the most failures on the annotated frames.

The variants of Strip-IRLS tuned on the synthetic development masks had fewer synthetic errors, but they were not better on the development part of the annotated real frames. Synthetic test sets written by the same author as the extractors find failure cases, but they can favor changes that do not carry over to real masks, so the choice between versions was made on real annotated frames.

For Strip-IRLS, RANSAC edges and EdgeNet, most gross errors, synthetic and real, are classified as `ok`. A calibrated confidence or a temporal consistency check between frames would give the controller a further protection besides the `lost` status.

8.3.3 EdgeNet: first results

EdgeNet is a first experiment, and the results above are those of the second trained model (Section 6.3). It has no geometric parameters in the extraction. On the development part of the annotated frames it has 4 failures, against 7 for Strip-IRLS

and 6 for RANSAC edges, and on the test part 10, against 4 and 3 (Table 8.3). On the synthetic masks with adjacent corridors and the curve it has no gross errors, where Strip-IRLS has 2.3% and RANSAC edges 18.5%, and its gross errors are in the tomato-shaped masks with the apex below the control row and the centerline near the image border. On the real masks it classifies 89.4% of the frames as `ok`, between Strip-IRLS (87.7%) and RANSAC edges (92.8%). Its accuracy depends on how well its generator covers the masks met in the field (Section 8.1.2).

Adapting the network to a new crop or camera mounting means changing the generator and retraining it, not tuning thresholds in pixels. The design choices are not removed but moved into the generator, which has 16 disturbance probabilities and many hand-chosen ranges, and the status still depends on the thresholds 0.5, 8 px, 0.38 and 0.60. The loss gives double weight to row 156, the row of the evaluation. The geometric extractors also use the same parameters for all three crops, so one model for all crops is not an advantage of the network alone. The network was not tested with another camera or mounting. The second generator was written after the errors of the first model on the real masks had been inspected, and these masks include the sequences of the annotated test frames, although not their annotations.

At this early stage, with one model for the three crops, the network is close to the geometric extractors on the annotated real frames but not better. These first results make learning the extraction from the mask a promising direction. The failures above indicate what further work has to address: the extension of the lines when the apex lies below the control row, a confidence calibrated on real frames instead of the thresholds of Strip-IRLS, and a test on frames recorded after the generator was written.

Chapter 9

Conclusions and Future Work

This chapter summarizes the main findings, discusses the contributions and limitations of the work, and identifies directions for future research.

9.1 Main contributions

This thesis investigated vision-based local guidance for agricultural row navigation through segmentation model selection, dataset preparation with automatic SAM 3 annotation, and geometric reference extraction. The main contributions are:

- **Architecture comparison for robotic segmentation:** FCN, DeepLabV3, and LR-ASPP were compared on manually annotated lettuce images. LR-ASPP was selected for its accuracy–speed compromise: on the test images it reached a path IoU of 0.9532 with a recorded inference time of 8.21 ms, against 59.10 ms and 86.90 ms for the other architectures, at the cost of an mIoU about 1.1 percentage points lower.
- **Automatic dataset annotation with SAM 3:** Training labels were generated with SAM 3 text prompts and used without any manual correction. Networks trained on these labels for lettuce were compared with a network trained on manual masks, and the binary labeling was then extended to the complete lettuce, tomato, and cucumber collection to train the network deployed on the robot.
- **Evaluation of output label configurations:** Networks with four-class, three-class, and binary path outputs, trained on SAM 3 lettuce labels, were compared against the manual ground truth. Their path IoU values differ by less

than 0.5 percentage points in the realistic scenario. The binary configuration was adopted for the robot because the path is the only class used by the geometric extraction.

- **Edge-based geometric extractor and its evaluation:** An extractor that converts corridor masks into image-space navigation references was implemented (Strip-IRLS). It uses horizontal strip extraction, bottom-up interval association with selection of the chain that gives the most edge samples, exclusion of the edge samples clipped by the image border, and separate fits of the left and right edges with iteratively reweighted least squares. It was compared with three baseline extractors (Center points, Column histogram, RANSAC edges) on synthetic masks with exact ground truth, on 1604 real predicted masks of three crops, and on real frames with manually annotated corridor edges.

- **Edge regression network, a first experiment:** A small network that takes the binary mask and regresses the two corridor edges was designed and trained on synthetic masks. It has the same interface as the geometric extractors and was compared with them in the same tests. It is at an early stage.

9.2 Conclusions on segmentation and annotation

The architecture comparison supports LR-ASPP as a compromise between segmentation accuracy and recorded inference time on the robot hardware. Whether its segmentation is sufficient for navigation is assessed through the geometric extraction and the robot tests, not from IoU alone.

On lettuce, the manual model remains the best: path IoU 0.9532 on the complete test set and 0.9587 in the realistic scenario. Networks trained only on uncorrected SAM 3 labels reach a path IoU of about 0.90 in the realistic scenario (0.9013 for the binary network), but only 0.39–0.41 on the complete test set, and the 4-class network reaches an mIoU of 0.5925. Fully automatic labels therefore provide path supervision of about 0.90 IoU in the operational conditions, but they are not a replacement for manual annotation when all classes are needed or when the scene departs from those conditions. The multi-crop binary configuration used on the robot, trained on the complete SAM 3 collection without any manual annotation, reaches 0.9071 in the realistic scenario and 0.6369 on the complete test set when the lettuce test images are excluded from its training data. The deployed weights were assessed visually, in simulation, and on the robot.

The choice of output label configuration affects both annotation effort and downstream geometric extraction. Binary path segmentation focuses the model directly on the navigation target, simplifying both the labeling task and the

geometric stage. This comes at the cost of discarding plant and covering-material information that might be useful for other tasks.

9.3 Conclusions on path extraction

The geometric extraction stage converts binary corridor segmentation masks into navigation setpoints. The offline comparison of Chapter 8 gives the following results:

- On real frames with manually annotated edges, Strip-IRLS and RANSAC edges have the fewest failures, with 2.3% and 2.3% gross errors on the development frames and 2.5% and 0.8% on the test frames. The difference is a few frames and the order changes between the two parts. The network has 3.0% and 6.6%, Center points and Column histogram 5.5 to 18.8%.
- The synthetic masks show a different failure case for each extractor: for Strip-IRLS a hole wider than its merging gap and taller than its seed window, and a narrow row between adjacent corridors, for RANSAC edges adjacent corridors, for Center points disturbances on one side, for Column histogram an offset or rotated robot, and for EdgeNet an apex below the control row.
- Variants of Strip-IRLS tuned on the synthetic development masks were not better on the real annotated frames. Synthetic benchmarks written by the same author as the extractors are useful to find failure cases, but the choice between versions needs real annotated data.
- The parameters of Strip-IRLS were set by hand for the camera, its mounting and the fields of this work, and they have to be set again for a new scenario. EdgeNet needs no geometric parameters, so adapting it means changing its generator instead of tuning thresholds. With one model for three crops it is close to the geometric extractors but not better, and its accuracy depends on how well the generator covers the field masks. At this early stage, these results make learning the extraction from the mask a promising direction.
- For Strip-IRLS, RANSAC edges and EdgeNet the status is a weak indicator of a correct line, because most of their gross errors are classified as `ok`.

No extractor is best on every test set.

▶ DA COMPLETARE: Sintesi quantitativa finale dell'estrazione geometrica e navigazione

Sintetizzare i dati conclusivi:

- Margine di precisione medio dell'asse stimato (scostamento in pixel e in centimetri).
- Tasso medio di affidabilità e di reiezione dei frame lungo i test eseguiti.
- Conferma della sostenibilità del controllo alla frequenza della telecamera (30 fps) su piattaforma embedded Jetson in condizioni di campo.

9.4 Limitations

The main limitations of the work are:

- **Structural limits of Strip-IRLS:** The dependence on the seed in the bottom strips, the thresholds in pixels and the one-sided gaps are described in Section 6.5. In the offline evaluation they appear as the frames lost in `pomodori_maturi1`.
- **Geometric model constraints:** Strip-IRLS, RANSAC edges, Center points and the edge network model the corridor with straight lines. They cannot follow curved crop rows or navigate headland turnaround maneuvers where crop boundaries terminate or diverge.
- **Dataset annotation coverage:** Segmentation accuracy was measured against manual ground truth only on lettuce. The deployed weights have no independent IoU, because their training data include the lettuce test images. The multi-crop configuration was evaluated through a separate network trained without them. Tomato and cucumber have no manual ground truth. The random image-level split also takes training and test images from the same lettuce folders, each recorded along one path. No test image is used for training, but the reported IoU values may be optimistic for paths not seen during training.
- **Label definition mismatch:** The manual masks mark only the central traversable corridor as path, whereas networks trained on SAM 3 labels can also segment adjacent or non-traversable corridors. This lowers their IoU against the manual masks, but the size of the effect was not measured.
- **Quality of the SAM 3 labels:** Visual inspection showed that SAM 3 segmented the plastic mulch film poorly and also assigned the greenhouse plastic cover to the covering class, and that its path masks were less accurate

in images taken from unusual viewpoints or where the path between the rows is not clearly visible. Because the labels were used without correction, these errors enter the training data, and their extent was not measured.

- **Annotation effort measurement:** The time saved by automatic labeling with respect to manual annotation was not measured.
- **Offline test data:** The synthetic masks have straight edges, while the real predictions often have rounded boundaries. In the tomato-based set the ground truth above the apex is a linear extension with no physical meaning. Only 255 real frames have an annotated ground truth, marked once by one person, and the frames skipped or with a nearly horizontal edge, which are among the most difficult, are excluded. The 1604 masks of the SAM 3 dataset come from images used to train the network, so they are probably cleaner than predictions on new images.
- **Parameters of the geometric extractors:** The parameters of Strip-IRLS and of the baselines were chosen for the camera, its mounting and the fields of this work (Section 6.2.6). They have to be set again for another camera, mounting or crop.
- **Confidence of the extractors:** The confidence scores are heuristic and not calibrated, and most gross errors of Strip-IRLS, RANSAC edges and the edge network are classified as `ok`.
- **Closed-loop dynamics:** Offline benchmarks evaluate reference accuracy on static test images. They do not account for vehicle inertia, steering backlash, or dynamic latency variations under closed-loop field control.

9.5 Future work

Several directions for future research emerge from this work:

9.5.1 Dataset and annotation

- Measure the annotation time of manual and automatic workflows, and test whether a light correction of the SAM 3 labels improves the trained networks.
- Split the data by folder rather than by image, so that the test paths are not represented in training, and repeat the evaluation of all networks.
- Prepare manual ground-truth masks for tomato and cucumber to evaluate the deployed network on those crops directly.

- Investigate active learning strategies to select images for manual annotation that improve model generalization most effectively.
- Explore other foundation models or fine-tuned SAM variants specialized for agricultural segmentation.

9.5.2 Segmentation and model deployment

- Evaluate additional lightweight architectures (e.g., BiSeNet, FastSCNN) for further latency reduction.
- Investigate quantization and model compression techniques to enable deployment on edge devices.
- Test temporal consistency methods that use previous frames to improve segmentation and reduce flickering.
- Explore uncertainty estimation to detect when segmentation confidence is low and fallback strategies are needed.

9.5.3 Geometric extraction

- Extend the geometric representation from linear models to piecewise linear or polynomial formulations to support curved rows and headland turns.
- Implement top-down recovery logic to maintain guidance when bottom-up seeding fails, and handle a corridor that rises from a full-width band of path.
- Annotate more real frames, from new recordings and by more than one annotator, so that the differences of a few frames between the best extractors can be resolved and the annotation error can be measured.
- Calibrate the confidence of the extractors, or add a temporal consistency check, so that wrong lines can be rejected before they reach the controller.
- Extend the generator of the edge network to other cameras, mountings and crops, so that a new scenario needs a new training set and not a new tuning of geometric thresholds.
- Evaluate the edge network on annotated frames of new recordings and on the Jetson AGX Orin, and calibrate its status thresholds for its own confidence.
- Adapt spatial gate thresholds dynamically using vehicle attitude telemetry from an onboard IMU to prevent false tracking loss during pitch and roll variations.

- Incorporate temporal Bayesian tracking (such as an Extended Kalman Filter) to smooth reference parameters across frame dropouts.

9.5.4 System integration and field validation

- Complete closed-loop field trials to validate end-to-end navigation performance under variable lighting, growth stages, and row conditions.
- Measure lateral tracking error and success rate over extended distances and multiple crop types.
- Integrate temporal filtering and prediction to handle brief reference loss or occlusions.
- Investigate fusion with other sensors (e.g., depth, IMU) to keep a valid reference when the camera view is degraded.
- Develop a complete autonomy system including global path planning, obstacle avoidance, and task-specific behaviors.

9.6 Closing remarks

This thesis addressed the segmentation, dataset preparation, and geometric extraction stages of vision-based agricultural navigation. The selection of LR-ASPP balances accuracy and speed for robotic deployment. Labels generated by SAM 3 without manual correction were used to train the network deployed on the robot, and the geometric extractor converts masks into navigation references using horizontal strip selection and independent edge fitting.

Future research should complete field validation, extend the geometric model to handle curves and gaps, and evaluate the deployed network on independent, sequence-level test data.

Bibliography

[1] Yuhao Bai, Baohua Zhang, Naimin Xu, Jun Zhou, Jiayou Shi, and Zhihua Diao. «Vision-based navigation and guidance for agricultural autonomous vehicles and robots: A review». In: *Computers and Electronics in Agriculture* 205, 107584 (Feb. 2023). DOI: 10.1016/j.compag.2022.107584 (cit. on pp. 1, 2, 4, 5, 9, 13, 63).

[2] Rajitha de Silva, Grzegorz Cielniak, Gang Wang, and Junfeng Gao. «Deep Learning-Based Crop Row Detection for Infield Navigation of Agri-Robots». In: *Journal of Field Robotics* 41.7 (2024), pp. 2299–2321. DOI: 10.1002/rob.22238 (cit. on pp. 1, 4, 7–10, 13, 15, 16).

[3] Alessandro Navone, Mauro Martini, Marco Ambrosio, Andrea Ostuni, Simone Angarano, and Marcello Chiaberge. «GPS-Free Autonomous Navigation in Cluttered Tree Rows with Deep Semantic Segmentation». In: *Robotics and Autonomous Systems* 183, 104854 (Jan. 2025). DOI: 10.1016/j.robot.2024.104854 (cit. on pp. 1, 2, 4, 7, 8, 10–12, 15, 16, 63).

[4] Enrico Mendez, Javier Piña Camacho, Jesús Arturo Escobedo Cabello, and Alfonso Gómez-Espinosa. «Autonomous Navigation and Crop Row Detection in Vineyards Using Machine Vision with 2D Camera». In: *Automation* 4.4 (Sept. 2023), pp. 309–326. DOI: 10.3390/automation4040018 (cit. on pp. 2, 5, 7–9, 12, 15, 16).

[5] Johannes Kneip, Patrick Fleischmann, and Karsten Berns. «Crop Edge Detection Based on Stereo Vision». In: *Robotics and Autonomous Systems* 123, 103323 (Jan. 2020). DOI: 10.1016/j.robot.2019.103323 (cit. on pp. 4, 7, 12, 14–16).

[6] Xia Li, Junhao Su, Zhenchao Yue, and Fangtao Duan. «Adaptive Multi-ROI Agricultural Robot Navigation Line Extraction Based on Image Semantic Segmentation». In: *Sensors* 22.20, 7707 (Oct. 2022). DOI: 10.3390/s22207707 (cit. on pp. 5, 7, 10, 12, 13, 15, 17).

[7] Jiqing Chen, Hu Qiang, Jiahua Wu, Guanwen Xu, and Zhikui Wang. «Navigation Path Extraction for Greenhouse Cucumber-Picking Robots Using the Prediction-Point Hough Transform». In: *Computers and Electronics in Agriculture* 180, 105911 (Jan. 2021). DOI: 10.1016/j.compag.2020.105911 (cit. on pp. 5, 7, 12, 15, 16).

[8] Alessandro Navone, Mauro Martini, Andrea Ostuni, Simone Angarano, and Marcello Chiaberge. «Autonomous Navigation in Rows of Trees and High Crops with Deep Semantic Segmentation». In: *2023 European Conference on Mobile Robots (ECMR)*. IEEE, 2023, pp. 1–6. DOI: 10.1109/ECMR59166.2023.10256334 (cit. on p. 10).

[9] Jonathan Long, Evan Shelhamer, and Trevor Darrell. «Fully Convolutional Networks for Semantic Segmentation». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 3431–3440. DOI: 10.1109/CVPR.2015.7298965 (cit. on pp. 10, 18–22).

[10] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. «Rethinking Atrous Convolution for Semantic Image Segmentation». In: *arXiv preprint arXiv:1706.05587* (2017) (cit. on pp. 10, 20, 22–24).

[11] Andrew Howard et al. «Searching for MobileNetV3». In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, pp. 1314–1324 (cit. on pp. 10, 24–27, 29).

[12] Alexander Kirillov et al. «Segment Anything». In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2023, pp. 4015–4026. arXiv: 2304.02643 [cs.CV] (cit. on pp. 11, 30, 31).

[13] Nicolas Carion et al. «SAM 3: Segment Anything with Concepts». In: *International Conference on Learning Representations (ICLR)*. 2026. arXiv: 2511.16719 [cs.CV]. URL: https://proceedings.iclr.cc/paper_files/paper/2026/hash/e0982cbc81401df3430ee1ff780dc7a2-Abstract-Conference.html (cit. on pp. 11, 30, 31).

[14] Letian Wu, Yongzhi Cui, Huifeng Shi, Xiaoli Sun, Jiayan Yang, Xinwei Cao, Ping Zou, and Ya Liu. «An Improved DeepLabV3+-Based Method for Crop Row Segmentation and Navigation Line Extraction in Agricultural Fields». In: *Sensors* 26.10, 3142 (May 2026). DOI: 10.3390/s26103142 (cit. on pp. 13, 63).

[15] Martin A. Fischler and Robert C. Bolles. «Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography». In: *Communications of the ACM* 24.6 (June 1981), pp. 381–395. DOI: 10.1145/358669.358692 (cit. on pp. 13, 63).

[16] Matthias Müller, Alexey Dosovitskiy, Bernard Ghanem, and Vladlen Koltun. «Driving Policy Transfer via Modularity and Abstraction». In: *Proceedings of the 2nd Conference on Robot Learning (CoRL)*. Zürich, Switzerland, 2018. arXiv: 1804.09364 (cit. on pp. 14, 60).

[17] Arun Narenthiran Sivakumar, Sahil Modi, Mateus Valverde Gasparino, Che Ellis, Andres Eduardo Baquero Velasquez, Girish Chowdhary, and Saurabh Gupta. «Learned Visual Navigation for Under-Canopy Agricultural Robots». In: *Robotics: Science and Systems (RSS)*. 2021. URL: https://www.roboticsproceedings.org/rss17/p019.pdf (cit. on p. 14).

[18] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn and PyTorch: Concepts, Tools, and Techniques to Build Intelligent Systems*. 1st ed. O'Reilly Media, 2026. ISBN: 979-8-341-60798-9 (cit. on pp. 18–21, 23, 39, 40).

[19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. «Deep Residual Learning for Image Recognition». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90 (cit. on p. 20).

[20] TorchVision Maintainers and Contributors. *TorchVision: Models and Pre-trained Weights — FCN-ResNet50*. URL: https://pytorch.org/vision/stable/models/fcn.html (visited on 10/23/2026) (cit. on pp. 22, 38).

[21] TorchVision Maintainers and Contributors. *TorchVision: DeepLabV3 segmentation models, source code torchvision/models/segmentation/deeplabu3.py*. URL: https://github.com/pytorch/vision/blob/main/torchvision/models/segmentation/deeplabv3.py (visited on 10/23/2026) (cit. on pp. 24, 38).

[22] TorchVision Maintainers and Contributors. *TorchVision: LR-ASPP segmentation model, source code torchvision/models/segmentation/lraspp.py*. URL: https://github.com/pytorch/vision/blob/main/torchvision/models/segmentation/lraspp.py (visited on 10/23/2026) (cit. on pp. 27, 38).

[23] Devansh Arpit et al. «A Closer Look at Memorization in Deep Networks». In: *Proceedings of the 34th International Conference on Machine Learning*. Ed. by Doina Precup and Yee Whye Teh. Vol. 70. Proceedings of Machine Learning Research. 2017, pp. 233–242. URL: https://proceedings.mlr.press/v70/arpit17a.html (cit. on p. 42).

[24] Philip H. S. Torr and Andrew Zisserman. «MLESAC: A New Robust Estimator with Application to Estimating Image Geometry». In: *Computer Vision and Image Understanding* 78.1 (2000), pp. 138–156. DOI: 10.1006/cviu.1999.0832 (cit. on p. 89).